

ECSE 2920: Design Methodology

Technical Specification and Description Report

Group 5

Table of Contents

Introduction **5**

Objective **7**

Constraints **8**

System Overview **11**

Hardware Design **13**

Power Supply *13*

Input and Voltage Division 17

Op-Amp Control Stage 17

MOSFET Output Stage 18

Voltage Clamping and Protection 18

Filtering and Stability 19

Generated Voltage Rails 19

Derivation of 5V and 3.3V Rails 20

Power Distribution 20

Summary 21

Ohmmeter *21*

Measurement Principle 23

Divider Stage 23

Op-Amp Buffer Stage 24

Comparator Stage 24

Reference Voltage Generation 25

Successive Approximation Method 25

Digital Interface 26

Operating Range and Accuracy 26

Summary 26

Voltmeter *27*

Measurement Principle 30

Input Scaling Stage 31

Op-Amp Buffer Stage 31

Comparator Stage 32

Reference Voltage Generation 32

Successive Approximation Method 32

Digital Interface 33

Operating Range and Accuracy 33

Summary 33

DC Voltage Reference *34*

Stage 1: Reference Generation 37

Stage 2: Gain Stage 37
Stage 3: Level Shifting 38
Stage 4: Output Stage 38
Design Considerations 39
Summary 39

Signal Generation (Square + Sine) *39*
Square Wave Generation 39
Sine Wave Generation 43

Frequency Measurement *48*
Stage 1: Signal Conditioning 51
Stage 2: Edge Detection 51
Stage 3: Frequency Calculation 51
Design Considerations 52
Summary 52

Software Design 52

Software Architecture Overview *53*

System Setup *56*
GitHub Repository Setup 56
Creating SSH Key 57
Git Commands 58
Raspberry Pi 4 Setup and Initial Configuration 61

pigpio Library Functions *62*
Rotary Encoder 64
SPI 65
I2C 66

Software – Operation *67*

Driver Program *67*
State Dictionary 69
Running Measurements and Outputs on Menu 70
Updating Measurements 73
Running the Function Generator 74
Running DC Voltage Output 74

Callbacks Helper File *75*
Drawing the Menu 75
Picking Options Menu 77
Rotary Encoder 78

LCD *85*

Successive Approximation Method *89*
Software Implementation 91

Ohmmeter *93*

Voltmeter 97
External DC Reference 99
Internal DC Reference 99
Square Wave Generator 101
Sine Wave Builder 108
Converting Amplitude to Step 109
Building the Sine Wave 114
Sine Wave Frequency Measurement 118
Obtaining Voltages on Raspberry Pi 120
PCB Design and Considerations 120
Ethics 124
Design Integrity and Original Work 124
Ethical Considerations for Real-World Application 125
Self-Critique 126
Revision Plan 127
Individual Reflections 129
George Braun 129
Tillman Clark 130
Kai DeVito 131
Logan Griffy 132
Andrew Primiano 133
Appendix 134
Appendix A: Software 134
Appendix B: Power Supply 135
Appendix C: Ohmmeter 135
Appendix D: Voltmeter 135
Appendix E: DC Voltage Reference 136
Appendix F: Square Wave Generator 136
Appendix G: Sine Wave Generator 136
Appendix H: Frequency Measurement 136

Introduction

The Electronics Tool Bench is a fully integrated system designed to perform both electrical measurement and signal generation functions within a single, compact platform. The primary goal of this system is to replicate the functionality of traditional laboratory equipment, such as multimeters, function generators, and DC power supplies, while reducing the need for multiple standalone devices. By consolidating these tools into one system, the Electronics Tool Bench provides a portable, efficient, and cost-effective solution for testing and analyzing electronic circuits.

The system consists of several key subsystems, including an ohmmeter, voltmeter, square wave generator, sine wave generator, frequency measurement module, and DC voltage reference. Each subsystem is designed to meet specific performance requirements related to voltage range, frequency range, resolution, and accuracy. These subsystems are not independent devices, but instead operate as part of a unified system that shares power, control logic, and a common user interface.

At the center of the system is the Raspberry Pi 4, which serves as the primary controller for all operations. The Raspberry Pi is responsible for executing software routines, managing system states, processing user inputs, and communicating with hardware components. Communication between the Raspberry Pi and external hardware is achieved through standard interfaces, including General Purpose Input/Output (GPIO), Serial Peripheral Interface (SPI), and Inter-Integrated Circuit (I2C) protocols. These interfaces allow the system to control digital potentiometers, read comparator outputs, drive display hardware, and generate pulse-width modulated (PWM) signals.

A defining characteristic of the Electronics Tool Bench is the integration of analog and digital subsystems. Analog circuitry is used to perform signal conditioning, amplification, filtering, and waveform shaping. Components such as operational amplifiers, resistive networks, capacitors, and comparators are used to manipulate electrical signals in continuous form. In contrast, digital components such as the Raspberry Pi and digital potentiometers provide discrete control and decision-making capabilities. This combination enables the system to approximate analog values using digital algorithms, reducing hardware complexity while maintaining acceptable accuracy.

One of the primary techniques used in the measurement subsystems is successive approximation. In this approach, the system iteratively adjusts a reference value using a digital potentiometer and compares it to an unknown input using a comparator. Based on the comparator output, the system converges on an approximate value for the measured quantity. This method is used in both the ohmmeter and voltmeter subsystems, allowing the system to perform measurements without requiring high-precision analog-to-digital converters.

In addition to measurement capabilities, the system includes signal generation features. The square wave generator uses pulse-width modulation (PWM) generated by the Raspberry Pi, combined with analog amplification and signal conditioning, to produce a waveform with adjustable frequency and amplitude. The sine wave generator is implemented using a combination of PWM-based signal generation and analog filtering and amplification stages to produce a smooth sinusoidal waveform within the required operating range. These signal generation subsystems are designed to operate under high impedance conditions, ensuring that they do not supply significant current to external loads.

The system also includes a frequency measurement feature, which allows it to determine the frequency of an input sine wave. This feature is integrated into the overall system and provides real-time feedback to the user through the display interface. By combining signal generation and measurement capabilities, the Electronics Tool Bench provides a complete environment for both producing and analyzing electrical signals.

User interaction is handled through an LCD display and a rotary encoder. The LCD display provides real-time feedback, including measured values, selected parameters, and system status. The rotary encoder allows the user to navigate menus, adjust values, and select operating modes. The interface is designed to be consistent across all subsystems, ensuring that users can easily transition between different functionalities without confusion.

The system is designed with modularity and integration in mind. Each subsystem is developed and tested individually before being incorporated into the full system. This approach simplifies debugging and ensures that each component meets its requirements before system-level integration. Once integrated, all subsystems operate together under a unified control structure, allowing the Electronics Tool Bench to function as a cohesive and reliable system.

The purpose of this technical document is to describe the final design, implementation, and performance of the Electronics Tool Bench. The focus is on the completed system, including hardware architecture, subsystem design, and testing results. This document is intended to provide a clear and detailed explanation of how the system operates and how it meets all specified requirements.

Objective

The objective of this project is to design, implement, and validate a fully integrated Electronics Tool Bench that combines multiple electrical measurement and signal generation functions into a single, cohesive system. The system must meet all specified performance requirements while maintaining reliability, accuracy, and ease of use.

The Electronics Tool Bench is required to perform resistance measurement using an ohmmeter within a range of 500Ω to 10kΩ. The system must determine the value of an unknown resistance and report the measured value along with an associated tolerance that reflects the resolution and limitations of the design. This measurement must be performed using a combination of analog circuitry and digital control.

The system must also perform voltage measurement using a voltmeter capable of measuring DC voltages within a range of ±5V. The measured value must be displayed to the user along with a corresponding tolerance. The voltmeter must be capable of measuring both externally applied voltages and internally generated reference voltages, allowing for system verification and calibration.

In addition to measurement capabilities, the system must generate electrical signals. The square wave generator must produce a waveform with a frequency range from 100Hz to 10kHz, a duty cycle of 50 percent, and an amplitude adjustable up to ±10V. The waveform must be centered around 0V and must maintain consistent performance across the full operating range.

The system must also generate a sine wave with a voltage range from 0V to 10V and a frequency range from 1kHz to 10kHz. The amplitude of the sine wave must be adjustable in increments of 0.625V, and the waveform must appear sinusoidal when verified using an oscilloscope. The sine

wave generator must operate under high impedance conditions and must not provide significant current to external loads.

A frequency measurement feature must be implemented to determine the frequency of an input sine wave within the required operating range. The system must accurately detect and report the frequency of the input signal, providing real-time feedback to the user through the display interface.

The system must also include a DC voltage reference capable of producing an output within ±5V in increments of 0.625V. This reference must operate with high impedance and must not drive significant current. The output must be controllable through the user interface and must be measurable using the system's voltmeter.

All subsystems must be controlled using a Raspberry Pi 4, which serves as the central processing unit for the system. The Raspberry Pi must manage all control logic, communication, and user interaction. Communication with hardware components must be implemented using GPIO, SPI, and I2C interfaces.

The system must include a user interface consisting of an LCD display and a rotary encoder. The interface must allow the user to navigate through system modes, adjust parameters, and view real-time measurement and output values. The interface must follow the required menu structure and must provide consistent operation across all subsystems.

Finally, the system must demonstrate full integration of all subsystems into a single platform. Each subsystem must function correctly both independently and as part of the complete system. The final implementation must meet all design constraints and performance requirements while maintaining stable and reliable operation.

Constraints

The design of the Electronics Tool Bench is constrained by both hardware availability and system-level requirements that define how the final system must be implemented. These constraints influence circuit design, component selection, and overall system architecture.

One of the most significant constraints is the limited availability of specific critical components provided in the project kit. Only these components may be used for active circuitry, including comparators, MOSFETs, and digital potentiometers. While many passive components and supporting elements are readily available, certain active components are limited in quantity and must be carefully allocated across all subsystems.

The major constrained components that influenced the design are summarized in Table 1.

Part	Quantity
Quad Comparator (LM339)	2
P Channel Mosfet (IRF4905)	1
N Channel Mosfet (IRFZ34)	1
Dual Digipot (MCP4231)	1
Single Digipot (MCP4131)	2

Table 1. Available quantities of key active components used in the Electronics Tool Bench design.

These component constraints, particularly for devices such as digital potentiometers, comparators, and MOSFETs, required design decisions that prioritized efficiency and simplicity. These components are essential to multiple subsystems and cannot be easily substituted, making their allocation a key factor in the overall system design. Circuits were designed to reuse components where possible and avoid unnecessary duplication of functionality.

Another critical constraint is the requirement to derive all necessary voltage levels from a single 24V power source. The system must generate stable ±12V rails to support analog circuitry, including operational amplifiers and signal conditioning stages. This requires a reliable power supply design capable of maintaining consistent output under both no-load and load conditions. The Raspberry Pi operates independently and cannot be used to power external circuitry due to current limitations.

The system is also constrained by the requirement to use the Raspberry Pi 4 as the sole controller. All system functionality, including measurement logic, signal generation control, and user interface operation, must be implemented in Python. Communication with external components must be performed using GPIO, SPI, and I2C interfaces. This constraint requires careful management of available pins and communication resources to ensure all subsystems can operate simultaneously without conflict.

Performance constraints are also a major factor in the design. The ohmmeter must measure resistance within a range of 500Ω to 10kΩ, while the voltmeter must measure voltages within ±5V. The DC voltage reference must provide an output within ±5V in increments of 0.625V and must operate under high impedance conditions, meaning it cannot supply significant current.

The signal generation subsystems must also meet specific requirements. The square wave generator must produce a waveform with a frequency range from 100Hz to 10kHz, a duty cycle of 50 percent, and an amplitude adjustable up to ±10V. The waveform must be centered around 0V and maintain consistent performance across the full operating range. The sine wave generator must produce a waveform within a range of 0V to 10V, with a frequency range from 1kHz to 10kHz and amplitude adjustable in increments of 0.625V. The waveform must appear sinusoidal when verified using an oscilloscope and must operate under high impedance conditions.

An additional system requirement is the implementation of a frequency measurement feature. The system must be capable of measuring the frequency of an input sine wave within the specified operating range and displaying the result through the user interface.

The user interface is constrained by a predefined structure and must be controlled exclusively using a rotary encoder and LCD display. The interface must allow navigation between system modes, adjustment of parameters, and display of real-time values in a consistent and organized manner.

Finally, the system must be designed with proper integration and stability in mind. All subsystems must operate together without interference, requiring careful consideration of grounding, signal routing, and power distribution. The final system must maintain stable operation and produce reliable, repeatable results under all operating conditions.

These constraints define the boundaries of the design and guide the implementation of the Electronics Tool Bench.

System Overview

The Electronics Tool Bench is designed as a fully integrated system that combines multiple measurement and signal generation subsystems into a single platform. The system is centered around a Raspberry Pi 4, which coordinates all hardware interactions and user interface operations.

At a high level, the system consists of three main components: the control system, the analog hardware subsystems, and the user interface. These components work together to perform measurement, signal generation, and real-time data display.

The Raspberry Pi 4 serves as the central controller, executing control logic, processing user inputs, and managing communication with external hardware. Communication is achieved through GPIO for digital input and output, SPI for controlling digital potentiometers, and I2C for interfacing with the LCD display.

The analog hardware subsystem includes circuits responsible for signal generation and measurement. These circuits use operational amplifiers, digital potentiometers, comparators, and passive components for voltage scaling, waveform generation, and signal conditioning. The ohmmeter and voltmeter use comparator-based designs with digital potentiometers to approximate analog values through successive approximation.

Signal generation is achieved using both square and sine wave generators. The square wave is produced using PWM from the Raspberry Pi and then conditioned and amplified, while the sine wave is generated through PWM-based shaping followed by filtering and amplification. A frequency measurement subsystem is also included, allowing the system to determine the frequency of an input signal and display the result to the user.

The system also includes a DC voltage reference that provides an adjustable output under high impedance conditions. User interaction is handled through an LCD display and rotary encoder, allowing navigation, parameter adjustment, and real-time feedback in a consistent interface.

All analog subsystems are powered by regulated ±12V rails derived from a 24V input source. Additional 5V and 3.3V rails are generated from the 12V supply for digital components. This power architecture ensures stable operation while supporting both analog and digital circuitry.

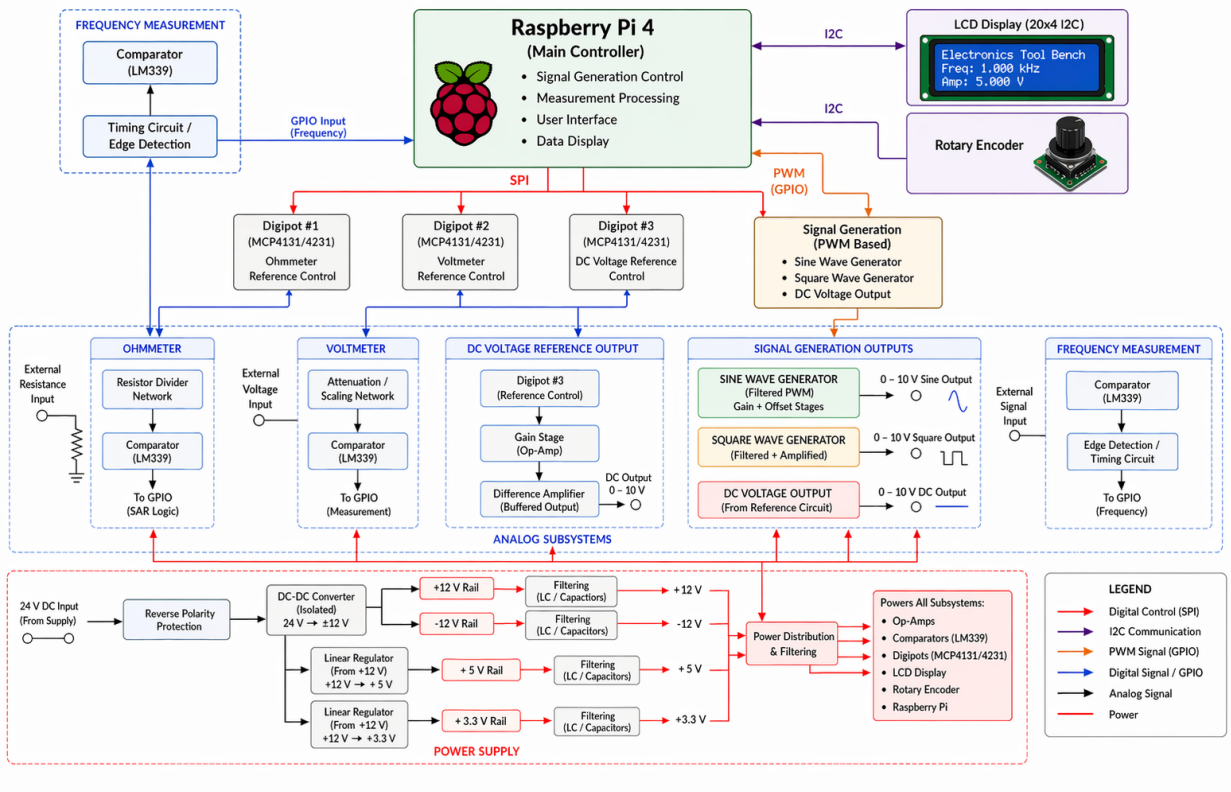


Figure 1. High-level system architecture of the Electronics Tool Bench showing the interaction between the Raspberry Pi, analog subsystems, user interface, and power distribution.

Hardware Design

Power Supply

The power supply subsystem is responsible for converting a single 24V DC input into all voltage rails required for operation of the Electronics Tool Bench. This includes the ±12V rails used for analog circuitry and the additional 5V and 3.3V rails used for digital and control circuitry. Because every subsystem in the Electronics Tool Bench depends on a stable supply, the power supply must provide regulation, filtering, and distribution while minimizing noise and maintaining reliable operation under load.

The final power supply design is shown in Figure 2. The first stage generates the ±12V rails from the 24V input, and the second stage derives 5V and 3.3V from the 12V supply for lower-voltage digital components. The physical implementations of both stages are shown in Figure 3 and Figure 4. The first physical implementation corresponds to the ±12V generation stage using the op-amp and MOSFET configuration, while the second physical implementation corresponds to the secondary regulation stage used to derive the 5V and 3.3V outputs.

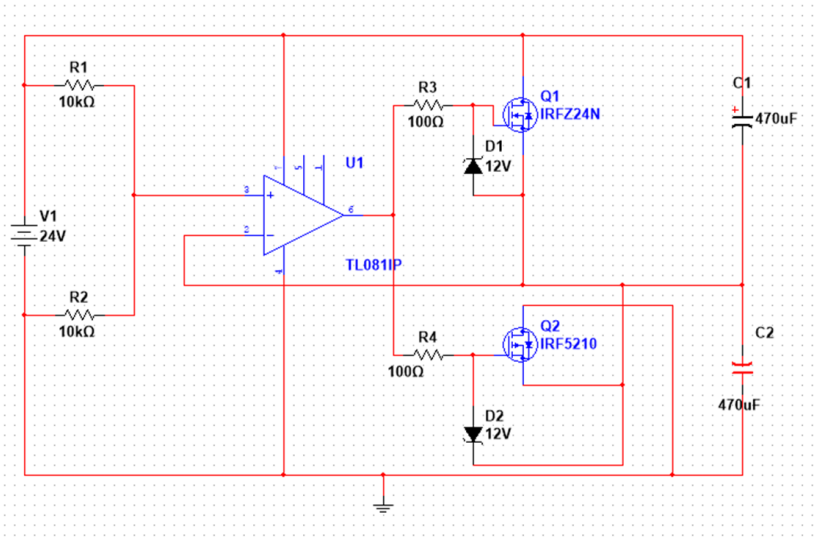


Figure 2. Final Multisim implementation of the power supply used to generate ±12V rails from a 24V input.

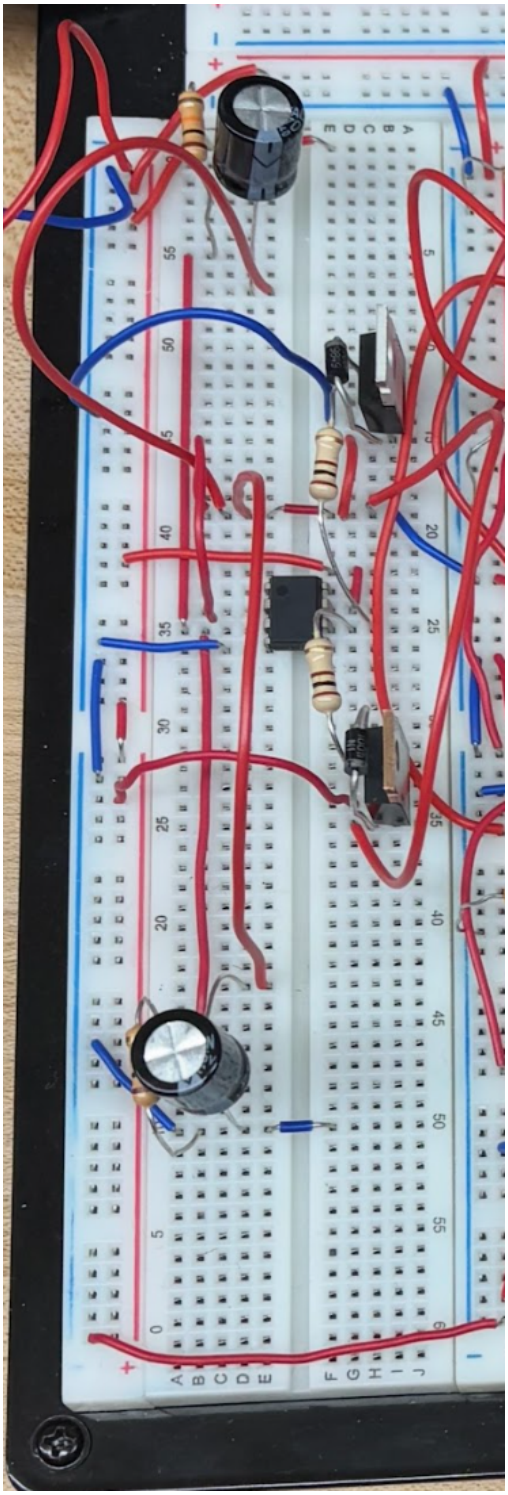


Figure 3. Physical implementation of the power supply stage used to convert a 24V input into regulated ±12V rails using an op-amp controlled MOSFET configuration with filtering capacitors.

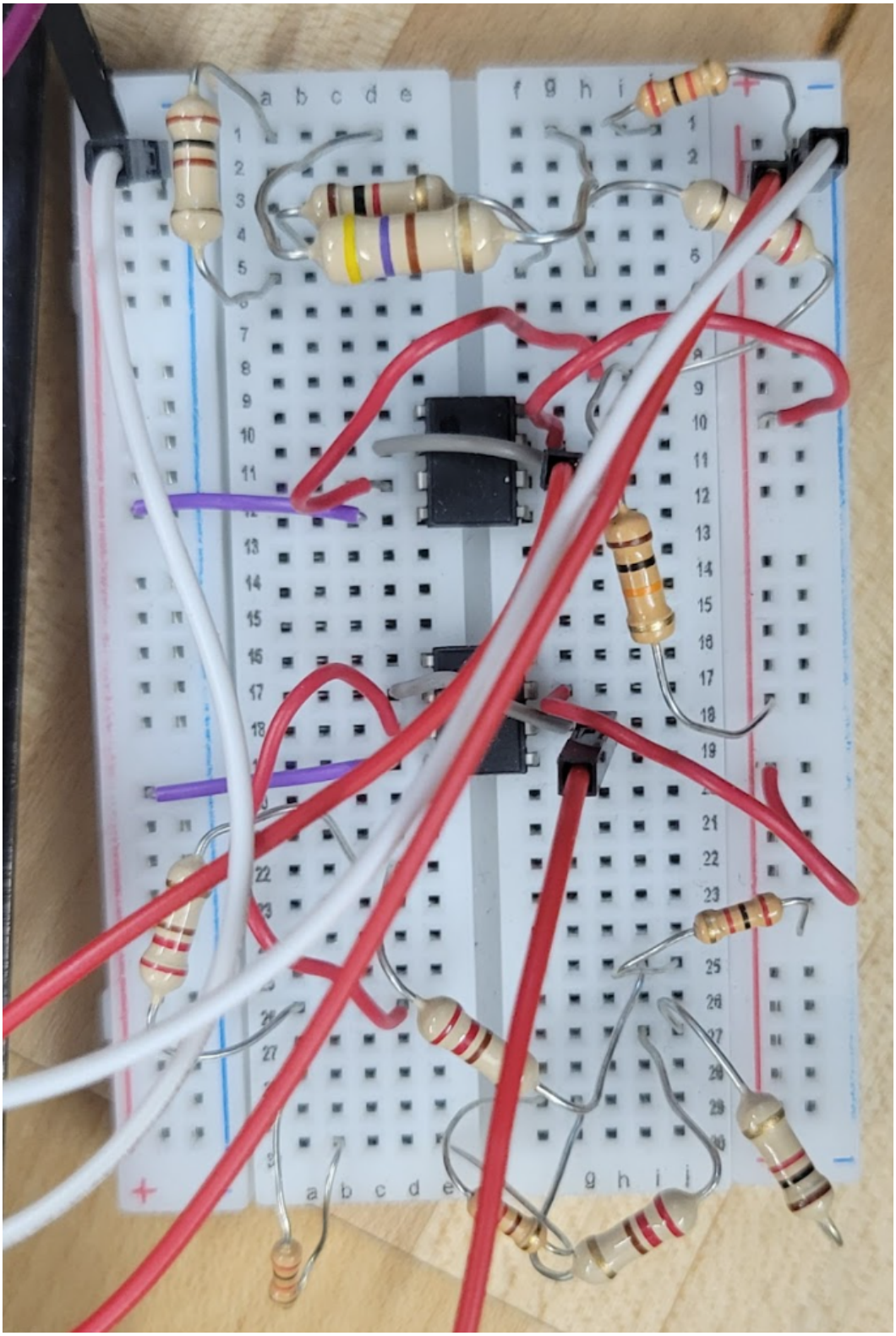


Figure 4. Physical implementation of the secondary regulation stage used to derive stable 5V and 3.3V outputs from the 12V supply for digital components and control circuitry.

To better organize the operation of the power supply, Table 2 summarizes the major stages of the design.

Stage	Main Components	Function	Input	Output
1: Input Division	V1, R1, R2	Creates the midpoint reference from the 24V input	24V DC	~12V reference
2: Control Stage	U1 (TL081)	Regulates and stabilizes the midpoint through feedback	Midpoint reference	Gate drive control
3: Output Stage	Q1, Q2, R3, R4	Generates the positive and negative rails using MOSFETs	Op-amp output	±12V rails
4: Protection Stage	D1, D2	Clamps MOSFET gates voltages	Gate drive nodes	Protected gate swing
5: Filtering Stage	C1, C2	Reduces ripple and improves stability	Raw rail outputs	Filtered ±12V rails
6: Secondary Regulation	Additional resistor and op-amp stages	Derives lower-voltage rails from the 12V supply	+12V rail	+5V and +3.3V

Table 2. Summary of the major stages of the power supply design, including component groups, functional roles, and resulting voltage outputs.

Input and Voltage Division

The system begins with a 24V DC input source. This input is the only primary supply for the hardware subsystem, so all required voltages must be derived from it. The first step in the power supply is a resistor divider formed by R1 and R2, each with a value of 10 kΩ. Because these resistors are equal, the divider produces a midpoint voltage of approximately 12V. This midpoint acts as the reference used to split the 24V input into symmetrical positive and negative rails.

The purpose of this stage is to establish a stable reference around which the analog supply can be centered. Instead of using two separate supplies, the circuit creates the equivalent of a positive and negative rail from a single input by regulating around this midpoint. This is essential for the rest of the Electronics Tool Bench because many analog circuits, especially op-amp based signal conditioning stages, require both positive and negative supply voltages to accurately process bipolar signals.

Op-Amp Control Stage

The midpoint reference created by R1 and R2 is fed into the non-inverting input of the TL081 operational amplifier, U1. This op-amp serves as the control element of the power supply. Its role is to monitor the midpoint condition of the circuit and drive the MOSFET output stage so that the positive and negative rails remain balanced.

The TL081 is used in a feedback configuration. The op-amp compares the divider reference with the output node and adjusts its output voltage so that the two sides of the supply remain symmetric. In other words, the op-amp continuously corrects for any imbalance by changing the drive sent to the MOSFETs. This makes the supply much more stable than a purely passive rail-splitting approach and allows it to better support the varying loads introduced by the analog subsystems.

This stage is especially important because the Electronics Tool Bench includes measurement and waveform generation circuits that depend on clean and balanced supply rails. If the midpoint drifts, then the effective +12V and -12V rails would no longer be symmetric, which would introduce offset errors and reduce output range in later analog stages.

MOSFET Output Stage

The output stage consists of two MOSFETs arranged in a complementary push-pull configuration. Q1 is the N-channel MOSFET and Q2 is the P-channel MOSFET. These two devices are controlled by the output of the TL081 through gate resistors R3 and R4, each 100 Ω. Together, they act as the active devices that create the regulated rail outputs.

The N-channel MOSFET supports the positive rail, while the P-channel MOSFET supports the negative rail. When the op-amp output changes, it alters the gate voltages of the MOSFETs and therefore controls how strongly each device conducts. This push-pull arrangement allows the circuit to source and sink current as needed to maintain the desired rail balance.

This active output stage is the part of the power supply that actually provides the current drive capability for the rest of the system. The resistor divider and op-amp establish the reference and control law, but the MOSFETs provide the power handling needed to support real circuit loads. This is why the output stage is essential for translating the theoretical midpoint regulation into a practical supply that can power op-amps, comparators, and conditioning circuits across the Electronics Tool Bench.

Voltage Clamping and Protection

To protect the MOSFET gates, the circuit includes two 12V Zener diodes, D1 and D2. These diodes clamp the gate voltages so that the gate-to-source voltage of each MOSFET does not exceed safe limits. Without this protection, the op-amp could potentially drive the MOSFET gates beyond their acceptable range, damaging the transistors or causing unstable behavior.

R3 and R4 also contribute to protection and stability by limiting current into the MOSFET gates. Since MOSFET gates behave capacitively, sudden voltage changes can produce sharp current spikes. The series gate resistors reduce this effect and help prevent oscillation or excessive switching transients.

This protection stage improves the durability of the design and makes the power supply more robust in real physical implementation. Since the Electronics Tool Bench was built and tested on

breadboards before full integration, protecting the active devices from accidental overdrive was important for maintaining reliable operation.

Filtering and Stability

The rail outputs are filtered using two large electrolytic capacitors, C1 and C2, each 470 µF. These capacitors are connected across the output rails and serve as local energy reservoirs. Their purpose is to reduce ripple, smooth transient changes, and improve load stability.

When other parts of the system draw current, the instantaneous demand can cause temporary dips or fluctuations in the supply. The capacitors help absorb these changes by storing charge and releasing it when needed. This prevents the voltage rails from sagging or becoming noisy during operation.

Filtering is especially important in this project because the Electronics Tool Bench contains analog measurement and waveform generation circuits that are sensitive to noise. Any instability on the supply rails can directly affect comparator thresholds, op-amp offsets, waveform shape, and measured values. By adding large capacitors at the rail outputs, the design improves the consistency and repeatability of all downstream subsystems.

Generated Voltage Rails

The primary outputs of this stage are the +12V and -12V rails. These rails form the main analog supply of the Electronics Tool Bench. They are used to power operational amplifiers, comparators, and other analog conditioning stages throughout the hardware design.

The +12V and -12V rails are especially important for circuits that must process signals centered around 0V or handle both positive and negative voltage excursions. This includes signal generation stages, measurement front ends, and gain stages used elsewhere in the project. By generating true bipolar rails, the power supply allows those circuits to operate without artificial limitations caused by single-supply biasing.

Derivation of 5V and 3.3V Rails

In addition to the ±12V rails, the system also requires lower-voltage rails for digital and control components. These include a 5V rail and a 3.3V rail. As shown by the second physical implementation, these voltages are derived from the 12V supply using secondary regulation stages built from resistor networks and op-amp based buffering.

The 5V rail is used where a stable reference or low-voltage subsystem supply is needed. The 3.3V rail is particularly important because it matches the logic level used by the Raspberry Pi and supporting digital components. These voltages must be kept stable because any variation would directly affect GPIO compatibility, comparator pull-up levels, and the performance of digital potentiometers and control circuitry.

Even though these lower-voltage rails do not supply the same analog load as the ±12V rails, they are still critical to system operation. The Electronics Tool Bench relies on both analog and digital subsystems working together, so the lower-voltage rails must remain clean and well regulated.

Power Distribution

Once the voltage rails are generated, they are distributed to the rest of the Electronics Tool Bench through a centralized power distribution network. This network routes the correct rail to each subsystem while maintaining separation between analog and digital supply domains where needed.

The analog subsystems primarily rely on the ±12V rails, while the digital and interface circuitry use the 5V and 3.3V rails. By keeping these functions separated, the design reduces the chance that digital switching noise will interfere with sensitive analog measurements. Additional filtering is included as needed to improve stability and reduce coupling between subsystems.

This distribution strategy is important because the Electronics Tool Bench contains several different functions operating at the same time, including signal generation, voltage measurement, resistance measurement, and user interface control. Stable and organized rail distribution ensures that one subsystem does not negatively affect another.

Summary

The power supply subsystem converts a single 24V DC input into all voltage rails required by the Electronics Tool Bench. The first stage uses a resistor divider, TL081 control stage, complementary MOSFET output stage, Zener protection, and large filtering capacitors to generate stable ±12V rails. A secondary regulation stage then derives the 5V and 3.3V rails needed for digital and control circuitry.

This design allows the Electronics Tool Bench to support both analog and digital subsystems from a single power source while maintaining stability, symmetry, and noise reduction. Because all later circuits depend on these rails, the power supply serves as the foundation of the complete system.

Ohmmeter

The ohmmeter subsystem is designed to measure an unknown resistance within the range of 500 Ω to 10 kΩ by converting the resistance into a corresponding voltage and determining its value using a comparator-based successive approximation method. The design integrates analog circuitry with digital control from the Raspberry Pi, allowing resistance measurement without the use of a dedicated analog-to-digital converter. Due to the simplicity of the hardware, which primarily consists of a resistor divider, buffer stage, and comparator, the Multisim design fully captures the functionality of the subsystem and clearly illustrates the signal flow and measurement process.

The final implementation of the ohmmeter is shown in Figure 5.

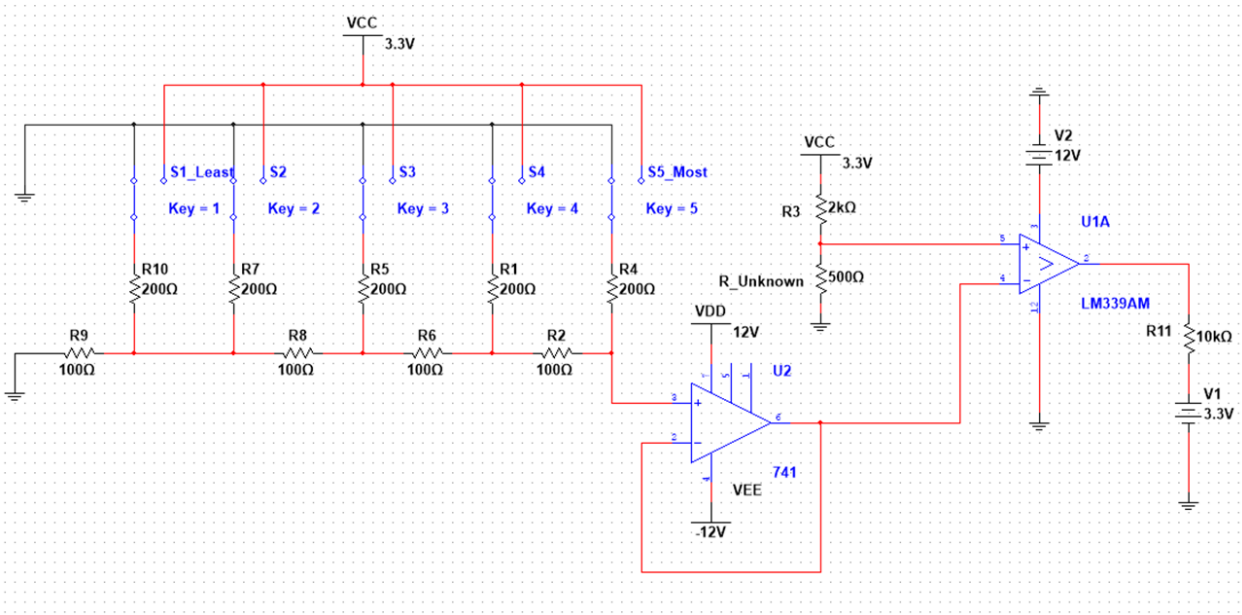


Figure 5. Final Multisim implementation of the ohmmeter subsystem showing the resistor network, op-amp conditioning stage, and comparator interface.

To better understand the operation of the ohmmeter, Table 3 summarizes the major functional stages of the circuit.

Stage	Main Components	Function	Input	Output
1: Divider Stage	Unknown resistor, reference resistor network	Converts resistance into a measurable voltage	Unknown resistance	Analog Voltage
2: Buffer Stage	TL081 op-amp	Stabilizes and isolates the signal	Divider voltage	Buffered voltage

3: Comparator Stage	LM339 Comparator	Compares measured voltage to reference	Buffered voltage, reference voltage	Digital HIGH/LOW
4: Reference Control	Digital Potentiometer (MCP4131/4231)	Generates adjustable comparison voltage	SPI control from Pi	Reference voltage
5: SAR Logic	Raspberry Pi GPIO + software	Determines resistance iteratively	Comparator output	Final resistance value

Table 3. Summary of the ohmmeter subsystem stages, showing the conversion of an unknown resistance into a digital measurement using comparator-based successive approximation.

Measurement Principle

The ohmmeter operates based on the principle of voltage division. An unknown resistance is placed in series with a known reference resistor, forming a voltage divider. The voltage at the midpoint of this divider is directly related to the value of the unknown resistance.

If the unknown resistance increases, the midpoint voltage shifts accordingly. By measuring this voltage, the system can determine the resistance value. Instead of directly measuring the voltage with an ADC, the system compares it against a controllable reference voltage and determines the resistance through an iterative approximation process.

Divider Stage

The divider stage consists of the unknown resistor connected in series with a known reference resistor. This configuration converts the resistance into a proportional voltage that can be measured by the rest of the circuit.

The voltage at the divider node is given by:

$$V_{div} = V_{supply} \cdot \frac{R_{unknown}}{R_{unknown} + R_{reference}}$$

This relationship forms the foundation of the measurement system. The goal of the remaining circuit is to determine this voltage and map it back to the corresponding resistance value.

The selection of the reference resistor is important because it determines the measurement sensitivity across the desired range. The design is optimized for the specified 500 Ω to 10 kΩ range to provide reasonable resolution across all values.

Op-Amp Buffer Stage

The output of the divider is connected to a TL081 operational amplifier configured as a voltage follower. This stage provides a high input impedance and low output impedance, ensuring that the divider voltage is not affected by loading from later stages.

Without this buffer, the comparator or digital potentiometer could draw current from the divider node, altering the measured voltage and introducing error. By isolating the divider from the rest of the circuit, the buffer ensures that the voltage being measured accurately reflects the unknown resistance.

The TL081 is powered using the ±12V rails, allowing it to handle the full range of voltages present in the system while maintaining linear operation.

Comparator Stage

The buffered voltage is fed into one input of an LM339 comparator. The comparator compares this measured voltage to a reference voltage generated by the digital potentiometer.

The comparator outputs a digital signal indicating whether the measured voltage is greater than or less than the reference. This effectively converts the analog comparison into a binary decision that can be read by the Raspberry Pi through a GPIO pin.

The comparator is a critical component because it bridges the analog and digital domains. It allows the system to interpret continuous voltage values as discrete logic levels, enabling software-driven measurement.

Reference Voltage Generation

The reference voltage is generated using a digital potentiometer controlled by the Raspberry Pi through the SPI interface. By adjusting the wiper position of the potentiometer, the system can create a variable voltage that serves as the comparison threshold.

This voltage is stepped through a range of values during the measurement process. Each step corresponds to a different approximation of the unknown resistance.

The resolution of the digital potentiometer determines how finely the system can adjust the reference voltage. This directly impacts the accuracy and resolution of the ohmmeter.

Successive Approximation Method

The resistance measurement is performed using a successive approximation algorithm implemented in software on the Raspberry Pi. This process operates similarly to a SAR (Successive Approximation Register) analog-to-digital converter.

At each step:

- The Raspberry Pi sets a reference voltage using the digital potentiometer
- The comparator indicates whether the measured voltage is higher or lower than the reference
- The system adjusts the reference voltage accordingly

This process continues in a binary search manner until the reference voltage closely matches the measured divider voltage. The final digital value corresponds to the resistance of the unknown component.

This approach allows the system to perform analog measurements using relatively simple hardware combined with digital control logic.

Digital Interface

The comparator output is connected to a GPIO pin on the Raspberry Pi. The Pi reads this signal during each step of the approximation process and uses it to determine how to adjust the reference voltage.

All control of the digital potentiometer is performed through the SPI interface, allowing precise and repeatable adjustments. The measurement process is therefore a combination of hardware comparison and software decision-making.

Operating Range and Accuracy

The ohmmeter is designed to operate over a resistance range of 500 Ω to 10 kΩ. The accuracy of the measurement depends on several factors, including:

- Tolerance of the reference resistor
- Resolution of the digital potentiometer
- Comparator switching characteristics
- Noise in the power supply and signal path

Because the system uses discrete steps rather than a continuous ADC, the measured value includes a tolerance that reflects the step size of the approximation process. This tolerance is reported alongside the measured value to indicate the expected accuracy.

Summary

The ohmmeter subsystem measures resistance by converting it into a voltage and using a comparator-based successive approximation method to determine its value. The combination of a resistor divider, op-amp buffer, comparator, and digital potentiometer allows the system to perform resistance measurement without dedicated ADC hardware.

This design demonstrates the integration of analog and digital techniques, using hardware to generate and compare signals and software to interpret the results. The result is a flexible and

efficient measurement system that meets the required operating range and performance constraints of the Electronics Tool Bench.

Voltmeter

The voltmeter subsystem is designed to measure DC voltages within the range of −5V to +5V by scaling the input signal and determining its value using a comparator-based successive approximation method. Similar to the ohmmeter, the design avoids the use of a dedicated analog-to-digital converter by combining analog signal conditioning with digital control from the Raspberry Pi.

Because the input can be both positive and negative, the signal is first scaled from ±5V to a 0–3.3V range using a resistor network. This allows the system to process the signal using a unipolar comparator and digital potentiometer reference. The scaled voltage is buffered using an op-amp to maintain signal integrity and prevent loading effects, ensuring that the measured signal accurately represents the input.

A second op-amp stage is used to route the reference voltage generated by the digital potentiometer to the comparator, where it is compared against the buffered input signal. The Raspberry Pi iteratively adjusts this reference voltage using a successive approximation algorithm until the comparator output converges, allowing the system to determine the measured voltage. The final implementation of the voltmeter is shown in Figure 6, and the corresponding physical implementation of the scaling and buffering stage is shown in Figure 7.

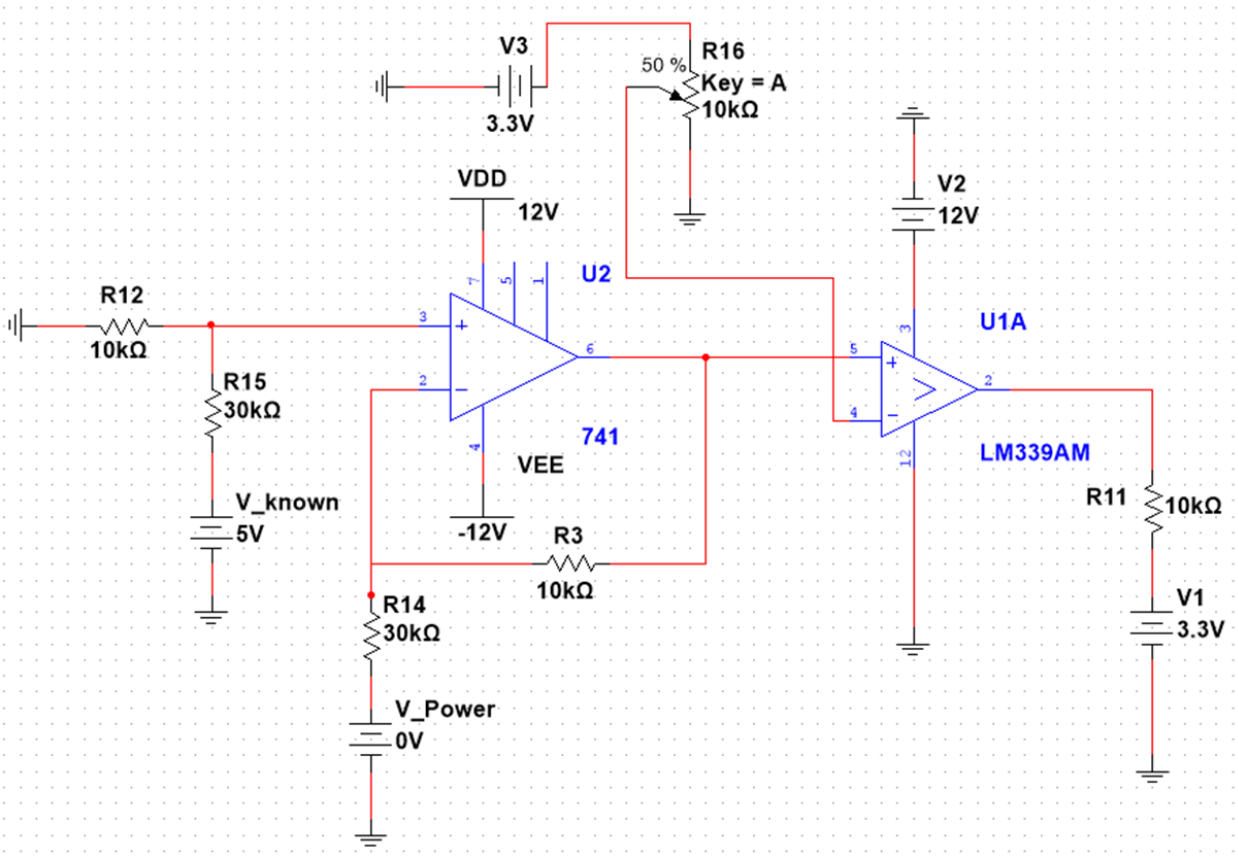


Figure 6. Final Multisim implementation of the voltmeter subsystem showing the input scaling network, op-amp conditioning stage, and comparator interface.

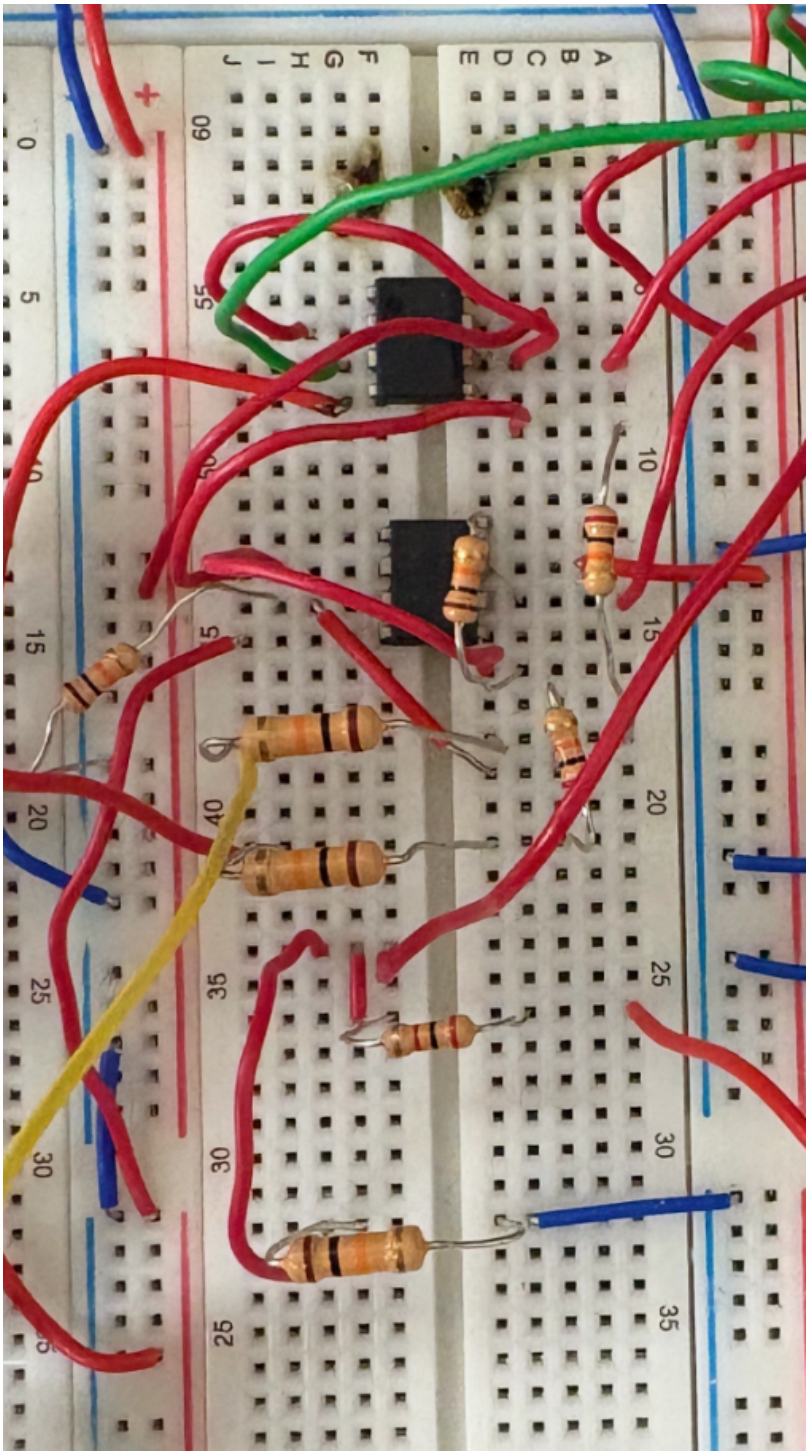


Figure 7. Physical implementation of the voltmeter scaling and buffering stage showing the resistor network used to map ±5V to a 0–3.3V range and the op-amp stages used for signal conditioning and reference voltage routing.

To better describe the operation of the voltmeter, Table 4 summarizes the major stages of the design.

Stage	Main Components	Function	Input	Output
1: Input Scaling	Resistor network	Maps ±5V input to measurable range	External voltage	Scaled voltage
2: Buffer Stage	TL081 op-amp	Stabilizes and isolates signal	Scaled voltage	Buffered voltage
3: Comparator Stage	LM339 comparator	Compares measured voltage to reference	Buffered voltage, reference	Digital HIGH/LOW
4: Reference Control	Digital potentiometer (MCP4131/4231)	Generates comparison voltage	SPI control	Reference voltage
5: SAR Logic	Raspberry Pi GPIO + software	Determines voltage iteratively	Comparator output	Measured voltage

Table 4. Summary of the voltmeter subsystem stages, showing the scaling of the input voltage and its conversion into a digital measurement using comparator-based successive approximation.

Measurement Principle

The voltmeter operates by first converting the input voltage into a scaled voltage that fits within the measurable range of the comparator and digital potentiometer. The system then uses a successive approximation method to determine the value of the input voltage by comparing it against a controllable reference.

Because the input voltage can be both positive and negative, the system must shift and scale the signal so that it can be processed using a unipolar comparison method. This is accomplished using a resistor network and op-amp conditioning stage.

Input Scaling Stage

The input scaling stage is responsible for mapping the input voltage range of −5V to +5V into a voltage range that can be handled by the comparator and digital control system.

This is achieved using a resistor network that shifts and scales the input signal into a positive range. In practice, this means that:

- −5V input is mapped close to 0V
- 0V input is mapped near the midpoint (~1.6V)
- +5V input is mapped close to 3.3V

This transformation allows the system to represent a bipolar input signal as a unipolar voltage, making it compatible with the comparator and digital potentiometer reference.

The scaling stage is critical because it ensures that the full input range is preserved while allowing the rest of the system to operate within safe voltage limits.

Op-Amp Buffer Stage

After scaling, the signal is passed through a TL081 operational amplifier configured as a voltage follower. This buffer stage provides high input impedance and low output impedance, ensuring that the scaled voltage is not affected by loading from the comparator or reference circuitry.

The buffer maintains signal integrity and ensures that the voltage being compared accurately represents the input signal after scaling. This is especially important because small errors at this stage would propagate through the measurement process.

Comparator Stage

The buffered voltage is fed into an LM339 comparator, where it is compared against a reference voltage generated by the digital potentiometer. The comparator produces a digital output indicating whether the measured voltage is higher or lower than the reference.

This output is read by the Raspberry Pi through a GPIO pin, allowing the system to interpret the result and adjust the reference voltage accordingly.

As in the ohmmeter, the comparator acts as the bridge between analog and digital domains, converting continuous voltage values into discrete logic signals.

Reference Voltage Generation

The reference voltage is generated using a digital potentiometer controlled by the Raspberry Pi through the SPI interface. By adjusting the wiper position, the system creates a variable voltage that can be compared against the measured signal.

This reference voltage is stepped through a range of values during the measurement process. The resolution of the digital potentiometer determines the smallest detectable change in voltage and therefore affects the accuracy of the voltmeter.

Successive Approximation Method

The voltmeter uses a successive approximation algorithm implemented in software to determine the input voltage. This process operates similarly to a SAR ADC.

At each step:

- The Raspberry Pi sets a reference voltage using the digital potentiometer
- The comparator determines whether the measured voltage is above or below the reference
- The system updates the reference voltage based on the result

This process continues until the system converges on a value that closely matches the scaled input voltage.

Once the scaled voltage is determined, the system converts it back to the original voltage using the inverse of the scaling relationship.

Digital Interface

The comparator output is connected to a GPIO pin on the Raspberry Pi, allowing the system to read the result of each comparison. The digital potentiometer is controlled through SPI, enabling precise and repeatable adjustments of the reference voltage.

The measurement process is therefore a combination of hardware comparison and software control, where the Raspberry Pi actively participates in determining the final value.

Operating Range and Accuracy

The voltmeter is designed to measure voltages within the range of −5V to +5V. The accuracy of the measurement depends on several factors, including:

- Precision of the resistor scaling network
- Resolution of the digital potentiometer
- Comparator switching characteristics
- Noise in the system

Because the measurement is based on discrete steps, the reported value includes a tolerance corresponding to the resolution of the system. This tolerance reflects the uncertainty introduced by the approximation process.

Summary

The voltmeter subsystem measures voltage by scaling the input signal into a measurable range and using a comparator-based successive approximation method to determine its value. The combination of analog signal conditioning and digital control allows the system to perform voltage measurement without dedicated ADC hardware.

This design complements the ohmmeter subsystem by using a similar measurement approach while adapting it to handle bipolar voltage inputs. Together, these subsystems demonstrate a consistent and efficient strategy for measurement within the Electronics Tool Bench.

DC Voltage Reference

The DC voltage reference subsystem is designed to generate a controllable output voltage that can be used for testing and measurement within the Electronics Tool Bench. This subsystem allows the system to act as a variable voltage source, providing a stable reference that can be adjusted digitally and used across different operating modes. By converting a digitally controlled input into a precise analog output, the design ensures consistent and repeatable behavior.

The circuit begins with a variable input voltage between 0 and 3.3 V, which is generated using a digital potentiometer. This signal is then passed into a non-inverting amplifier stage with a gain of 3, scaling the input to approximately 0 to 10 V. Following this, a second stage performs level shifting by subtracting a fixed 5 V reference. This shifts the amplified signal to a final output range of approximately −5 V to +5 V, allowing the system to produce a bipolar output centered around 0 V. Separating the gain and level shifting into two stages simplifies the design and improves stability.

This approach also allows each stage to be independently analyzed and verified. The gain stage ensures proper scaling of the input signal, while the level shifting stage guarantees that the output is centered correctly. Together, these stages provide a clean and predictable mapping from the digital input to the analog output.

Figures 8 and 9 show the Multisim implementation of the circuit at the extreme potentiometer settings, illustrating how the system behaves under minimum and maximum input conditions. Figure 10 shows the physical breadboard implementation of the same design, confirming that both the gain stage and level shifting stage operate as expected in hardware.

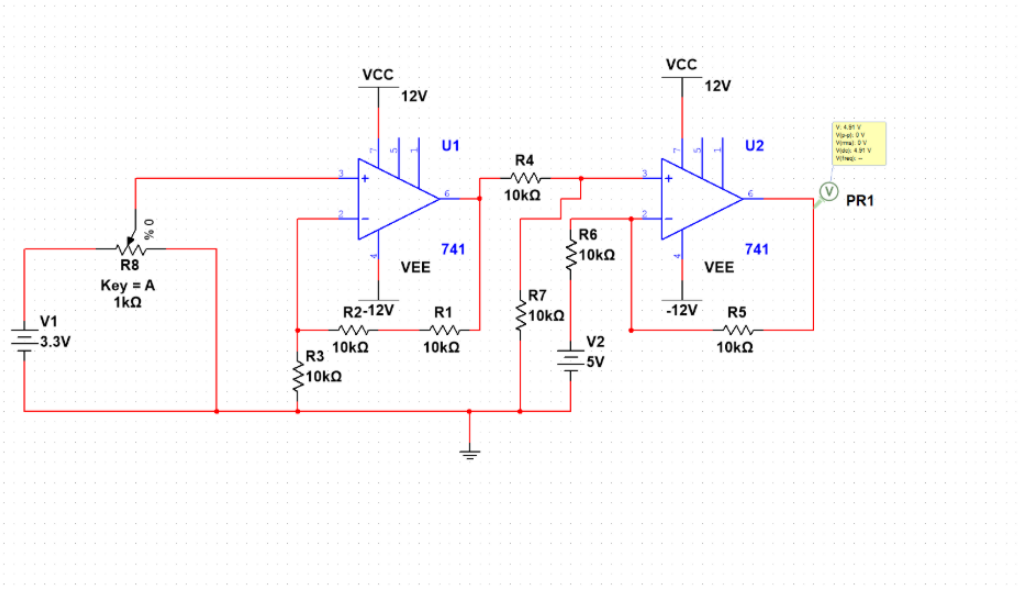


Figure 8. Multisim implementation of the DC voltage reference with the potentiometer set to 0 percent, showing the maximum input condition into the gain stage and the resulting amplified and level shifted output.

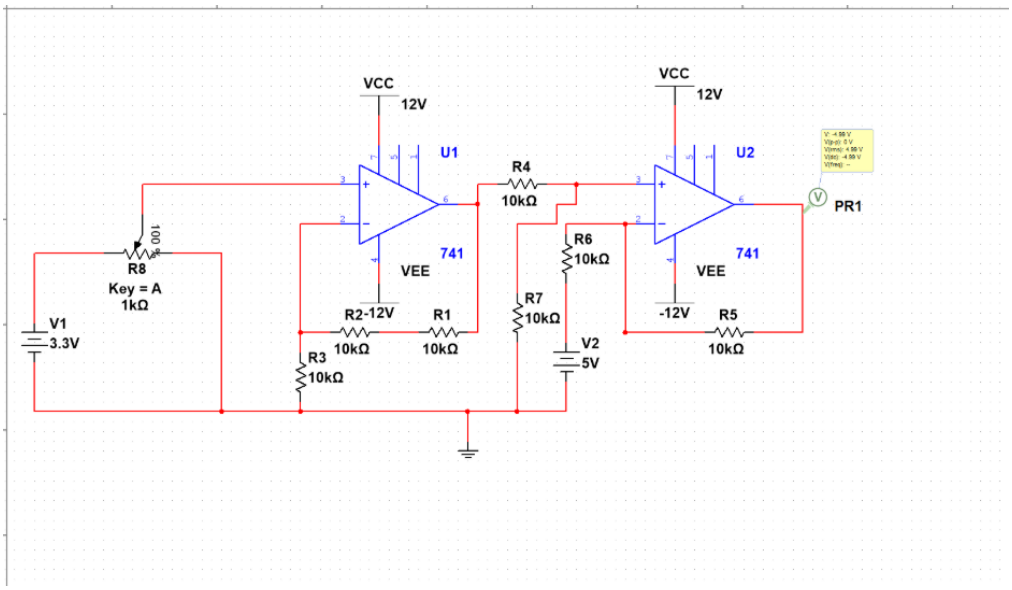


Figure 9. Multisim implementation of the DC voltage reference with the potentiometer set to 100 percent, illustrating the minimum input condition and the corresponding output after gain and level shifting.

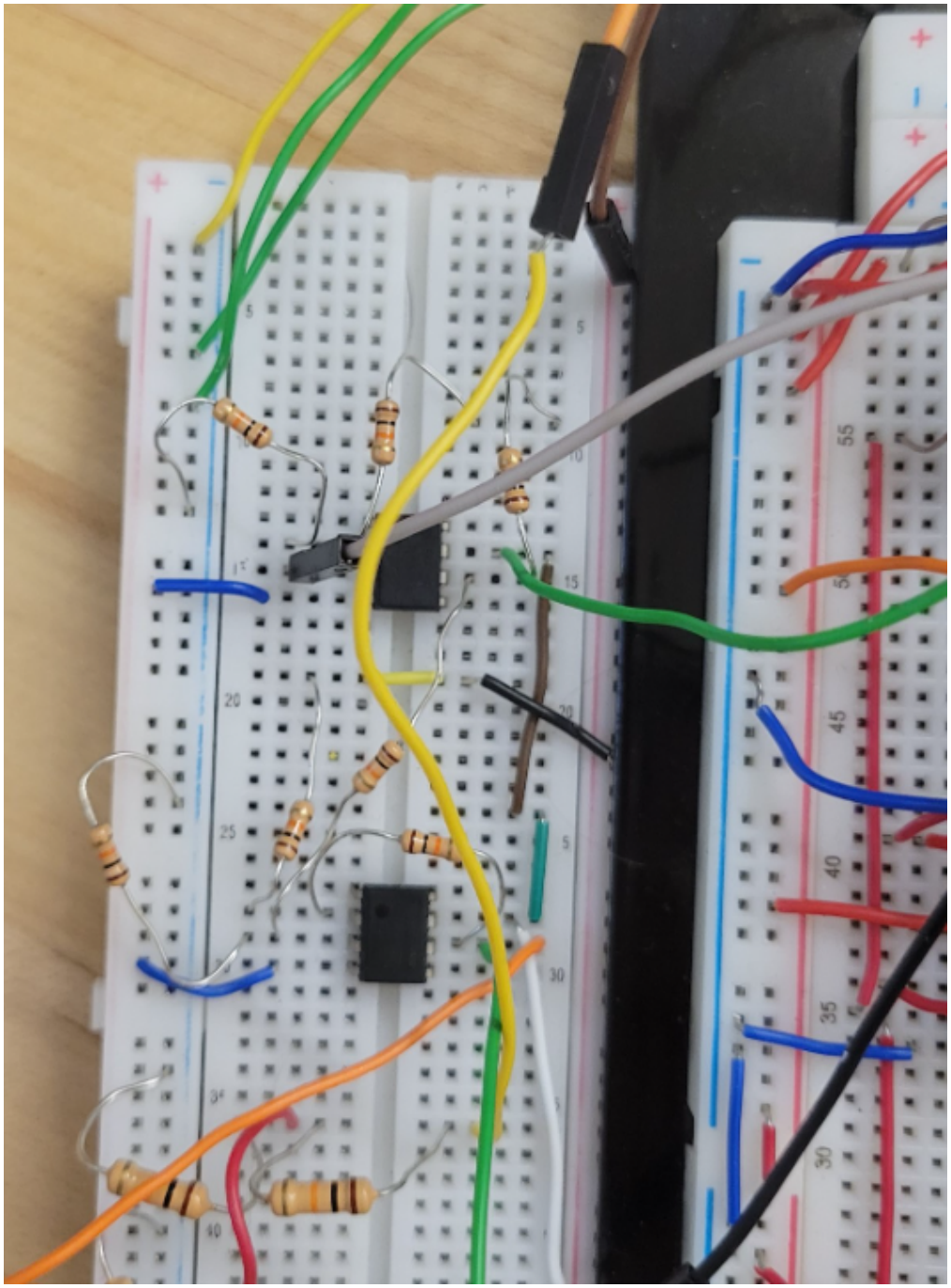


Figure 10. Physical breadboard implementation of the DC voltage reference subsystem, showing the gain stage and difference amplifier used to scale and level shift the output voltage.

Stage	Main Components	Function	Input	Output
1: Reference Generation	Digital potentiometer (MCP4131/4231)	Generates adjustable base voltage	SPI control from Raspberry Pi	0-3.3V signal
2: Gain Stage	TL081 op-amp	Amplifies input signal	0-3.3V	0-10V
3: Level Shifting	TL081 op-amp, resistor network, 5V reference	Shifts voltage range	0-10V	-5V to +5V
4: Output Stage	Buffering through op-amp	Provides stable output	Shifted voltage	Final output

Table 5. Summary of the DC voltage reference subsystem, outlining each stage from digital input generation through amplification, level shifting, and final buffered output.

Stage 1: Reference Generation

The first stage of the DC voltage reference subsystem is responsible for generating a controllable input voltage. This is accomplished using a digital potentiometer, which is driven by the Raspberry Pi through SPI communication. By adjusting the wiper position of the potentiometer, the system can produce a variable voltage between 0 and 3.3 V.

This stage is critical because it defines the resolution and controllability of the entire subsystem. Each step of the digital potentiometer corresponds to a discrete change in the output voltage, allowing the system to produce repeatable and predictable values. Since this signal serves as the input to the amplification stage, its stability directly impacts the accuracy of the final output.

Stage 2: Gain Stage

The second stage is implemented as a non-inverting amplifier using a TL081 op amp. The purpose of this stage is to scale the relatively small 0 to 3.3 V input up to a larger range that can

be more useful for testing applications. By selecting equal value resistors in the feedback network, the gain is set to exactly 3, resulting in an output range of approximately 0 to 10 V.

Using a non-inverting configuration ensures that the signal maintains the same polarity while being amplified. The choice of equal value resistors simplifies the design and reduces the chance of error due to resistor mismatch. Additionally, this stage takes advantage of the ±12 V supply rails, allowing the op amp to operate within a wide linear range without clipping.

Stage 3: Level Shifting

The third stage is a difference amplifier that performs level shifting by subtracting a fixed 5 V reference from the amplified signal. The purpose of this stage is to convert the unipolar 0 to 10 V signal into a bipolar output centered around 0 V. This is necessary for applications where both positive and negative voltages are required.

The difference amplifier is implemented using equal value resistors, which results in a unity gain configuration. This allows the circuit to perform a clean subtraction without introducing additional scaling. The 5 V reference is chosen because it represents the midpoint of the 0 to 10 V range, ensuring that the final output is symmetric about 0 V.

The overall transfer function of the subsystem is:

$$V_{out} = 3V_{in} - 5$$

This equation clearly shows how the input voltage is first amplified and then shifted to produce the final output.

Stage 4: Output Stage

The final stage of the subsystem ensures that the output voltage can be delivered reliably to external circuits. By buffering the output through the op amp, the system achieves a low output impedance, which minimizes the effect of loading when connected to other components.

This stage is important for maintaining signal integrity, especially when the output is used as a reference for measurement or testing. Without proper buffering, the output voltage could vary depending on the connected load, reducing the accuracy and usefulness of the subsystem.

Design Considerations

The use of separate stages for reference generation, amplification, and level shifting provides a clear and modular design. Each stage performs a specific function, making the system easier to analyze, debug, and modify. This approach also reduces the complexity of each individual stage, improving overall reliability.

Equal value resistors are used throughout the design to simplify calculations and reduce sensitivity to component variation. This allows the circuit to maintain consistent performance even with standard tolerance components. The choice of a 5 V reference for level shifting ensures that the output range is centered properly, maximizing the usable voltage swing.

Summary

The DC voltage reference subsystem converts a digitally controlled 0–3.3 V input into a bipolar output ranging from approximately −5 V to +5 V. This is achieved through a combination of amplification and level shifting stages, each designed to perform a specific function. The result is a stable and predictable output that can be used reliably for testing and measurement applications within the Electronics Tool Bench.

Signal Generation (Square + Sine)

Square Wave Generation

The square wave generator subsystem is designed to produce a 50% duty cycle waveform with an adjustable frequency and amplitude. The waveform is generated digitally by the Raspberry Pi and then conditioned using analog circuitry to meet the required voltage range and signal quality.

The final implementation of the square wave generator is shown in Figure 11.

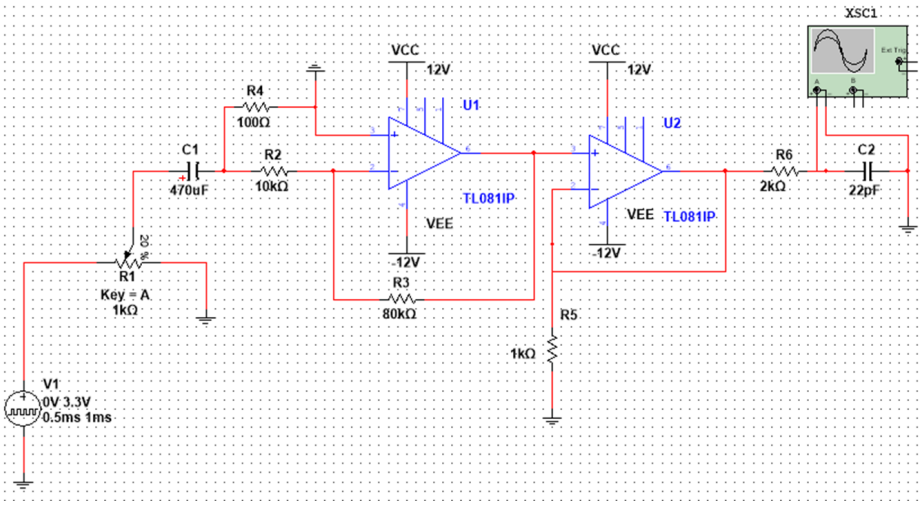


Figure 11. Final Multisim implementation of the square wave generator subsystem showing PWM input, conditioning stages, and output circuitry.

To describe the operation of the square wave generator, Table 6 summarizes the major stages of the design.

Stage	Main Components	Function	Input	Output
1: PWM Source	Raspberry Pi GPIO	Generates square wave signal	Software-defined frequency	0-3.3V square wave
2: Buffer/Conditioning	Op-amp stage	Stabilizes and isolates signal	PWM signal	Buffered square wave
3: Amplitude control	Digital potentiometer	Adjusts signal amplitude	Buffered signal	Scaled signal
4: Output Stage	Op-amp gain stage	Expands voltage range	Scaled signal	±10V square wave

Table 6. Summary of the square wave generation subsystem, showing the progression from the Raspberry Pi PWM signal through buffering, amplitude control, and final amplification to produce a ±10 V output waveform.

PWM Signal Generation

The square wave originates from a GPIO pin on the Raspberry Pi using pulse-width modulation (PWM). The PWM signal is configured with a fixed duty cycle of 50%, ensuring a symmetrical square wave.

The frequency of the waveform is controlled entirely in software by adjusting the PWM frequency. This allows the system to generate square waves over the required range of 100 Hz to 10 kHz.

The initial PWM output is limited to a voltage range of 0V to 3.3V, which is insufficient for the required output specifications. Therefore, additional analog stages are required to scale and condition the signal.

Buffer and Conditioning Stage

The PWM signal is first passed through an operational amplifier configured as a buffer. This stage isolates the Raspberry Pi from the rest of the circuit and provides a stable, low-impedance signal for further processing.

Without this stage, the loading effects of later components could distort the waveform or affect the GPIO output. The buffer ensures that the digital signal is preserved while allowing the analog circuitry to operate independently.

Amplitude Control Stage

A digital potentiometer is used to control the amplitude of the square wave. The Raspberry Pi adjusts the wiper position of the potentiometer through the SPI interface, allowing the amplitude to be varied in discrete steps.

This stage allows the system to meet the requirement of adjustable amplitude while maintaining consistent waveform shape. Because the square wave is a digital signal, scaling its amplitude does not affect its frequency or duty cycle.

Gain and Output Stage

After amplitude control, the signal is passed through an op-amp gain stage that increases the voltage to the required output range. The goal of this stage is to produce a waveform that spans approximately ±10V, centered around 0V.

The use of ±12V supply rails allows the op-amp to generate both positive and negative voltage levels, enabling a bipolar output waveform. This is necessary to meet the system requirement of a square wave centered around 0V.

The gain stage ensures that even small input signals from the digital potentiometer can be amplified to the full output range.

Signal Characteristics

The resulting waveform is a square wave with the following characteristics:

- Frequency range: 100 Hz to 10 kHz
- Duty cycle: 50%
- Amplitude: Adjustable up to ±10V
- Center: 0V

Because the waveform is generated digitally, it maintains a clean and stable frequency. The analog stages ensure that the amplitude and voltage levels meet the required specifications.

Design Considerations

Unlike the sine wave generator, the square wave does not require filtering because the square shape is the desired output. Any filtering would distort the waveform by rounding the edges and reducing harmonic content.

The main design challenge is therefore not waveform shaping, but rather amplitude scaling and ensuring that the signal remains stable across different frequencies and loads.

The use of a buffer stage and controlled amplification ensures that the waveform remains consistent while protecting the Raspberry Pi from direct interaction with the analog circuitry.

Summary

The square wave generator subsystem produces a controllable square wave using PWM from the Raspberry Pi combined with analog conditioning and amplification. By separating signal generation from signal conditioning, the design maintains flexibility in frequency control while achieving the required output voltage range.

This subsystem complements the sine wave generator by providing a simple and reliable waveform source for testing and analysis within the Electronics Tool Bench.

Sine Wave Generation

The sine wave generator subsystem is designed to produce a smooth sinusoidal waveform with adjustable frequency and amplitude using a combination of PWM generation and analog signal conditioning. The input signal originates as a PWM output from the Raspberry Pi and is processed through filtering, buffering, and amplification stages to approximate a continuous sine wave.

A square wave contains a fundamental frequency component along with higher-order harmonics. This relationship can be expressed as:

$$v(t) = \frac{4V}{\pi}\left[\sin(\omega t) + \frac{1}{3}\sin(3\omega t) + \frac{1}{5}\sin(5\omega t) + \cdots\right]$$

This shows that the desired sine wave exists within the signal, along with higher-frequency harmonic components that must be removed through filtering.

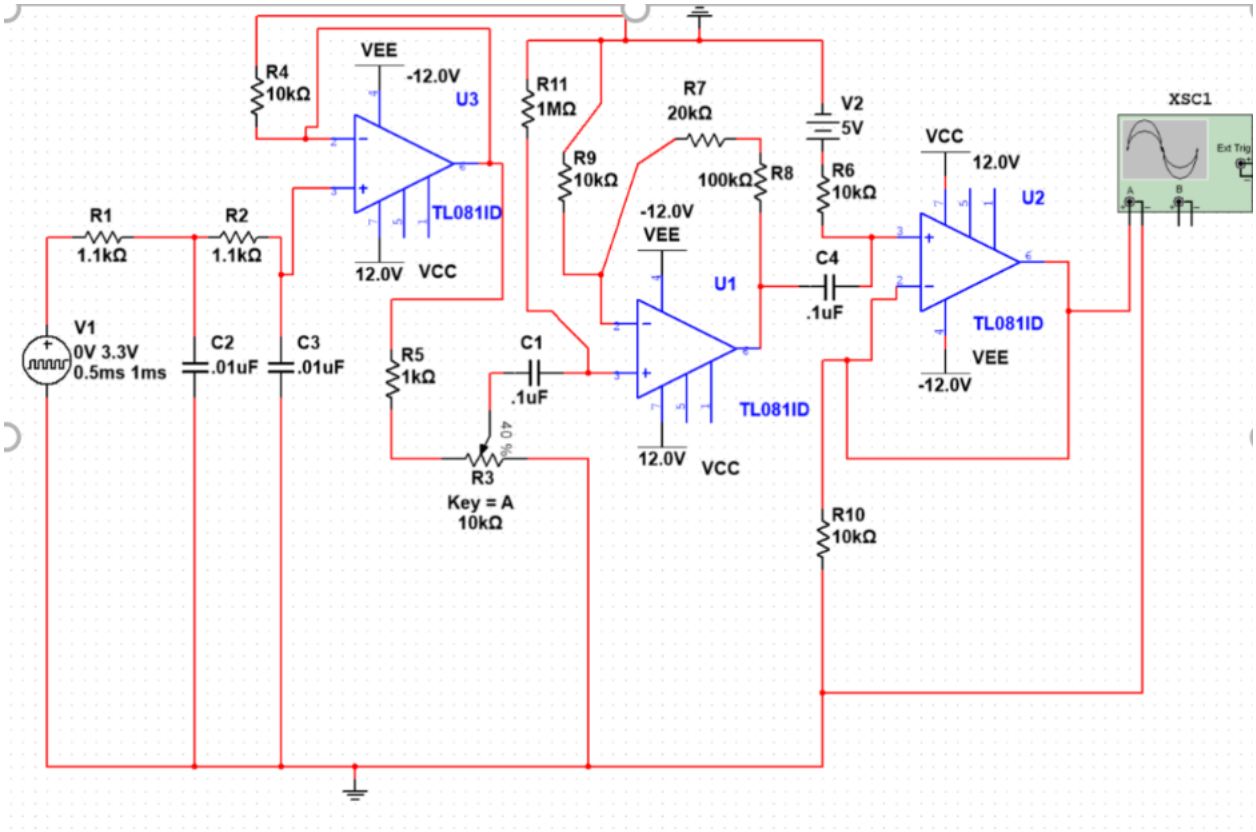


Figure 12. shows the Multisim implementation of the full sine wave generator, including all filtering, buffering, amplitude control, and gain stages. The oscilloscope is connected at the output to verify that the resulting waveform closely matches a sine wave.

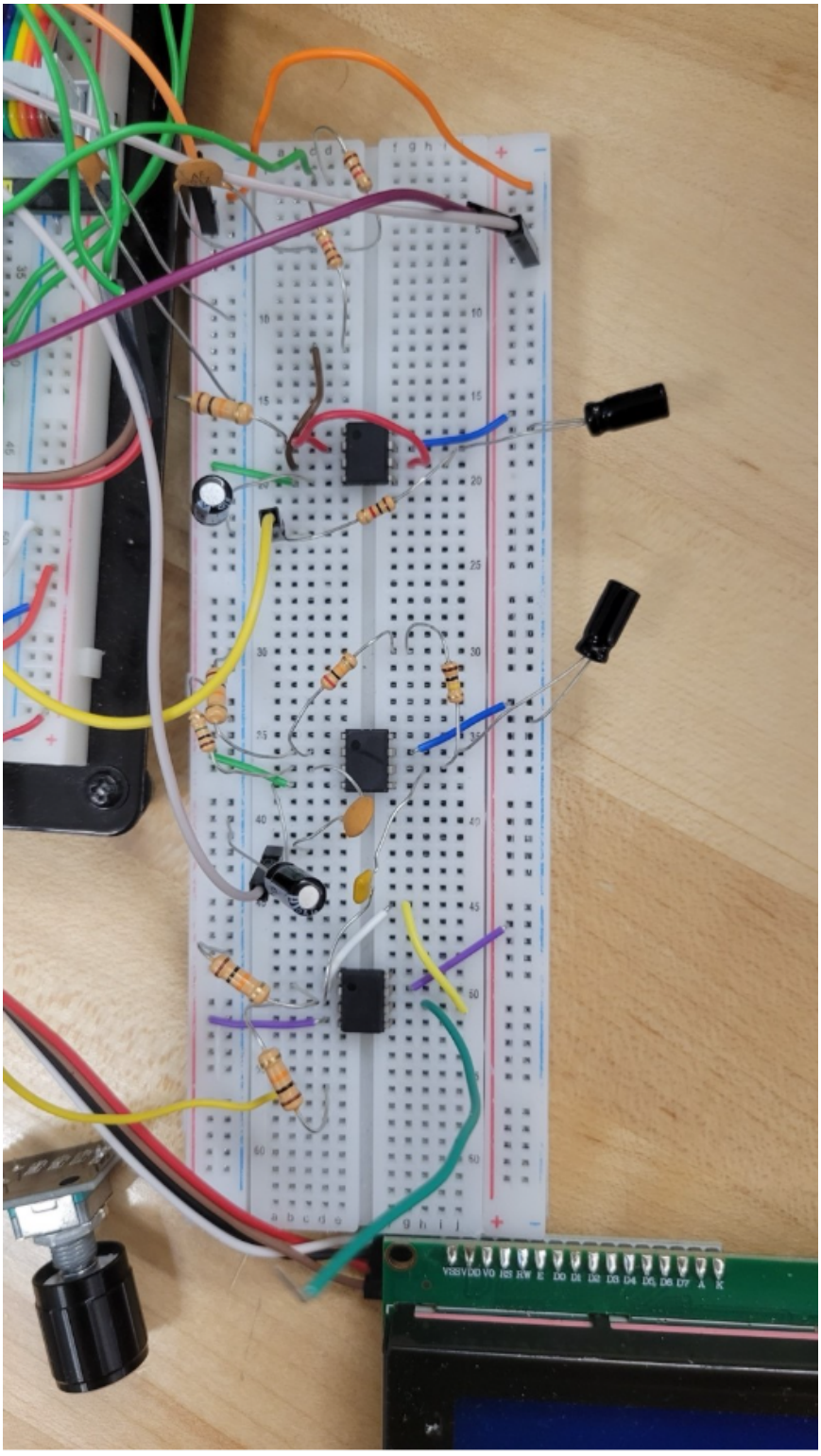


Figure 13. Physical breadboard implementation of the sine wave generator showing the filtering stages, digital potentiometer for amplitude control, and op-amp gain stages used to produce the final output waveform.

To better organize the operation of the subsystem, Table 7 summarizes the major stages of the sine wave generator.

Stage	Main Components	Function	Input	Output
1: PWM Source	Raspberry Pi GPIO	Generates PWM square wave	Software-defined frequency	0-3.3V square wave
2: Filtering Stage	RC low-pass networks	Removes higher-order harmonics	PWM signal	Smoothed waveform
3: Buffer Stage	Op-amp (voltage follower)	Stabilizes and isolates signal	Filtered signal	Buffered signal
Amplitude Control	Digital Potentiometer (MCP4131/4231)	Adjusts signal magnitude	Buffered signal	Scaled signal
Gain/Output Stage	Op-amp gain stage	Expands voltage range and sets offset	Scaled signal	0-10V sine wave

Table 7. Summary of the sine wave generation subsystem, showing the progression from PWM signal generation through filtering, buffering, amplitude control, and final amplification to produce a smooth sinusoidal output.

PWM Signal and Filtering Stages

The input to the system is a PWM signal generated by the Raspberry Pi, producing a 0–3.3V square wave. To extract the fundamental sine component, the signal is passed through cascaded RC low-pass filters. The cutoff frequency of each filter is given by:

$$\left[f_c = \frac{1}{2\pi RC}\right]$$

Buffer and Amplitude Control

After filtering, the signal is passed through an op-amp buffer stage to prevent loading effects and preserve waveform integrity. A digital potentiometer is then used to adjust the amplitude of the signal. This allows the Raspberry Pi to control the output magnitude through SPI without affecting the waveform shape or frequency.

Gain and Output Stage

The final stage consists of an op-amp gain stage that amplifies the filtered signal to the required output range. The waveform is scaled to approximately 0–10V and centered around a DC offset of 5V, ensuring compatibility with the system's output requirements.

Signal Characteristics

- Frequency range: 1 kHz to 10 kHz
- Amplitude: 0V to 10V (adjustable in steps)
- Offset: Centered around 5V
- Output condition: High impedance

Design Considerations

The effectiveness of the sine wave generator depends on proper filter design. The RC values must be chosen to sufficiently attenuate harmonic content while still allowing the desired frequency range to pass. Buffering ensures that the waveform is not distorted by loading effects, and careful gain selection prevents clipping while achieving the required amplitude.

Summary

The sine wave generator converts a PWM square wave into a sinusoidal waveform by filtering out higher-frequency harmonics and amplifying the remaining fundamental component. By combining digital control with analog processing, the subsystem provides a stable and adjustable sine wave for use within the Electronics Tool Bench.

Frequency Measurement

The frequency measurement subsystem is designed to determine the frequency of an external input signal and display the result through the Electronics Tool Bench. This subsystem allows the system to function as a frequency counter by converting an analog waveform into a digital signal that can be processed by the Raspberry Pi.

The circuit operates by conditioning the incoming signal using a comparator. The comparator converts the analog waveform into a clean digital signal by switching between HIGH and LOW states whenever the input crosses a reference threshold. This process effectively transforms the input waveform into a square wave, making it suitable for digital edge detection regardless of the original waveform shape.

Figure 14 shows the Multisim implementation of the comparator stage used for signal conditioning. In this design, the input signal is compared against a fixed reference voltage, and the output transitions sharply between the supply rails. This creates a digital waveform that preserves the timing characteristics of the original signal while removing noise and ambiguity near zero crossings.

Figure 15 shows the physical breadboard implementation of the same circuit. The comparator output is routed to a Raspberry Pi GPIO pin, where the digital signal is used for timing analysis. This physical implementation confirms that the comparator successfully produces a clean digital waveform suitable for frequency measurement.

Once converted to a digital signal, the waveform is processed by the Raspberry Pi by measuring the time between rising edges. This allows the system to determine the period of the signal. By taking the inverse of the measured period, the system calculates the frequency of the input signal. This approach allows accurate measurement across a wide range of input frequencies while keeping the hardware relatively simple.

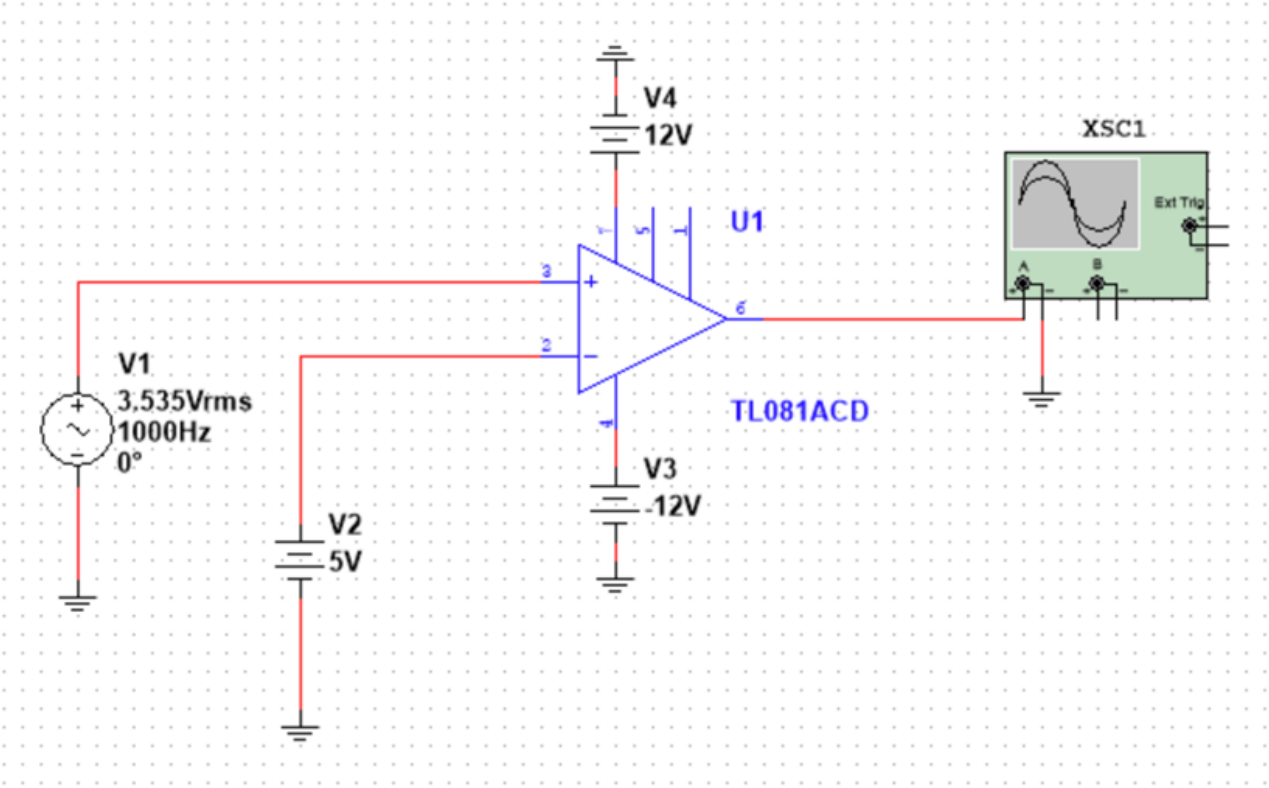


Figure 14. Multisim implementation of the frequency measurement subsystem showing the comparator stage used to convert the analog input signal into a digital waveform for edge detection.

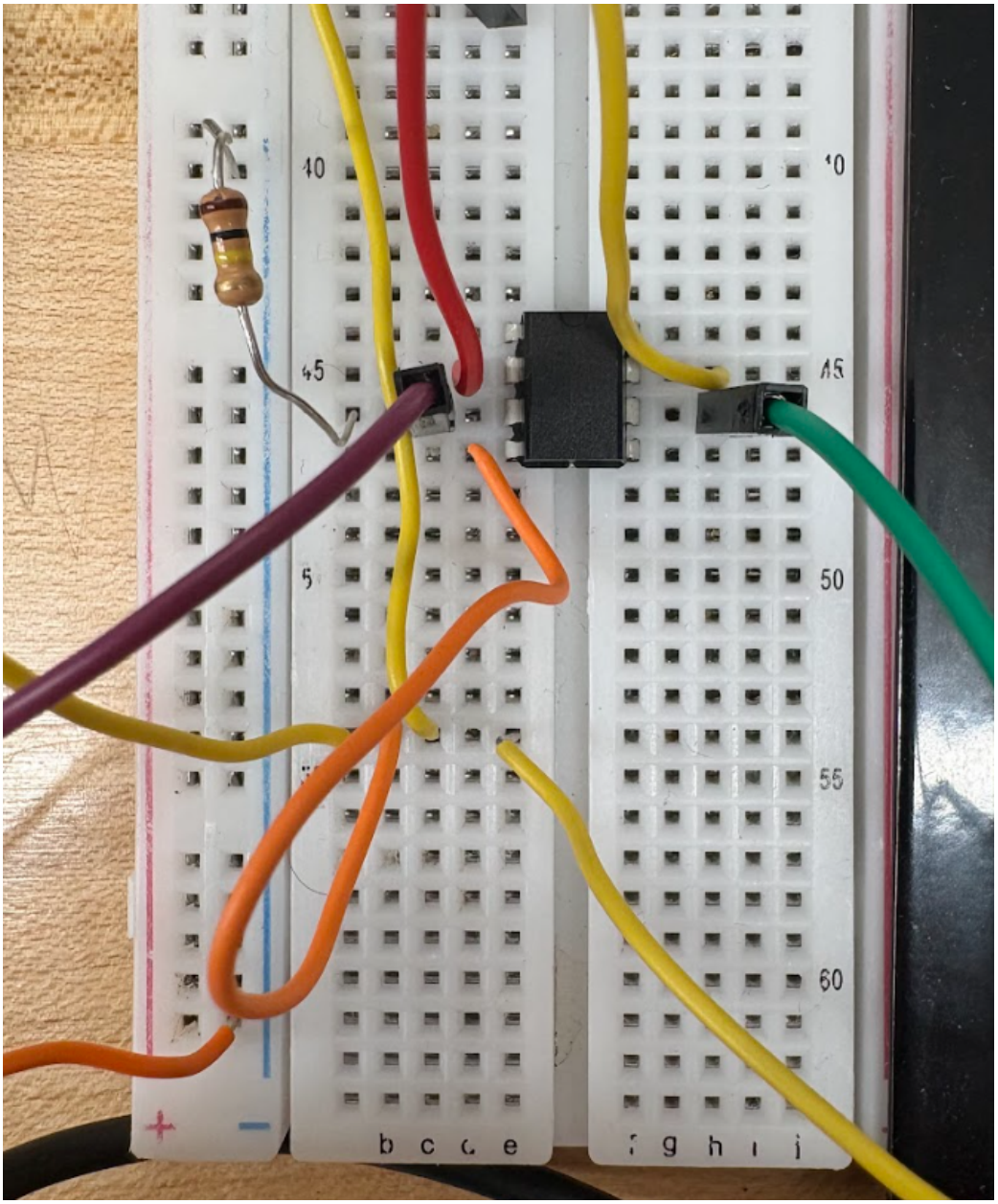


Figure 15. Physical breadboard implementation of the frequency measurement subsystem, showing the comparator circuit used to condition the input signal before it is sent to the Raspberry Pi for frequency calculation.

To better organize the operation of the subsystem, Table 8 summarizes the major stages of the design.

Stage	Main Components	Function	Input	Output
1: Signal Conditioning	Comparator (LM339 or TL081 configured as comparator)	Converts analog signal to digital waveform	External input signal	Digital HIGH/LOW signal
2: Edge Detection	Raspberry Pi GPIO	Detects signal transitions	Digital waveform	Timing intervals
3: Frequency Calculation	Raspberry Pi software	Computes frequency from period	Timing intervals	Frequency value

Table 8. Summary of the frequency measurement subsystem, illustrating the process of converting an analog input signal into a digital waveform and computing its frequency through edge detection and timing analysis.

Stage 1: Signal Conditioning

The first stage uses a comparator to convert the incoming analog signal into a digital waveform. The comparator compares the input signal against a fixed reference voltage and switches its output whenever the input crosses this threshold. This produces a square wave that preserves the timing of the original signal while eliminating variations in amplitude.

Stage 2: Edge Detection

The second stage detects transitions in the digital waveform using a GPIO pin on the Raspberry Pi. By monitoring rising edges, the system can determine the time interval between successive transitions. This timing information is essential for calculating the signal period.

Stage 3: Frequency Calculation

The final stage calculates the frequency by taking the inverse of the measured period. The Raspberry Pi processes the timing data and converts it into a frequency value that can be

displayed to the user. This method provides an efficient way to measure frequency using minimal hardware.

Design Considerations

Using a comparator for signal conditioning simplifies the hardware design while ensuring reliable operation. The comparator allows the system to accept a wide range of input waveforms and produce a consistent digital output. The overall accuracy of the subsystem depends on the timing resolution of the Raspberry Pi and the stability of the input signal.

Summary

The frequency measurement subsystem converts an analog input signal into a digital waveform and determines its frequency using timing measurements on the Raspberry Pi. By combining simple analog conditioning with digital processing, the system provides a reliable and efficient method for frequency measurement within the Electronics Tool Bench.

Software Design

The software design of the Electronics Tool Bench brings the physical components to life and fosters the communication between the Raspberry Pi 4 and the electric circuit parts. It facilitates the bidirectional conversion between digital and analog signals to support the software's compatibility with the hardware design.

For the entire system to perform as designed, the blueprint for the software system must support separate subsystems that are the ohmmeter, voltmeter, square wave generator, sine wave generator, and DC voltage reference. At the same time, each tool is further analyzed to determine how to appropriately assign each section of its overall logic. For example, the code for the square wave generator is divided into constants, conversion from amplitude into 7-bit MCP4131 steps, and a class with properties along with set and write functions to change the step value of the wiper. This design pattern grants each subsystem is fulfill their procedures to for the entire system to run.

A major consideration in the software design is the top-down approach of dividing the Electronics Tool Bench into the tools that constitute it. This allows each tool to be tested in

conjunction with their hardware counterparts without affecting the other. This manner of organization also lessens the complexity of the codebase by having functions and calculations reside only inside their respective tools, with all essential parts imported into the driver program where the main loop executes.

Another aspect of the design that took deliberation is translating from a set of abstract and concrete constraints into a sequence of procedures. Once the whole is distilled down into realizable components, each file is designed starting with defining constants. Then, the abstract constraints are deciphered into high-level functions, which are each further elaborated with calculations, modifying states, reading or writing into the GPIO pins, or conditional statements and loops. By then, the parameterized and local variables have already been defined, and the entire electronic tool has been defined into software instructions.

The GPIO pins allow an exchange of digital data through SPI and I2C interfaces to activate each tool's functionality and deliver a friendly graphical interface to the user. This is achieved with the provided functions from the *pigpio* library; its documentation of each function is explanatory, and it provides many code examples that are inherited into the Tool Bench, such as the attached Python file for writing to the LCD. The procedures that each tool follow have their design requirements simplified down to *pigpio* library function calls.

The entire software design follows a systematic organization of programs to facilitate the communication of signals from the Raspberry Pi 4 to the electronic circuit elements. Without this, the Electronic Tool Bench cannot provide a user-friendly experience with non-functioning tools.

Software Architecture Overview

The software architecture of the Electronics Tool Bench organizes the program logic into distinguishable parts that comprise and regulate the general system of exchanging data. While the system performs the necessary calculations and logic to run crucial components, it also provides a cohesive structure of the page tree behind a responsive UI through user input.

The control flow within the software enables the execution of programs sequentially. It is initiated by the driver program, which activates the Raspberry Pi 4 and evaluates the selections to be chosen by the user. All selections load their respective pages and initialize the necessary programs to run the logic for activating the desired tool.

Each tool is separated into files containing a specialized set of logic to be run from the driver program. This software approach of functional decomposition divides the system into components with individual attributes containing specified inputs, outputs, and functions. If the entire software architecture situates inside one file, then additional time is required to locate bugs and modify improve an area within the code. Instead, the control flow starts at the driver program and continues down the necessary programs depending on the selected tool.

Functions carry the control flow into an executable block of code that returns a value to the source of the call. They allow organization of the program logic in the same manner separating files allow separating concerns of the software architecture. Additionally, functions accept parameters from the calling source to perform calculations or change a state within the program.

Callback functions are those passed as parameters to another function. Because they require an event before computing and returning a resulting value, they are important for asynchronous programming with the Raspberry Pi 4. This approach helps regulate executions of multiple processes; an example of this is writing to an LCD at the same time as detecting the rotary encoder's movement.

The global interpreter lock (GIL) is a feature in Python that allows only one thread, or a currently running process, to be executed in a point of time. This enhances multi-threading because it eliminates the issue of race conditions, such as in cases when many threads update the same variable at once. The GIL effectively turns a multi-threaded program and a single-threaded one, simplifying memory control while improving code performance.

Interrupts are unpredictable events that a processor responds to, such as a rotary encoder knob turn or button press. The interrupted program has its execution paused for the processor to immediately handle the event. Since Python executes on a single thread due to the GIL, an

interrupt be situated on another thread, but instead, special callback functions can wait for an event to occur to execute their responses.

Shared state refers to the variables whose values are changed depending on the current condition of the program. They are defined under a single dictionary inside the driver program, which is where the software system of the Electronics Tool Bench begins. With the user free to explore the user interface, the states are reassigned when a rotary encoder input is detected or when a tool is being prepared or operated.

Imports allow different files to interact with one another, and Python files are referred to as modules, as denoted by the .py extension. Because of imported modules, the software architecture retains its hierarchical structure with the driver program as its terminus and the other tools as the connected stations that function independently from one another. Certain sections of code of the desired module can be ignored, specifically those not part of a class, such as functions and constants. This is only done inside the driver program because the idea of separation of concerns forces it to use the most abstract functions from within their corresponding tools to control the flow down the user interface tree.

One distinctive design choice into the software architecture is having the driver program module contain more code than the main loop. For the Electronic Tool Bench, the purpose of a driver is to initialize the Raspberry Pi 4 and shared state in preparation for handling the user interface. The importance of this module is extended to contain the interfaces for setting, measuring, or adjusting values for a selected electronic tool. The driver defines no new constants but instead imports those specific to the other programs of the software system.

Another important design decision is combining the rotary encoder and LCD callback functions into one file. These include interrupts that wait for any event of a rotary encoder, a handler for displayed page options, and helper functions for adjusting numeric values on rotation. However, the most significant role of this is file is defining the functions for handling array of active callbacks inside the state dictionary. This file is comparable to a supervisor responsible for declaring orders when a user input is detected and defines functions. By having a standalone file dedicated to callbacks, it frees the driver file and electronics tool programs from dealing with callbacks by themselves.

The architecture is such that the tools are assigned to satisfy their functional requirements, and the driver program gives instructions to operate the complete software system. This allows the multiple subsystems to independently communicate with their respective hardware without interference with one another. Furthermore, this manner of organization streamlines the debugging process and the additions of new features.

System Setup

The development on the Electronics Tool Bench centers around GitHub, which is a shared platform where multiple programmers collaborate on a software project. It is situated inside of a repository, a project folder that contains its files and revision history, allowing for off-site uploads of new additions or modifications of its contents.

This repository stores the project's overview, progress report, codebase, user manual, technical description, pictures and videos for in-depth documentation. The GitHub platform depends on the Git version control system to reserve and display regular updates to the project repository. Git commands handle the tasks of saving changes on an individual system and forwarding changes to the public project repository. They are executed directly on the terminal (or command prompt) or through an IDE (integrated development environment), such as Visual Studio Code.

If the project management is restricted to one machine, then collaboration is obsolete and decision-making is filtered into a single version of the repository. GitHub supports uploads of different versions, or branches, which may contain features which are not finalized into the main project. More importantly, with development spread across multiple machines, progress on the software system is accelerated and changes or new additions can be published off-site, away from testing equipment.

GitHub Repository Setup

On a machine, a new project repository is created from the GitHub website from the "+" dropdown menu under "New repository", where it is appropriately named. This machine's owner therefore owns the project folder and has direct access to it. For another machine to have remote access to this folder, through the terminal (or command prompt) or IDE, a dedicated location is decided on with this command to change to the desired directory:

$ cd <directory path>

For situations where the incorrect directory is , this cd command changes to the home directory, or the default current directory on a Terminal launch:

$ cd .

And this cd command changes to the parent or the immediate previous directory:

$ cd ..

Then, this command is executed:

$ git clone <SSH Key>

This is a Git command which creates a local copy of the project through an SSH key, or a secure shell key, which prompts a password before unlocking access to the repository. An SSH key is more secure than HTTPS because the data is encrypted by a key pair; a private key is handled by a group member to sign, and a public key is handled by GitHub but kept in the Git account which created the repository. However, other machines must have their own private SSH keys to unlock access.

Creating SSH Key

An SSH key is generated on a machine by running this command:

$ ssh- keygen -t ed25519 -C “<student_email@uga.edu>”

When the terminal displays the following messages:

> Generating public/private ALGORITHM key pair.
> Enter a file in which to save the key (/Users/YOU/.ssh/id_ALGORITHM):

The ↵ Return key is pressed to save the key to the default file location:

Including the passphrase for the SSH key is optional.
> Enter passphrase (empty for no passphrase):

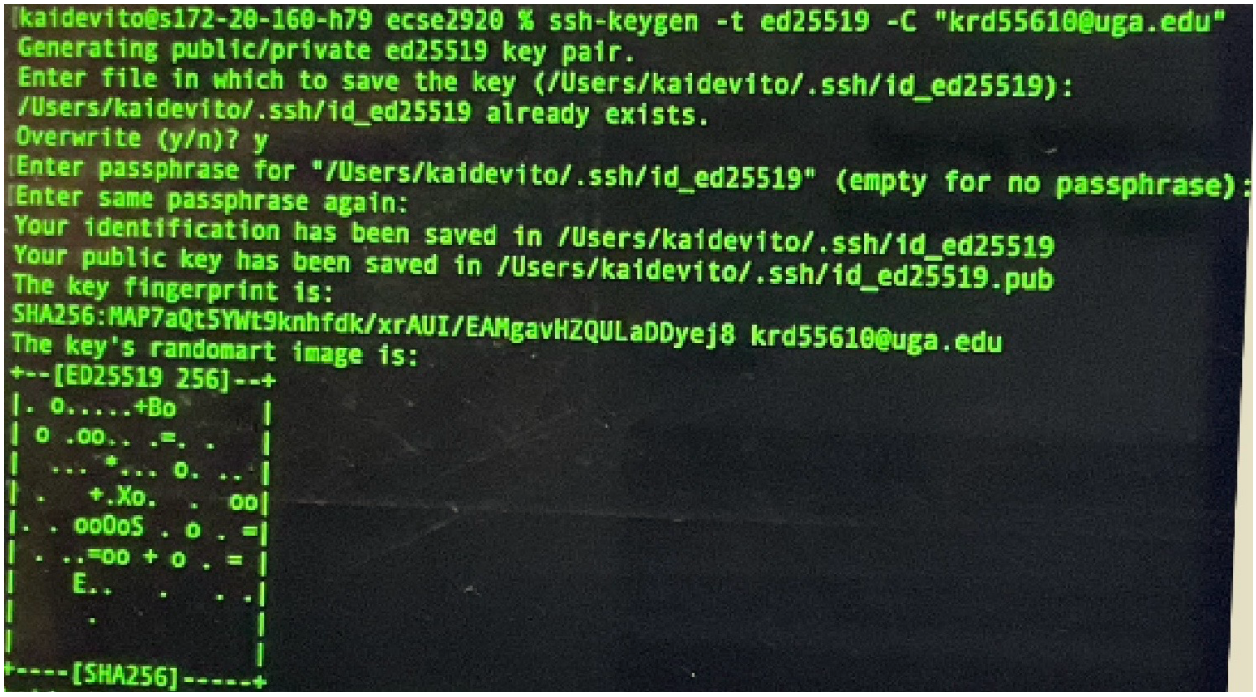


Figure 16. A screenshot of the Terminal showing the successfully created public SSH key on a machine without direct access to the GitHub repository.

Once the SSH key is saved into the .pub file, the following commands copy and pastes its contents to display:

$ pbcopy < ~/.ssh/id_ed25519.pub

$ pbpaste

The result from $ pbpaste is then inserted into the textbox under GitHub Profile Picture, Settings, SSH and GPG keys, and New SSH key. Through this procedure, the machine and its associated GitHub account has access to the project folder and its contents through the secure key.

Git Commands

The Git workflow sustains collaboration by storing local project saves to a singular machine, especially to one where the repository is not created but cloned through the SSH key. Since GitHub is a cloud-based platform organized for maintaining a large codebase, it uses Git to track and warn of save conflicts between multiple machines before publishing. For example, when two

copies of the same file contain a block of code that are typed differently, Git identifies this as a conflict to be fixed first, keeping project management stable.

There are three areas where saves are kept locally: the copied (or working) directory on the machine with locally made changes, then the staging area for organizing the files for pushing, or publishing to the Git repository; then the local folder storing the cloned repository.

After the repository has been cloned, the machine can catch up to the most recent changes by proceeding to add or remove changes published on the public repository through this command:

$ git pull

This is similar to the clone command since the stored files on the public repository are copied to the local folder. Once more, cloning the repository is a prerequisite to pulling it.

The following table provides detailed descriptions of the essential Git commands to control the workflow, with important options to consider and the flow of the project's contents:

Git Workflow Commands			
Command	**Options**	**Action**	**Flow**
$ git pull	N/A	Transfer public repository contents to local machine	Public Repository → Local Repository
$ git add	<file-name>	Adds the file to the staging area	Working Area → Staging Area
	.	Adds all contents to the staging area	
$ git commit	-m "message"	Saves to the local repository without a popup screen to include the message	Staging Area → Local Repository
$ git push	N/A	Publish changes to the public repository	Local Repository → Public Repository

Table 9. The Git commands which sustain the project workflow from a remote machine to the public repository kept on GitHub. The three areas for keeping saves locally, as well as the public repository, are indicated in the direction of flow of the project files.

This command can be used to verify the current state of the working directory, the staging area, and the local repository are:

$ git status

Handling Conflicts

At times, two or more machines may have conflicting variants of the same local repository, caused by one or multiple files having modifications that override one another. Similarly, one machine can have many branches that cannot be merged into the main branch. Both situations are examples of conflicts that arise in development of the software system.

Git issues these warnings individually to each machine owner, and one solution is to resolve them manually. An IDE makes this convenient by reporting individual lines inside the local repository which conflict with those in the public repository, or lines inside the main branch with conflict with the remaining branches.

A machine could have an outdated version of the repository during the project development process. If pulling the public repository raises conflicts on the local repository, the pull command can be reconfigured with this command:

$ git config pull.rebase true

Rebasing reduces merge conflicts by overlaying the local changes with the contents of the public repository, preserving newly included files from the machine. Alternatively, this setting can be set to false, like so:

$ git config pull.rebase false

In this manner, a merge is attempted during each pull, which has a disadvantage of complicating the branch history and increasing merge conflicts.

https://graphite.com/guides/git-pull-rebase

Raspberry Pi 4 Setup and Initial Configuration

With the GitHub project repository prepared with both direct and SSH key access from all machines, the Raspberry Pi 4 is required to be set up as the primary hardware platform for the project. On the same machine where the public repository was created, the Raspberry Pi OS is installed and configured to allow it to function independently as a development and demonstration system. Once the system is running, the software setup is proceeded.

Because the UGA WiFi network requires authentication methods that are not easily supported by the Raspberry Pi, the device is unable to maintain a stable wireless connection. One way to resolve this issue is with an Ethernet cable to create a bridge connection between the Raspberry Pi 4 and a machine to host the OS. The machine can connect to the WiFi network and share its Internet connection with the Raspberry Pi 4 through the Ethernet port.

This approach allows the Raspberry Pi 4 to reliably access the Internet and continue the setup process without interruption. Once the bridge connection was established, the Raspberry Pi 4 can be updated and prepared for development use. This is done with the following commands:

```
$ sudo apt update

$ sudo apt upgrade
```

Python Environment Setup on the Raspberry Pi OS

The Raspberry Pi OS has a terminal where all program files interact to the hardware system through execution. Because the codebase uses the Python language, it is installed on the OS using this command:

```
$ sudo apt install python3
```

To keep project dependencies organized and separate from the system environment, a Python virtual environment is created on the Raspberry Pi. This allowed the installation and management of project specific libraries without affecting other software on the device.

```
$ python3 -m venv <environment-name>

$ source <environment-name>/bin/activate
```

Finally, the pigpio library can be installed through the following procedures taken from the *pigpio* library's download and install page **(A.1):**

```
$ wget https://github.com/joan2937/pigpio/archive/master.zip
$ unzip master.zip
$ cd pigpio-master
$ make
$ sudo make install
```

GitHub Integration on the Raspberry Pi

Once Python is confirmed to be working on the Raspberry Pi 4, Git is set up on the OS for the microcontroller to be used as part of the routine development workflow. For it to include the project repository, the SSH protocol is prepared in the same manner as with machines with indirect access.

This allows the public repository to be cloned on the Raspberry Pi 4 to directly access and run the Python programs on the hardware without modification.

pigpio Library Functions

The pigpio library defines functions for Python modules to control the Raspberry Pi 4 and its GPIO pins. All modules within the software system employ this library to a considerable extent, including the driver program, the specialized callbacks file, and the implementations of the electronic tools. Before running the program, the *pigpio* daemon must be active so the script can connect to the Pi's GPIO resources. A daemon, or a daemon thread, is a background process that stops the system from terminating. This daemon is triggered using the pigpiod utility, and on the Raspberry Pi OS, the basic Terminal command to do so is:

```
$ sudo pigpiod
```

While there is the option to include further configurations, following 'pigpiod', this is not necessary to run the Raspberry Pi 4.

The following table lists the basic *pigpio* functions used across the codebase:

Function Name	Action	Return Type
pigpio.pi	Initialize the Pi connection; add Pi resources	pigpio Object
connected	Check Pi connection status	Boolean
set_mode	Set pin mode (GPIO, INPUT, OUTPUT, ALT)	Void
set_glitch_filter	Set glitch filter; report level changes when level is stable for specified microseconds	Void
set_pull_up_down	Set pin pull-up/down resistor (PUD_UP, PUD_DOWN, PUD_OFF)	Void
callback	Calls the parameterized function at GPIO edge detection	Callback Object
pigpio.tickDiff	Returns the difference between two ticks in microseconds	Integer
stop	Stop the Pi connection; release Pi resources	Void

Table 10. The basic *pigpio* library functions used to control the Raspberry Pi 4's basic operation and settings.

Because the return type of pigpio.pi is an object, the convention to write pigpio.pi before using other functions and constants inside the *pigpio* library is:

```
pi = pigpio.pi()
```

The object pi can be accessed when using any *pigpio* function through dot ('.') notation, such as these examples:

```
if not pi.connected():
```

Similarly, to use library constants, the library name pigpio is used instead:

```
pigpio.INPUT
```

The function set_mode requires two arguments: gpio and mode. In the case of the Raspberry Pi 4, gpio is limited to the integer range [0, 27]. The only values for mode employed when assigning a GPIO pin are pigpio.INPUT and pigpio.OUTPUT.

An important detail to add about the use of set_pull_up_down is that pigpio.PUD_DOWN is never used inside the driver program. For the rotary encoder, pigpio.PUD_UP is passed for all its pins because of the preferred pattern of keeping a high signal when no input is detected. If the rotary encoder is idle (no input), then the pull-up resistor flows current to have a high voltage output. This contrasts with the pull-down resistor, where if there is no input, then no current flows to the output (grounded).

The decision to program with GPIOs behaving as pull-up resistors is a more intuitive approach because the change in signal dramatizes the cause of bouncing, making the problem easier to solve with a short delay. This is done to the button pin of the rotary encoder by applying the set_glitch_filter, with a delay of 10,000 microseconds, or 10 milliseconds.

The mode pigpio.PUD_OFF is parameterized for both comparator pins because the SAR ADC must compare the digital-to-analog signal to the incoming voltage for the voltmeter and ohmmeter. This reasoning is expanded further in the **SAR ADC** section which introduces the implementations of the ohmmeter and voltmeter.

Rotary Encoder

The *pigpio* function callback requires three arguments: the GPIO pin, the signal edge to detect from that pin, and the function which to be executed by callback. The only situation this function is called is for the button debounce function, which is run upon detecting a pigpio.FALLING_EDGE on the rotary encoder's button pin. Because of the pull-up resistor, a falling edge indicates a changing signal and therefore a button press.

The tickDiff function is incorporated in button debouncing as the time difference between two high signals. If this value is less than the standardized debouncing delay of 200 milliseconds, then the button press is not recognized.

The set_glitch_filter function delays the reporting of a high signal until 10 milliseconds pass without change, and the tickDiff function collects two timestamps of a high signal due to the sensitivity of the rotary encoder. For button holds, debouncing is not an issue, so a simple comparison to the constrained minimum button hold length of 2 seconds is made.

SPI

SPI (Serial Peripheral Interface) is a synchronous serial protocol with a master-and-slave architecture. SPI uses four main signal lines, which are MISO (master in), MOSI (master out), CS/CE (chip select), and SCLK (clock) **(A.2)**. The master is the Raspberry Pi 4, and the slaves are the SPI devices, which are the digital potentiometers MCP4131(single) and MCP4232 (dual).

The master provides the clock (SLCK), selects the device using chip select (CS/CE), and sends command/data bytes on MOSI to set the wiper position. The Raspberry Pi 4 only needs to send commands to a digital potentiometer and not receive them, so MOSI, SCLK, and CS/CE are required. MISO is optional unless the program is reading data back.

The SPI handle is the number referencing the SPI device, which that of a potentiometer.

The following table lists the SPI functions used for communicating with the potentiometers:

Function Name	Action	Return Type
spi_open	Open the SPI handle	Void
spi_write	Write data bytes to the SPI device based on its handle	Void
spi_close	Close the SPI handle	Void

Table 11. The SPI *pigpio* library functions used to control the MCP4131 and MCP4231 from the Raspberry Pi 4.

The following table list the SPI chip select pins, their corresponding variable names, target programs, and their utilization by either MCP4131 or MCP4231.

MCP4131 and MCP4231 Pins in Software System				
Chip Select	**GPIO Pin**	**Variable Name**	**Target Programs**	**Potentiometer**
CE0	8	spi_ce0	Square+Sine Wave Generator, DC Reference	MCP4231
CE1	7	spi_ce1	Ohmmeter, Voltmeter	MCP4131

Table 12. The SPI Chip Select pins with corresponding variable names, target programs, and their assigned digital potentiometers.

I2C

I2C (Inter-Integrated Circuit) is another synchronous serial protocol with a master-and-slave architecture, however instead of four signal lines it uses two: the clock (SCL) and serial data (SDA). Because there is no chip select line, no GPIO pin is defined when establishing a connection; instead, GPIO 3 connects to SCL, and GPIO 2 connects to SDA.

An I2C device owns a unique address which the master transmits data to. Only one device, the LCD screen, requires the I2C protocol.

Like the SPI handle, the I2C handle is the number referencing the I2C device, which that of the LCD screen.

The following table lists the SPI functions used for communicating with the LCD:

Function Name	Action	Return Type
i2c_open	Open the I2C handle	Void
i2c_write_device	Write data bytes to the I2C device based on its handle	Void
i2c_close	Close the I2C handle	Void

Table 13. The I2C *pigpio* library functions used to control the LCD from the Raspberry Pi 4.

The following Terminal commands are used to detect the unique addresses found in all I2C devices connected and recognized by the Raspberry Pi 4. First, since the detection tool is contained within the i2c-tools package, install it through sudo:

```
$ sudo apt-get install i2c-tools
```

Use the detection tool, i2cdetect, with the option -y which enters the view-only mode and the I2C bus numbered 1 (I2C1):

```
$ i2cdetect -y 1
```

If successful, then the output of i2cdetect shows one or several viable addresses from the connected I2C device. Otherwise, the raspi-config settings on the Raspberry Pi OS must be rechecked to verify the I2C has been enabled.

The differences between SPI and I2C protocols can be read in **(A.2a)**.

Software – Operation

Driver Program

The software architecture of the Electronics Tool Bench brings the physical components to life and fosters the communication between the Raspberry Pi 4 and the electric circuit parts. The driver program is where the control flow begins and is distributed across the rest of the tools to be controlled by the user. This means that it houses the necessary imports to operate from a superficial level, making the user interface flexible to any expansion and easy to add or modify menu screens.

Before the software system can boot up the hardware and activate the LCD screen, safety measures should be done to confirm that all modules have been terminated, and the LCD's display is shut down. The additional states designated for the function (square/sine) generator and the DC reference voltage outputs are assigned to false from the state dictionary, which stores all shared states while the program is running. These measures are followed by the function ensure_all_off, the first block of code to be run in the entire software system.

Then, the top screen is to be displayed on the LCD. The LCD screen's backlight and display address are enabled, and a preliminary menu page appears. This page shows a title, "System",

and the options list containing “OFF” and “Mode Select”. Selecting “OFF” stops any currently remaining processes and switches off the shared states for voltage output, while selecting “Mode Select” transfers to the main menu.

Each page can be decomposed into two components which consist of the title and options. The options are stored inside of a list for the user to feasibly navigate through. All electronic tools for the Tool Bench are choices from the main menu that store their individual menu pages. This is best reflected in the main loop, a while loop that cycles through each choice for potentially running their branches of pages on select.

```
print("Starting...")
ensure_all_off()
try:
    while True:
        run_top_screen()

        while True:
            choice = pick_menu(
                "Mode Select",
                ["Function Generator", "Ohmmeter", "Voltmeter", "DC Reference", "Frequency Meas.", "Back", "Main"],
            )

            if choice == "Function Generator":
                result = run_function_generator_menu()

            elif choice == "Ohmmeter":
                result = run_ohmmeter()

            elif choice == "Voltmeter":
                result = run_voltmeter_menu()

            elif choice == "DC Reference":
                result = run_dc_reference_menu()

            elif choice == "Frequency Meas.":
                result = run_frequency_measurement()

            elif choice in ("Back", "Main"):
                break  # go back to top screen

except KeyboardInterrupt:
    print("\nStopping...")
```

Figure 17. The complete main loop inside the driver program. When the ensure_all_off function is ran to verify that all tools have been turned off prior to a new session, the top screen is displayed before the main menu is shown to the user. Each choice contains its own series of menu pages that can be interpreted as branches of the user interface tree.

State Dictionary

The component with all the variables that together controls the user interface logic is the state dictionary. This includes the states for the square/sine wave generator and DC reference, since their menus require the user to first specify the characteristics of the voltage signals before switching the output on and off. This contrasts with the other tools whose menus execute the necessary changes on confirmation at the same menu, so they do not need states inside the dictionary.

```
state = {
    'active_callbacks': [],
    'button_last_tick': None,
    'button_press_tick': None,
    'encoder_delta': 0,
    'button_pressed': False,
    'long_press': False,

    # Function generator
    'fg_freq': 1000,
    'fg_amp': 0.0,
    'fg_output_on': False,

    # DC Reference
    'dc_voltage': 0.0,
    'dc_output_on': False,
}
```

Figure 18. The state dictionary inside the driver program contains overhead states that can change at any menu page. The states for the function generator and the DC voltage reference are also specified because the menus are set up so that the user adjusts and enables/disables the voltage outputs on separate pages.

The LCD display is limited to four lines, so each page establishes the string values to draw on the screen. All pages follow the same, if not a similar structure as the main menu page. They have “BACK” and “MAIN” options that each switch to the previous page and the main menu page respectively. On each selected choice, the program makes the needed changes to the state dictionary through direct access.

Running Measurements and Outputs on Menu

A measuring tool, which are the voltmeter, ohmmeter, and the sine wave frequency measurement, contains multiple pages that branch off into different options. Similarly, there are tools that deliver some output commanded from the Raspberry Pi 4’s software system, which are the DC reference and function generator.

Based on the user interface tree, these two classifications of electronic tools share groupings of lines, including the aforementioned “BACK” and “MAIN” lines. One observation from this tree is that all the measuring tools have pages where the output goes through a live update with the “BACK” and “MAIN” lines being the only options. It makes sense to define a function that creates this page by drawing the lines on the LCD.

Another observation is that the measuring tools require adjusting some value by turning the knob and pressing the button on the rotary encoder. These two functionalities are also defined as separate functions.

This means that all menu pages for the measuring tools are standardized to have three functions, which are shown in the following table.

Measuring Tools Standardized Menu Functions	
Function Name	**Action**
_draw_nav	Draws a page with a live-updated measurement with the options “BACK” and “MAIN”
_on_rotate	Callback for rotating the knob for adjusting a value

_on_button	Callback for pressing the button on confirmation of the set value

Table 14. The table portraying the three functions defined within the menu pages of the measuring tools: voltmeter, ohmmeter, and the sine wave frequency measurement.

The case for the tools that deliver some output voltages is trickier to situate because of the scopes of the states which they require before the outputs are created. The function generator requires frequency and amplitude, whose states are stored inside the state dictionary. The DC reference requires the type of voltage source (internal or external), a local state inside the voltmeter module, and the voltage value, a global state inside the state dictionary.

The DC reference menu is stored inside the voltmeter module instead of the driver program because of the dependency the voltmeter has to this tool to attain accurate measurements. From this architecture, this means that the DC reference menu may use _on_rotate and _on_button, and the function generator does not use any of the menu functions.

The function _on_rotate updates the encoder_delta state to a version with the direction subtracted from it. This direction variable can be either positive for clockwise rotations or negative for counterclockwise rotation.

The debounce button constant DEBOUNCE_US, is 0.2 seconds, and its importance is explained in the **Button Debouncing** subtopic under **Rotary Encoder**.

The next function, _on_button, ensures that the button pin signal is low, indicating that the button is pressed. Once it grabs the timestamp of the most recent tick, it performs a debounce check with DEBOUNCE_US. Lastly, the global states for this timestamp and the flag for a button press are updated such that the button is confirmed to be pressed.

Both _on_rotate and _on_button serve as callbacks for the decoder and the button pin GPIO respectively. Since button presses are detected when the signal is low, the program seeks a change from a high to a low signal. The active callbacks array is then updated accordingly.

```
def _on_rotate(direction):
    state['encoder_delta'] = state.get('encoder_delta', 0) - direction

def _on_button(_gpio, level, tick):
    if level != 0:
        return
    last = state.get('button_last_tick')
    if last is not None and pigpio.tickDiff(last, tick) < DEBOUNCE_US:
        return
    state['button_last_tick'] = tick
    state['button_pressed'] = True

decoder = rotary_encoder.decoder(pi, PIN_A, PIN_B, _on_rotate)
cb_btn = pi.callback(ROTARY_BTN_PIN, pigpio.FALLING_EDGE, _on_button)
state['active_callbacks'] = [decoder, cb_btn]
```

Figure 19. The _on_rotate and _on_button methods are implemented on menus of all measurement tools, including that of the DC reference, which lives inside of the voltmeter module to bridge both tools' duality. This block of code taken from run_frequency_measurement inside the driver program for measuring the sine wave's frequency.

The _draw_nav function draws a section of a page spanning two rows, or indices, of the LCD. The nav_idx value is checked so that the arrow is points at either "Back" (nav_idx = 0; index = 2 (third line)) or "Main" (nav_idx = 1; index = 3 (fourth line)). The rest of the page is filled up depending on the measurement value that is being retrieved.

```
freq_meter = FrequencyMeter(pi)

def _draw_nav():
    if nav_idx == 0:
        lcd.put_line(2, ">Back")
        lcd.put_line(3, " Main")
    else:
        lcd.put_line(2, " Back")
        lcd.put_line(3, ">Main")

lcd.put_line(0, "Freq Measurement")
lcd.put_line(1, "Waiting...")
_draw_nav()
```

Figure 20. The section of code within the function run_frequency_measurement which initializes the frequency meter class and defines a shorthand for drawing the navigation portion displaying “Back” and “Main”. The index nav_idx points the arrow to either selection.

Updating Measurements

Inside a loop before the screen is drawn on the screen, a timestamp variable last_update is declared for managing a steady occurrence of updates to a measured value. The loop contains a series of disjointed conditional statements. The first statement checks the value of encoder_delta, which stores the intensity of the turning movement done on the rotary encoder. If there is movement, then the navigation section of the screen is updated.

Additionally, if the button is pressed on either “BACK” or “MAIN”, then the loop terminates with the program escaping the measuring page. And, if at least 0.5 seconds have passed since the last_update, then it retrieves and updates the value. At the end of the loop, button_pressed is assigned to false and all callbacks are cleared.

```
last_update = 0.0
try:
    while True:
        delta = state.get('encoder_delta', 0)
        if delta != 0:
            nav_idx = (nav_idx + delta) % len(nav_options)
            state['encoder_delta'] = 0
            _draw_nav()

        if state.get('button_pressed'):
            state['button_pressed'] = False
            return "BACK" if nav_idx == 0 else "MAIN"

        now = time.time()
        if now - last_update >= 0.5:
            last_update = now
            freq = freq_meter.get_frequency()
            if freq > 0:
                lcd.put_line(1, f"{freq:.1f} Hz")
            else:
                lcd.put_line(1, "No signal")

        time.sleep(0.05)
finally:
    freq_meter.cleanup()
    state['button_pressed'] = False
    clear_callbacks(state)
```

Figure 21. The loop demonstrating an updating measured value is contained within the run_frequency_measurement, which snatches an updated frequency value every 0.5 seconds. The first conditional statement uses the encoder_delta state to change the selection under the navigation section.

Running the Function Generator

For the function generator, an object that shares the setter functions of both square and sine waves is created. This is based on the fg_type state that was determined while choosing to either generate a sine wave or a square wave. Once these are set and adjusted, the tool starts operation and transitions to a screen indicating that the output is turned on. When the button is pressed to return to the previous screen, the generator stops, marking the fg_output_on state to false.

Running DC Voltage Output

The DC reference voltage have two options to provide voltage from: an internal and external source. When the user selects “Internal Reference”, the program brings the user to a branch of the user interface tree prompting either “Voltage Value Input” or “Output”. When the user selects

“External”, it runs the measurement page from the voltmeter module which resembles other functions defined in the driver program, such as **Running Frequency Measurement.**

Within “Internal Reference” on option “Voltage Value Input”, the program immediately adjusts the voltage, which goes further in detail in **Adjusting Values** section under the **Rotary Encoder.** But, selecting “Output” runs the DC output while reading the voltage value while its corresponding page is open. The navigation to this page demonstrates that all branches of the user interface are prescribed constraints which have been thoroughly followed.

Callbacks Helper File

The callbacks helper file contains the concrete implementations of several tasks that are relied on by the driver program to ensure a feasible user experience. These include inserting menu options and scrolling capabilities for each page and describing the control logic for the rotary encoder and the LCD. If all its functions are not separated from the driver program, then readability for its source code is strained and makes debugging unnecessarily challenging.

While the driver file contains specific state logic for each individual branch of the user interface tree, the callbacks file contains general code that draws on the LCD and tracks rotary encoder turns, button presses and holds. For each menu change or select values that can be adjusted, the callbacks file updates the active callbacks of the state dictionary alongside other states.

The setup_callbacks function initializes the global states that the rest of the methods rely on. There are three global states to be used across the file: _s, the state dictionary, _pi, the Raspberry Pi 4 object from the pigpio library, and _lcd, the LCD object from its dedicated file, the i2c_lcd.py. Similarly, as the name suggests, the clear_callbacks function goes through the entire active callbacks array to cancel all callbacks and remove them from the array.

Drawing the Menu

The callbacks file contains a function that is directly called from the driver file, which is draw_menu. A menu screen is a scrollable screen that can fit four lines at a time. A typical menu screen has a title on the first line and a row of options from the options list. Any option is preceded by a ‘>’ symbol, an arrow that points to the current option, or a whitespace ‘ ‘ character.

start_row is the index to start drawing lines to the end of the LCD. If the title is not null, then it is drawn first, and start_row is moved from 0 to 1. The remaining available rows is the difference between 4 and the start_row index. The index to start drawing the remainder of the window, or w, is found through the following conditions:

$$l \leq r : w = 0$$

$$l \leq r : w = \max(0, \min(i - 1, l - r))$$

This is where l is the number of options to choose from, r is the remaining number of rows to draw, and i is the index where the '>' is drawn. The window index applies to unreserved lines to freely draw on; the title would be on a reserved line and cannot be overwritten, so index 0 is the immediate line below it. The function can enter a loop which draws each option, including the '>' symbol, to lines that are scrollable.

```
def _draw_menu(title, options, idx):
    """Render a scrollable 20x4 LCD menu. Line 0: title, lines 1-3: options."""
    start_row = 0
    if title:
        _lcd.put_line(0, title)
        start_row = 1

    max_rows = 4 - start_row
    n = len(options)
    if n <= max_rows:
        window_start = 0
    else:
        window_start = max(0, min(idx - 1, n - max_rows))

    for row in range(max_rows):
        i = window_start + row
        if i < n:
            prefix = ">" if i == idx else " "
            _lcd.put_line(start_row + row, prefix + options[i])
        else:
            _lcd.put_line(start_row + row, "")
```

Figure 22. The draw_menu function contains the procedures for preparing a screen with selectable options, placing the '>' symbol based on the current index that the user lands on and displays the options in their respective rows as passed as an argument from the driver program.

Picking Options Menu

Another callbacks helper function that the driver file can directly call is pick_menu. This function is responsible for displaying a screen that contains one or multiple options for the user to choose from. These options are specified inside the driver program, and each one may transfer to another screen or enable/disable a specific feature from the user interface. The options menu is navigable by turning the rotary encoder.

Because the options are contained inside of a list, each of them has an index, which starts at zero, or the top of the screen. The routine procedures of resetting input flags, attaching input callbacks, and drawing the menu with a title and options are done to prepare the screen.

When there is a rotation, the index is changed as such:

$$index = (index + \partial) \,\%\, l$$

The variable ∂ is the value of the state encoder_data, and l is the number of options to choose from. The modulus % is a precaution to prevent the program from crashing, for any situation where the knob is turned very quickly. The encoder_data is reset to zero because when a new screen is written by the current selection, no rotation is being done. This function returns the choice within the options list that is selected by the user on a button press, and all callbacks are cleared.

```
def pick_menu(title, options, start_idx=0):
    """Generic rotary menu.  Returns the selected option string."""
    idx = max(0, min(start_idx, len(options) - 1))
    _reset_input_flags()
    _attach_input_callbacks()
    _draw_menu(title, options, idx)

    try:
        while True:
            delta = _s.get("encoder_delta", 0)
            if delta != 0 and options:
                idx = (idx + delta) % len(options)
                _s["encoder_delta"] = 0
                _draw_menu(title, options, idx)

            if _s.get("button_pressed"):
                _s["button_pressed"] = False
                return options[idx]

            time.sleep(0.02)
    finally:
        clear_callbacks(_s)
```

Figure 23. The pick_menu function contains the functionality for scrolling through a menu page with selectable options, and the loop that cycles over the options depending on the value of encoder_delta, changing the index number up or down the menu. It returns the selected option, otherwise it draws a new screen with an added line above or below the previous one.

Rotary Encoder

The callbacks file allows the driver program to keep a high-level on the user interface by allocating the responsibilities of detecting rotary encoder movements and making menu page scrolls and transitions. The following table shows the three rotary encoder pins which detect input movements to handle them in the software.

Rotary Encoder Pins in Software System		
Pin	**Constant Name**	**GPIO Pin**
CLK	PIN_A	22
DT	PIN_B	27
SW	ROTARY_BTN_PIN	17

Table 15. The pins of the rotary encoder along with their constant names within the callbacks file and their assigned GPIO pins.

To best understand how the callback file interacts with the rotary encoder, this table shows the five states inside the dictionary that controls the rotary encoder.

Rotary Encoder States			
State Name	**Action**	**Initial Value**	**Return Type**
button_last_tick	The timestamp of the high signal for a valid button press	None	Float
button_press_tick	The time elapsed for a button press	None	Float
encoder_delta	The rotation distance detected by PIN_A and PIN_B	0	Float
button_pressed	Flags if the button was pressed	False	Boolean
long_press	Flags if the button was held for 2 seconds or longer	False	Boolean

Table 16. The pins of the rotary encoder along with their constant names within the callbacks file and their assigned GPIO pins.

The following functions are not imported into the driver program but instead serve as helper functions for when the user is adjusting and confirming values or browsing options for a menu page. First, the function _reset_input_flags reset states encoder_delta, button_pressed, and button_last_tick to their initial values. Second, the function _attach_input_callbacks clear the active callbacks array and insert the knob rotation and button presses into it.

```
def _attach_input_callbacks():
    clear_callbacks(_s)
    decoder = rotary_encoder.decoder(_pi, PIN_A, PIN_B, _on_rotate)
    cb_btn = _pi.callback(ROTARY_BTN_PIN, pigpio.FALLING_EDGE, _on_button)
    _s["active_callbacks"] = [decoder, cb_btn]
```

Figure 24. The _attach_input_callbacks function clears all callbacks and defines the decoder, the class that decodes rotary input, and callback to a button press. This callback looks at the falling edge because the signal moves from high to low on a button press.

Adjusting Values

The instructions to change or verify states when a rotary encoder is used to set values for a non-measuring tool is under the function adjust_value. Therefore, it is directly called by the driver program. The constants BUTTON_DEBOUNCE_US and HOLD_US correspond to the debounce and button hold times, which are 0.2 and 2 seconds respectively.

When a menu page is entered for adjusting, all input flags are reset and all callbacks are cleared. Then, the rotation debouncing (decoder) and button press callbacks are defined and inserted into the active callbacks array. Next, the LCD screen is drawn instructing the user to turn the rotary encoder to adjust the value, and to hold the button to go back to the previous menu screen.

The function enters the user on a loop which looks for a rotation, button press, and long press. For rotation, it relies on encoder_delta to change the displayed value based on the slow or fast turning. The value, x, first goes through $x = x + s \cdot \partial$, where s is +/-10 in slow rotation and +/-100 in fast rotation, and ∂ is the value of encoder_data. It is then clamped within the constrained

minimum and maximum values dictated by each electronic tool. Finally, the value is changed like so:

$$x = round(round(\frac{x}{s}) \cdot s)$$

The fraction $\frac{x}{s}$ is rounded to the nearest integer and is multiplied with the change in slow or fast rotation. As a safety measure, if this calculation fails, then a slow rotation is assumed for the value to change by only +/-10. The value is ready to be displayed on the screen.

The user can stop adjusting the value on either a button press to confirm the selected value, or on a long press to return to the previous screen. Nonetheless, when the user leaves the screen, all callbacks are cleared.

```
try:
    while True:
        delta = _s.get("encoder_delta", 0)
        if delta != 0:
            value += step * delta
            value = max(min_val, min(max_val, value))
            value = round(round(value / step) * step, 10)
            _s["encoder_delta"] = 0
            _lcd.put_line(1, fmt_fn(value))

        if _s.get("button_pressed"):
            _s["button_pressed"] = False
            return value

        if _s.get("long_press"):
            _s["long_press"] = False
            return None

        time.sleep(0.02)
finally:
    clear_callbacks(_s)
```

Figure 25. The loop inside the adjust_value function, which changes the LCD display due to the slow or fast rotations from the encoder. The step variable corresponds to the +/-10 for slow rotations and +/-100 for fast rotations, and delta corresponds to the change in distance rotation seen by both pins for detecting knob spins. The rotary encoder states are changed accordingly in the loop, and active callbacks are cleared when the user exits the screen.

Running Output

Whenever an output voltage is being run, the LCD displays a screen that waits for the user to return to the previous screen. This pertains to the function generator and DC reference, two electronic tools that generate some voltage output. The task to transition to the previous screen is handled by the driver program since either electronic tool have prescriptive user interface branches.

The procedure starts by first resetting input flags and attaching input callbacks, next drawing the screen, and finally entering a while loop waiting for a button press. If the state dictionary indicates that the button has been pressed, then it updates it to false and clears all callbacks.

```
def wait_for_back(line_fn):
    """Display 4 lines and wait for a button press to return.

    line_fn() should return a tuple/list of 4 strings.
    """
    _reset_input_flags()
    _attach_input_callbacks()

    lines = line_fn()
    for row, text in enumerate(lines[:4]):
        _lcd.put_line(row, text)

    try:
        while True:
            if _s.get("button_pressed"):
                _s["button_pressed"] = False
                return
            time.sleep(0.02)
    finally:
        clear_callbacks(_s)
```

Figure 26. The function which enacts a loop that waits for a button press. This is used for screens that display a live voltage output generated by an electric tool, either a function generator or a DC reference. The function that this procedure is stored within is wait_for_back.

Button Debouncing

The encoder pin ROTARY_BTN_PIN detects a press if the button goes low, and a release if the button goes high. The constant BUTTON_DEBOUNCE_US is preset to 0.2 seconds, or the minimum threshold between two ticks before confirming the button press. This elapsed time is an invalid button press if it falls short of this threshold. The helper function_on_button deals with this logic, and, on a valid button press, reassigns the button_last_tick state and the button_pressed state.

Rotation Debouncing

Inside the codebase, a separate program extends from the callback file by specializing in rotation debouncing for the rotary encoder. The encoder pins PIN_A and PIN_B detect rotations where one pin is activated before the other. Both pins look for both rising and falling edges.

Both pins must be high after an ordered sequence of changes to high signals for a valid rotation, which has direction. If PIN_A is high before PIN_B, then the direction is counterclockwise, and if PIN_B is high before PIN_A, then the direction is clockwise.

The program stores a decoder class which stores this logic. It initializes PIN_A and PIN_B with their GPIOs and assigns their initial levels to 0; since both pins look for rising and falling edges, their respective callback objects, cbA and cbB, are also initialized with both GPIOs seeking pigpio.EITHER_EDGE. These objects pertain to the function within the class which reports clockwise or counterclockwise directions to self.callback, the function that requires a rotation from rotary encoder input.

Finally, self.lastGpio is the GPIO pin that most recently observed a changing edge. Before direction is determined, it is compared to the parameterized GPIO pin to enact debouncing for ignoring sudden or rapid signal changes.

```
def __init__(self, pi, gpioA, gpioB, callback):
    self.pi = pi
    self.gpioA = gpioA
    self.gpioB = gpioB
    self.callback = callback

    self.levA = 0
    self.levB = 0
    self.lastGpio = None

    self.cbA = self.pi.callback(gpioA, pigpio.EITHER_EDGE, self._pulse)
    self.cbB = self.pi.callback(gpioB, pigpio.EITHER_EDGE, self._pulse)
```

Figure 27. The __init__ function of the decoder class initializes the GPIOs, their initial levels, and callback objects which assign them to detect either rising or falling edges. Moreover, self.callback necessitates any rotary input form the encoder and thus is a parameter for the class.

Both cbA and cbB store a method that waits for either edge to occur for determining direction. This function is suitably named _pulse because it focuses on the pulses of both GPIOs for PIN_A and PIN_B. Before evaluating direction, each time a GPIO sees a rising or falling edge, the method updates the current state of either PIN_A or PIN_B, which can be either high or low.

Since direction is only determined with two different pins, this GPIO must be different from self.lastGpio. Therefore, self.lastGpio is reassigned to the GPIO that saw an edge which is recorded as the most recent pin that underwent this event. This check successfully does debounce by ensuring that the same pin is not listened more than once. As a result, PIN_A and PIN_B always respond sequentially without disruption from tiny signals.

The direction is determined when both GPIOs reach a high level. To reiterate, if PIN_A becomes high first, the counterclockwise direction is parameterized as a value of -1 to some callback function. Conversely, if PIN_B becomes high first, the clockwise direction is parameterized as a value of 1 to some callback function.

```
def _pulse(self, gpio, level, tick):
    if gpio == self.gpioA:
        self.levA = level
    else:
        self.levB = level

    if gpio != self.lastGpio:  # debounce
        self.lastGpio = gpio
        if gpio == self.gpioA and level == 1:
            if self.levB == 1:
                self.callback(1)
        elif gpio == self.gpioB and level == 1:
            if self.levA == 1:
                self.callback(-1)
```

Figure 28. The function _pulse accepts a GPIO belonging to either PIN_A or PIN_B and updates the level. The debounce check is then enacted and the direction of either clockwise or counterclockwise is passed into some callback function which requires rotary encoder movement. Both directions' associated values are 1 and -1 respectively.

The callbacks file stores a function called _on_rotate, which updates the encoder_delta state based on the returned direction value from the decoder class.

LCD

The LCD is integrated into the software system as a class with various methods for initializing it through its I2C handle and for writing text into the lines of the screen. The Raspberry Pi 4 must establish connection with the LCD, and this is verified using the program i2cdetect to scan the I2C bus for occupied addresses. All addresses to send commands and flags to are 8-bit, and the correct ones may be specified as parameters for direct communication.

The *pigpio* library provides example Python code that establishes the methods targeting the proper addresses **(A.2b)**, and it is compatible with a 20x4 formatted display. This file is written by joan2937 to illustrate example code for the library and is licensed under the public domain.

All bytes are transmitted to the LCD by the _inst and _data methods for sending instructions and data. The LCD is written by a cursor situated at a row and column of the LCD, and instructions

may be sent through the _inst method to reposition it. This is to insert data, which includes symbols, characters, and strings that can be displayed on a line of the LCD display. Other instructions can be sent to special addresses if they are arguments to the _inst method.

_init Address Parameters	
Address (Hex)	**Actions**
0x33	Initialize 1
0x32	Initialize 2
0x06	Cursor increment
0x0C	Display on/off
0x28	4-bits, 1 line
0x01	Clear display

Table 17. The table with the common parameter addresses for _init and the actions that they execute.

The method which employs the relocation of the cursor is move_to, and the clear method clears the whole LCD display. Both methods use the _inst method and are typically used inside functions that send entire data to display.

The list _LCD_ROW is representative of the LCD display with four 8-bit addresses for the four rows of the LCD. The cursor can traverse along the rows and columns within this array of addresses.

_LCD_ROW	
LCD Row Number	**8-Bit Adress (Hex)**
1	0x00
2	0xC0
3	0x94
4	0xD4

Table 18. This table shows the four 8-bit addresses associated with the four lines of the LCD display.

With the _inst method, this sends the instructions to relocate the cursor by a determined row and column, with the assumption that self is the LCD object:

```
self._inst(self._LCD_ROW[row] + column)
```

The _data method makes it possible to draw symbols, characters, and strings for the LCD. The parameter is an index value, which is the position of the cursor with the combined information about row and column.

LCD Methods Using the _data Method	
Method	**Action**
put_symbol	Writes a symbol where the cursor is
put_chr	Writes a character where the cursor is
put_str	Writes a string where a cursor is
put_line	Replaces an LCD row with a new string

Table 19. The methods defined inside of an LCD class which are used to directly write into the LCD screen by importing the class into a file which utilizes the I2C handle for display.

```
def _inst(self, bits):
    """
    Send instruction byte.
    """
    msn = (bits >> 4) & 0x0F
    lsn = bits & 0x0F

    MSb = msn << self.B4
    LSb = lsn << self.B4

    self._byte(MSb, LSb)
```

Figure 29. The _inst method sends one byte to the LCD and allows the Raspberry Pi 4 to relocate the cursor for putting data that can be shown on the LCD.

```
def _data(self, bits):

    MSN = (bits >> 4) & 0x0F
    LSN = bits & 0x0F

    MSb = (MSN << self.B4) | self.RS
    LSb = (LSN << self.B4) | self.RS

    self._byte(MSb, LSb)
```

Figure 30. The _data method is a similar method to the _inst method, except that it puts in the symbols, strings, and lines for display.

One important feature the class demonstrates is its attempt to try multiple addresses when no address is passed through the initialization method. If a connection works with an address inside the I2C bus, then the Raspberry Pi 4 can detect this from its end and can send byte instructions to the LCD.

```
if addr is None:
    candidate_addrs = [
        0x27, 0x26, 0x25, 0x24,
        0x23, 0x22, 0x21, 0x20,
        0x3F, 0x3E, 0x3D, 0x3C,
        0x3B, 0x3A, 0x39, 0x38
    ]
else:
    candidate_addrs = [addr]
```

Figure 31. If no address is passed as a parameter for creating an instance of the LCD class, then it attempts these candidate addresses which can be seen using the i2cdetect utility tool from the Raspberry Pi OS Terminal.

The close method prompts the cursor to clear the screen and close its I2C handle, effectively shutting off the display and the entire device.

Successive Approximation Method

An analog-to-digital converter (ADC) is crucial for the Raspberry Pi 4 to interpret voltage outputs while measuring an unknown voltage or resistance value from a voltage divider. There are many ADC topologies to implement which all necessitate the weaving of hardware and software systems depending on preferences for higher bit resolution, potent signal filtering, and fast performance.

The SAR ADC is topology that became incorporated into the software logic. This involves the alternating input signal simplified through a sample-and-hold (S/H) circuit to become the first input of a singular comparator. The other input is each successive bit from the SAR that underwent the digital-to-analog conversion (DAC). The comparator outputs either a high or low signal by comparing the two inputs, and the hardware logic is laid out as follows:

$$V_{DAC} < V_{in} \rightarrow V_{out} = HIGH\ (1)$$

$$V_{DAC} > V_{in} \rightarrow V_{out} = LOW\ (0)$$

In these arguments, V_{DAC} is the output of the DAC from the SAR, V_{in} is the changed input signal after the S/H circuit, and V_{out} is the output of the comparator and the resulting voltage of the SAR ADC. The comparator pins are laid out so that V_{in} is the noninverting input, and V_{DAC} is the inverting input. A $HIGH$ output means a bit is deleted, while a LOW output means a bit is kept. The kept bits are summed to the digitalized output as it feeds back into the SAR to await the next bit value.

Many popular ADCs sum multiple voltages values together to produce a digitalized output. However, this output is limited by the voltage reference, which is constrained to 5V for the Electronics Tool Bench. This means that the output cannot exceed the voltage reference value fed into the converter. Moreover, the output is limited by the bit resolution which depends on the hardware configuration or the preestablished bit size of the integrated circuits used for an ADC design.

A flash ADC is one that uses a series of resistor dividers where each output voltage is fed as an input to a comparator, with another being some analog voltage **(A.3)**. The comparators output a high or low signal to be interpreted through digital encoder for the target integrated circuit. This conversion is one-way, making this method very fast compared to other ADCs, and it can filter noise or distortions from the original signal.

An integrating ADC is one that changes an alternating signal, such as a sine wave, into digital signals with greater precision than a flash ADC. A time interval is selected to find the area bounded by the wave and the horizontal axis, or the output of the integrator, which is converted into digital form **(A.3a)**. They tend to produce smooth outputs at high resolutions and filtered noise, and its speed and resolution can be improved by including decreasing integrator outputs into the calculation.

Both these topologies have their set of disadvantages, but their application in hardware and software were considered to end up not using neither one. Flash ADC is expensive, requiring many comparators with each one costing one GPIO pin. An 8-bit resolution is possible with two quad-comparators but with the use of 8 GPIOs. Meanwhile, the integrating ADC has tradeoffs between conversion speed and resolution, making it difficult to model and simulate a design that balances both characteristics.

The SAR ADC is one that is relatively easy to program and is the most straightforward to directly apply to a low resolution MCP4131, an SPI device which converts the bits from the SAR into analog. Although this topology converts in moderate speeds and resolution, it only requires an SPI pin and a GPIO pin to read comparator outputs for a tool using the ADC. This design proves to be extendable to any new file in the system thanks to its overhead codebase.

Software Implementation

The SAR ADC is used by the ohmmeter, voltmeter, and DC voltage reference to acquire an unknown measured resistance and voltage value respectively. The Raspberry Pi 4 plays an administrative role as the SAR, which analyzes each bit and writes to the DAC, the MCP4131, depending on its value.

The constraint to the software implementation is the resolution of the signals, which must be a minimum of 5-bits; the programmed resolution for the Electronics Tool Bench is exactly 5 bits. The SAR must have an established settle time after writing the digital value into the MCP4131, which is 20ms, so that it does not overwrite or duplicate the wiper readings and output an incorrect voltage signal.

Though the SAR logic is identical, the ohmmeter and voltmeter need separate comparator pins to determine whether to keep or delete the bit, which are GPIO 24 and GPIO 23 respectively. The DC voltage reference also uses GPIO 23 and shares it with the voltmeter.

First, the program goes through the nonnegative bit values [0, 4] in descending order. It assigns a variable, trial, to the combination of the current step value and the current bit's next significant bit. Once trial is written to the DAC, it sleeps for 20 milliseconds. If the GPIO signal is low, then keeps the bit with step to the trial variable.

```
def sar_measure(pi, spi_handle, comp_pin):
    """5-bit SAR: comp == 0 keeps bit, comp == 1 discards."""
    step = 0

    for bit_pos in range(4, -1, -1):
        """The trial is the step value plus the next significant bit."""
        trial = min(step | (1 << bit_pos), MCP4131_MAX_STEPS)
        _write_dac(pi, spi_handle, trial)
        time.sleep(_SETTLE_S)

        """The comp_pin read determines keep (0) or delete (1)."""
        if comp == 0:
            step = trial

    """Write the final step value to convert to analog."""
    _write_dac(pi, spi_handle, step)
    return step
```

Figure 32. The SAR implementation writing the depending on the reading of the GPIO pin which corresponds to the output of the comparator, V_{out}. This is done under the method _sar_measure, which combines the current step value with the next significant bit while staying within the 5-bit restriction.

The program writes to the DAC by checking if the step is within the bounds of 0 and 31, which is $2^5 - 1$. Then, because an MCP4131 is a 7-bit device, the step value is updated from its 5-bit form to its 7-bit form, which is then written to the MCP4131.

```
def _write_dac(pi, spi_handle, step):
    """Scale 5-bit step (0..31) to MCP4131 7-bit register DAC (0..127)."""
    step = max(0, min(step, MCP4131_MAX_STEPS))
    dac_code = round(step * 127 / MCP4131_MAX_STEPS)
    pi.spi_write(spi_handle, [0x00, dac_code])
```

Figure 33. The method _write_dac converts the resulting step value for a given bit from 5-bit to 7-bit, because the SAR is 5-bit and the MCP4131 is 7-bit.

The program can reduce noise of the resulting step value written to the DAC by rearranging an arbitrary number of measured GPIO readings in ascending order. The middle value is approximate to the voltage value that to the readings revolve around.

```
def _averaged_measure(pi, spi_handle, comp_pin, n=11):
    """Return the median step from n SAR conversions."""
    readings = sorted(_sar_measure(pi, spi_handle, comp_pin) for _ in range(n))
    return readings[n // 2]
```

Figure 34. The method _averaged measure returns the median step from an input number of SAR readings, which is set to 11. This is done to have smoothed out and accurate GPIO readings for transmitting to the DAC.

Ohmmeter

The ohmmeter is housed within its own file and uses the compare pin GPIO 24 for the SAR. The program converts a 5-bit step value from an MCP4131 into the target resistance's value, and the step values are inversely proportional to resistance. From repeated experimentation and subsequent adjustments to the hardware, a reference resistor of 2kΩ is chosen.

The constraints for the Electronic Tool Bench enforce the maximum measurable resistance to be 10kΩ, which is defined as a constant. The minimum is also defined as a constant as 500Ω. Furthermore, the constant MCP4131_MAX_STEPS specify that the SAR resolution of 5-bits ($2^5 - 1 = 31$.

The target margin of error is defined as a constant as 2% for LCD display. The constraints for this section describe a calibration table to calibrate certain measured readings to their target readings. In a process called autoranging, the program selects an appropriate range of Ohm values from this table without extensive measuring settings from the user.

Calibration Points	
Step	**Target Resistance (Ω)**
5	9,830.0
6	6,690.0
7	6,148.0
8	5,109.0
11	3,551.8
12	2,947.7
13	2,644.2
16	1,793.3
17	1,584.5
20	991.3
23	673.7
25	494.1

Table 20. The calibration table showing the calibration points required for the ohmmeter program to have more precise resistor readings. The measured resistance values were found experimentally through running and debugging the program.

The program defines the unknown resistance value, R_x, of a voltage divider of two resistors, with the other being a reference resistor, R_{ref}, which is 2000Ω. The calibration table is used to capture the step value from the MCP4131 and convert it into resistance. This data containing the steps and their corresponding resistance values were found through multiple trials collecting each pair using the R_{ref} as part of the voltage divider.

The conversion process begins with the evaluation for resistance values beyond the maximum resistance of 10kΩ. Because lower step values yield higher resistance values, any step value below 5 are assumed to yield an infinite resistance, which is an open circuit. Next, the program loops over the table to attempt to identify the desired step value and return its corresponding resistance value.

If the step value does not match those in the calibration table, then it needs to be calibrated through autoranging. This works through a piecewise-linear function, a mathematical approach that coordinate values between two points and is needed to predict a target resistor value to the parameterized step value.

If the step values are in the x-axis and the target resistor values are on the y-axis, then it creates a linear graph showing their inverse proportionality. The program loops over the table once more to collect the two surrounding step values that the parameterized step value is in. For example, if the step value is greater than 17 but less than 20, then autoranging happens between 1584.5Ω and 991.3Ω. The program is formatted so that the calibration table is flexible to contain new pairs of measured and target resistance values.

If the step value is not within any range inside the calibration table, then the program returns the autoranging result between the second-to-last and the last step value to prevent the program from having unexpected behavior.

```
def calibrate_step_to_resistance(step):
    """
    Convert SAR step directly to resistance using measured data.

    Behavior:
    - steps lower than the first calibration point act like open circuit
    - values between calibration points are linearly interpolated
    - values beyond the last calibration point are extrapolated
    """
    pts = sorted(STEP_CAL_POINTS)

    # Open circuit region
    if step < pts[0][0]:
        return float('inf')

    # Step values inside the calibration table
    for s, r in pts:
        if step == s:
            return r

    # Step values between known steps do linear interpolation
    for i in range(len(pts) - 1):
        s0, r0 = pts[i]
        s1, r1 = pts[i + 1]

        if s0 <= step <= s1:
            return _interp(step, s0, r0, s1, r1)

    # Step values between last 2 known steps
    s0, r0 = pts[-2]
    s1, r1 = pts[-1]
    return _interp(step, s0, r0, s1, r1)
```

Figure 35. This function finds the suitable range to do linear interpolation using the calibration table for autoscaling a resistance value. This is stored under calibrate_resistance, which is called by step_to_resistance for calculating the tolerance.

The resistance value is assigned as x to be used in the formula for linear interpolation. Its y-coordinate, the calibrated resistance, is found as follows, if $x_1 \neq x_0$:

$$y = y_0 + \frac{(x - x_0) \cdot (y_1 - y_0)}{(x_1 - x_0)}$$

```
def _interp(x, x0, y0, x1, y1):
    """Linear interpolation for predicting the correct resistance y
       between y0 and y1. x is the step value."""
    # x1 and x0 are the same
    if x1 == x0:
        return y0
    return y0 + (x - x0) * (y1 - y0) / (x1 - x0)
```

Figure 36. The linear interpolation formula is called within a known range of $[x_0, x_1]$, the step values known from the calibration table.

Once the step has been converted to its corresponding resistance value, the tolerance, or the permitted range or margin of error of the converted resistance, is found. Before it can do so, however, the program may return 0 if the resistance is either infinite or a negative value as a measure to avoid bugs in runtime.

The calibration table shows that larger step values correspond to smaller resistance values, however the resolution, or the "shortness" between two known resistance values, is enhanced. Repeated series of testing has proved that a solution to the change in resolution under 1000Ω is to have a different tolerance calculation. Therefore, the program can execute either tolerance calculation for resistor values less than or at least 1000Ω, which is returning the maximum between the worst percentage error and the resistance value multiplied by

```
def tolerance(step, r_ref=R_REF_OHMS):
    resistance = step_to_resistance(step, r_ref)

    if math.isinf(resistance) or resistance <= 0:
        return 0.0

    if resistance < 1000:
        return max(15.0, resistance * 0.03)

    return max(20.0, resistance * R_REF_TOLERANCE_PCT)
```

Figure 37. The procedures to derive the tolerance for any resistance values less than or at least 1000Ω.

Voltmeter

The voltmeter shares similarities with the ohmmeter in that it is its own separate file, and it uses the same SAR logic as the ohmmeter, except it uses pin GPIO 23. It uses these same constants: MCP4131_MAX_STEPS (31) and _SETTLE_S (20ms). It also uses a table to calibrate the voltage values, except it corresponds to all 32 step values, since a 5-bit SAR is with for $2^5 = 32$ steps.

The reason for this is because before calibration, the measured positive values are exactly their required values, but negative values yielded random offsets that cannot be translated into a mathematical function. In the code, the table is stored in the array STEP_TO_VOLT. The resolution of the SAR is also a constraint in the same way as the ohmmeter.

The calibration table is convenient for programming logic because it replaces mathematical equations, which are used heavily by the ohmmeter. There is no formula for converting a step to voltage because the table stores all possible values since each voltage value are separated by 0.25V.

```
def step_to_voltage(step):
    """Convert SAR step to calibrated voltage via STEP_TO_VOLT lookup."""
    step = max(0, min(step, MCP4131_MAX_STEPS))
    return STEP_TO_VOLT[step]
```

Figure 38. The step_to_voltage function uses a calibration table instead of mathematical conversions to translate a step into voltage. Before this, however, it checks if it is within bounds of [0, 31].

The tolerance is the acceptable variation around the certain value, which in this case is the measured voltage value. This value is to be displayed on the LCD; therefore it must be calculated and does not affect this voltage value.

The conversion from step to tolerance starts with bound checking in [0, 31]. If the step value is 0, then the tolerance becomes $\frac{-4.50-(-4.88)}{2} = 0.19$, where -4.50 and -4.88 correspond to the actual voltage values of -4.5V and -5V respectively. Likewise, if the step value is 31, then the tolerance becomes $\frac{5-4.75}{2} = 0.125$ within the actual value range of [4.75, 5].

For all other step values, this equation is performed in which the step value's corresponding voltage value is not included:

$$V_t = \frac{V_+ - V_-}{4}$$

In this equation, V_t is the tolerance, V_+ is the step-to-voltage value 1 step ahead, and V_- is the step-to-voltage value 1 step behind. Because this is an average based on the step's surrounding step values, the difference of voltage values is divided by 4, not 2.

```
def step_to_tolerance(step):
    """Tolerance based on voltage gap between neighboring steps."""
    step = max(0, min(step, MCP4131_MAX_STEPS))
    if step == 0:
        # use first 2 values of the lookup table
        return (STEP_TO_VOLT[1] - STEP_TO_VOLT[0]) / 2
    if step == MCP4131_MAX_STEPS:
        # use last 2 values of the lookup table
        return (STEP_TO_VOLT[-1] - STEP_TO_VOLT[-2]) / 2
    # use the surrounding 2 values of the step
    return (STEP_TO_VOLT[step + 1] - STEP_TO_VOLT[step - 1]) / 4
```

Figure 39. The function step_to_tolerance finds the tolerance value depending on the step value that is passed. This uses a calibration table to retrieve the equivalent voltage values instead of doing extra calculations to convert the steps.

External DC Reference

The external DC reference is contained inside the voltmeter module because the voltmeter must measure this voltage source to display back to the user. Therefore, no additional modules need to be created to achieve the tasks for the voltmeter.

Internal DC Reference

The internal DC voltage reference is required to supply a voltage value from the SAR ADC and display the live voltage measurement and tolerance percentage to the user. Its defining feature is that it uses a single MCP4131 to control the voltage range. This electrical tool was chosen to be stored inside a class because its methods are called directly from the driver for changing voltage or terminating the program. Its sole task is to convert a desired voltage into a 7-bitstep value, where 0 corresponds to +5V and 127 corresponds to -5V.

The internal DC reference is decomposed into two separate files: dc_reference_single.py, housing the voltage-to-step conversions and the class, and DC_ref_internal.py, containing the SAR and calls to return the live voltage measurement and tolerance. The latter file assigns the same SPI handle (MCP4131) and therefore the same wiper as the voltmeter, an artifact from one of the original circuit designs which combined the DC voltage reference with the voltmeter.

Voltage to Steps

The conversion from voltage to steps begins with the voltage value, which is clamped within the constrained voltage range of [-5, 5] before it is plugged into this equation:

$$t = \frac{V_x - V_{min}}{V_{max} - V_{min}}$$

This equation evaluates the value for t to be within the range of [0, 1] before being used to find the appropriate step value within the step range [0, 127]. V_x is the chosen voltage value, V_{max} is

the maximum voltage of 5V (step 0), and V_{min} is the minimum voltage of -5V (step 127). With these known values, the equation can be rewritten as so:

$$t = \frac{V_x + 5}{10}$$

Finally, the step value is calculated by rounding down the result of this equation:

$$step = S_{neg} + (S_{pos} - S_{neg}) \cdot t$$

In this equation, S_{neg} represent step 127 for +5V, and S_{pos} represent step 0 for -5V. Because the step values and DC voltage are inversely proportional to one another, an individual step value from the size of the range, $(S_{pos} - S_{neg})$, is determined by t to then subtract from S_{neg}. The resulting step value is proceeded to be written into the MCP4131.

```
def _volt_to_step(voltage):
    """
    Convert voltage [-5, +5] to digipot step (W1 only)
    """
    voltage = _clamp(voltage, MIN_VOLT, MAX_VOLT)

    # Normalize to a range of [0 1]
    t = (voltage - MIN_VOLT) / (MAX_VOLT - MIN_VOLT)

    # Linear interpolation
    step = round(STEP_AT_NEG5 + (STEP_AT_POS5 - STEP_AT_NEG5) * t)

    return _clamp(step, 0, 127)
```

Figure 40. This procedure takes an input voltage value, confirms it is within the range of [-5, 5], translates the voltage value into a decimal within [0, 1], and converts it into the step value using this decimal value to subtract from the maximum step value of 127. It is kept within the _volt_to_step function outside the class and is therefore ignored by the driver program.

Measuring DC Reference

The SAR logic handles the operations to retrieve the averaged measured step while the voltmeter converts steps to voltage and tolerance, therefor minimal additions need to me made to retrieve the necessary values. Before the DC voltage reference can return these values to display to the user, it first deactivates the pull up/down resistor on the comparator pin. Then the SAR grabs the averaged step value to be used by the imported conversion functions, which are returned for the driver program to display.

```
def measure_dc_ref(pi, spi_handle, n=11):
    """
    Measure the DC reference output voltage.

    Returns (voltage, tolerance) in volts.
    Uses the same STEP_TO_VOLT calibration table as the external voltmeter
    since it is the same ADC circuit.
    """
    pi.set_pull_up_down(COMPARATOR_PIN, pigpio.PUD_OFF)
    step = _averaged_measure(pi, spi_handle, n)
    voltage = step_to_voltage(step)
    tolerance = step_to_tolerance(step)
    return voltage, tolerance
```

Figure 41. The function measure_dc_ref is the sole function inside the DC_ref_internal.py file that does not establish the SAR logic. It returns the measured voltage and tolerance derived from the SAR step value.

Square Wave Generator

This section of the document discusses the design of the square wave built using the pulse-width modulation (PWM) to create a wave at the necessary frequency and duty cycle. The “pulse-width” refers to the duration of a high signal within one period which comprises of a high and low signal **(A.4)**. The duty cycle is constrained at 50%, the ratio of the high and low signal in any given period. This means that a high signal spans one half of the period, and a low signal spans another half of the period.

The period, T, determines the frequency, f, to be generated by a designated GPIO pin, through the following equation:

$$f = \frac{1}{T}$$

GPIO Pin 13 is chosen to generate the square wave because it is one of the Raspberry Pi 4 pins specifically designed to do PWM. Therefore, it consistently presets the wave to the correct frequency and is not manually handled by the software system.

On the other hand, the amplitude must be hardcoded since it is controlled by a 7-bit MCP4131. The resulting signal from the voltage divider it creates is stabilized by the capacitor, making it oscillate at 0V and preventing distortion from adjusting its amplitude. Although, higher amplitudes may cause it to lose its square-like shape since it is operated close to the power rails from the op-amps, resulting in noise or distorted curves. This issue is solved by first selecting steps to ensure accuracy when adjusting the potentiometer:

Measured Steps to Amplitude	
Step	**Amplitude (V)**
28	2.4
55	4.4
111	8.75
127	10

Table 21. The table depicting the chosen steps and their corresponding amplitudes necessary for calibrating the square wave generator, determined with equivalent ratios between the steps and amplitudes.

The amplitude and step values are then graphed into horizontal and vertical axes respectively and an equation is derived for the line that best fits these points. This equation is responsible for calibrating the remaining unspecified points whenever another desired amplitude is chosen from user input.

$$step = 13.046 \cdot I - 3.3339 \quad (1)$$

I represents amplitude, which is measured in voltage. Peak-to-peak voltage, or Vpp, is double the value for the amplitude, or $Vpp = 2I$. Therefore, the equation can be rewritten like so:

$$step = 6.523V_{pp} - 3.3339 \ (2)$$

Finally, to get rid of the coefficient for V_{pp} and present a direct relationship between V_{pp} and $step$, the equation is rewritten once more:

$$step = \frac{V_{pp} - 0.5111}{0.1534} \ (3a)$$

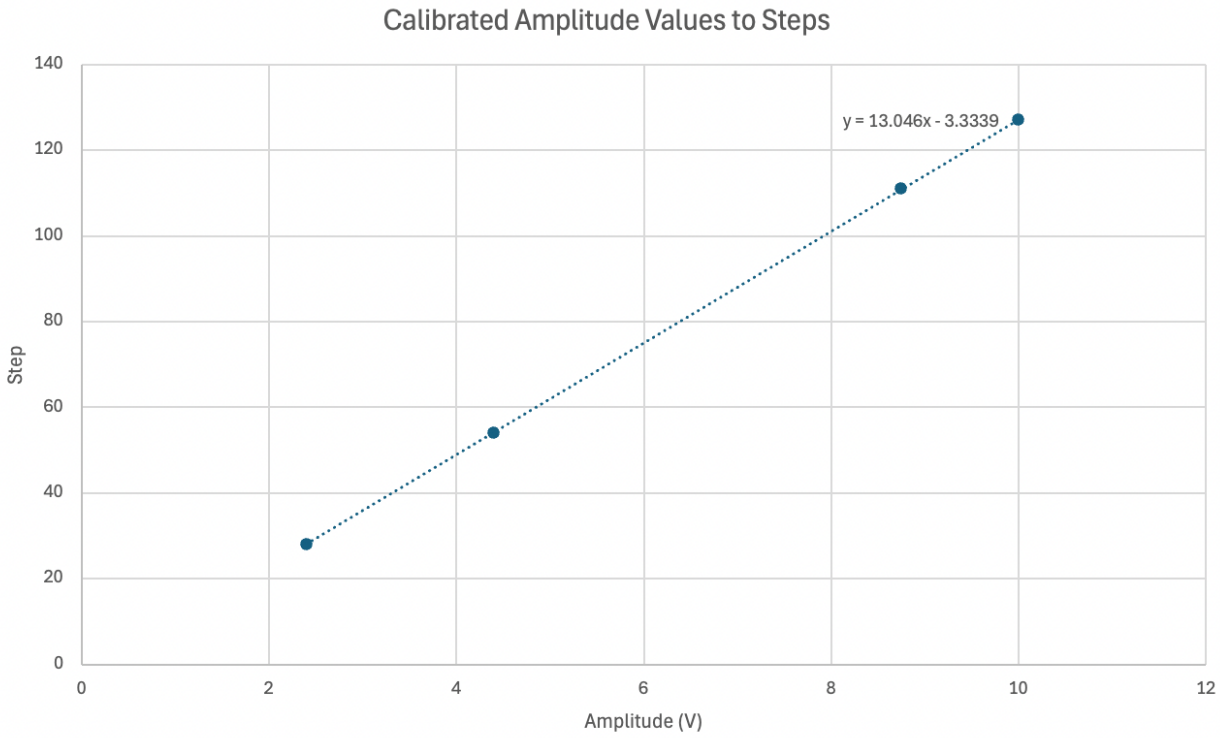


Figure 42. The graph plotting the table for measured steps to amplitude in **Figure X.** into an amplitude-to-step graph. Thus, a linear equation that fits these points is derived, which is Equation (1).

Equation (3a) is be interpreted with definitions for constants in any situation where the equation must be tweaked for enhanced accuracy.

$$step = \frac{2I - CAL_OFFSET}{CAL_SLOPE} \quad (3b)$$

CAL_OFFSET is the vertical translation of the graph and CAL_SLOPE is the coefficient of V_{pp}, which is 13.046V derived from Equation (1). However, from testing, the CAL_OFFSET was slightly adjusted from 0.5111 derived from Equation (3) to 0.45.

For any amplitude values within the range of [0, 10] V, they undergo Equation (3b) and the resulting step value is returned.

```
def _amp_to_step(amplitude):
    """Peak amplitude (0-10V) to MCP4131 wiper step using empirical calibration."""
    amplitude = _clamp(float(amplitude), 0.0, MAX_AMP)
    if amplitude <= 0.0:
        return 0
    step = round((2.0 * amplitude - CAL_OFFSET) / CAL_SLOPE)
    return int(_clamp(step, 0, MAX_WIPER))
```

Figure 43. The _amp_to_step convers the amplitude value to a 7-bit step value to then transmit to the MCP4131. The equation for the step value has been derived from ratios for the ranges of steps and amplitude values and underwent adjustments to correct the accuracy of the amplitude values observed with the oscilloscope.

1 Complete Transfer Function

Combining all stages, the end-to-end signal path from PWM output to final voltage is:

- PWM input: 0 to 3.3 V square wave at frequency f_{sine}, 50% duty cycle.
- Fourier fundamental amplitude: $A_1 = \frac{2}{\pi} \cdot 1.65\ \text{V} \approx\ 1.05\ \text{V}$ peak (relative to midpoint).
- After two-stage RC filter: $V_{filtered} = A_1 \cdot |\text{H(f)}|_{\text{total}} \cdot \sin(2\pi\text{f} \cdot \text{t})$
- After buffer U3: unchanged (unity gain, impedance converted).
- After digipot (code N): $V_{digipot} = V_{filtered} \cdot \frac{\text{N}}{255}$
- After C1 (AC coupling): DC blocked, AC sine unchanged.

- After gain stage U1: $V_{U1} = 13 \cdot \mathrm{V_{digipot}} = \pm 5$ V (with digipot calibrated)
- After C4 (AC coupling): DC blocked, ±5 V AC sine passed.
- At non-inverting input of U2: $V{+}{=}\ 5\ V + V_{U1} = 5 + 5\sin(2\pi \mathrm{f} \cdot \mathrm{t})$
- Output of U2 (unity gain): $V_{out} = 5 + 5\sin(2\pi \mathrm{f} \cdot \mathrm{t})$ [Volts]

The final output is a sine wave with a +5 V DC offset and ±5 V peak amplitude, swinging between 0 V and +10 V at the programmed frequency. This matches the oscilloscope measurement on the physical hardware.
For the same reasons brought up for the DC voltage reference, the square wave generator contains a class with methods directly called by the driver program. The class initiates and terminates the PWM hardware for writing the frequency and amplitude to the MCP4131. To retain the hierarchical structure where the electric tools are controlled through one file, the driver must have management over the PWM and the set functions inside this class.

The square wave signal goes through a custom-built circuit before reaching the output in order to meet the specifications of the project. In order to create an appropriate square wave at high and low frequencies, a set of necessary components needed to be included in the circuit.

Stage	Components	Function	Input	Output
1: PWM Source	RPi4 GPIO (PWM)	Generates PWM signal encoding sine wave	Software frequency control	0–3.3 V square wave, 50% duty cycle
2: Decoupling capacitor Stage 1	C(470 μf)	Gets rid if DC offset from Raspberry Pi	0–3.3 V PWM	Square Wave centered roughly around 0.
4: Inverting op amp (U3)	TL081, R(10 kΩ), R (80kΩ)	Provides a gain for square signal	~0.38 V AC square wave	-10 to 10 V AC square wave

5: Buffer	TL081, R(1 kΩ connected to ground)	Unity gain voltage follower	-10 to 10 V AC square wave	-10 to 10 V AC square wave (low Z)
6. High Pass Filter	R(2 kΩ), C($22pf$)	Reduce jumps at edges of sqauert waves, Smooths out signal	-10 to 10 V AC square wave (low Z)	Smooth Square Wave from -10 to 10 V

Table 22. The table depicting the different stages for generating a sine wave with the implemented components and their assigned functionality, as well as their inputs and outputs.

The circuit starts with the **MCP4131**, a digital potentiometer. By receiving SPI commands (CS, SCK, SDI), it varies the resistance at the output node. This allows for precise, programmatic control over the amplitude of the initial 3.3V PWM signal before it enters the analog processing stages.

The signal then hits an **AC coupling (High-Pass) filter** formed by ($470\mu f$) and ($10k\Omega$).

- **Function:** Since the digital PWM signal is strictly unipolar (0V to 3.3V), it has a 1.65V DC offset.
- **Result:** The large $470\mu f$ capacitor blocks this DC component, shifting the square wave to be centered around 0V (approximately +/- 1.65V). The low cutoff frequency ensures the square shape is maintained without significant "droop" during the high/low cycles.

The centered signal enters the **TL081 (U)**, configured as an **inverting amplifier**.

- **Gain Calculation:** The gain is determined by the ratio of the feedback resistor to the input resistor: A_v = $\frac{-80k}{-10k}$ = -8.
- **Clipping:** Because the op-amp is powered by +/- 12V$, an input of +/- 1.65V multiplied by a gain of 8 (13.2V) exceeds the supply rails. This forces the output to "slam" against the positive and negative rails, sharpening the edges and ensuring a consistent, high-voltage bipolar output.

After amplification, the signal enters a second **TL081** configured as a **voltage follower (buffer)**.

- Function: The output (pin 6) is tied directly back to the inverting input (pin 2), providing a gain of 1.
- Impedance Matching: This stage provides very high input impedance and very low output impedance. It isolates the high-gain stage from the final load, preventing the downstream components from drawing current that could distort the square wave's timing or voltage levels.

The last stage is a small **RC network** consisting of ($2k\Omega$) and **C4** ($22\ pF$).

- **Edge Control:** This acts as a very high-frequency low-pass filter. Its primary role is to suppress high-frequency "ringing" or EMI noise generated by the rapid rail-to-rail switching of the op-amps, providing a cleaner signal for the final output.

In one approach to top-down design, the function generator may be perceived as a complete unit with submodules that each output a square and sine wave. The software architecture could be designed so that both units share the same states, PWM GPIO, calibration logic, and approaches to achieve the desired amplitude.

However, this is complicated by the condition that the square wave module has been described down to its realizable components. For example, within its software architecture, it has its own calibration procedure for converting amplitude to steps, raising uncertainty on its compatibility with even a descended sine wave design. Its states and constants are already established and applying them to an extended user interface branch may break the entire flow with clashing logical errors.

Furthermore, both wave generators have different restrictions of the voltages they are permitted for analog output. While the square wave is allowed a wider [-10, 10]V range, the sine wave is only allowed to output in the nonnegative range of [0, 10]V. If a signal is outputted close to the defined op-amp rails of +/-12V, then increasing its amplitude may lead to higher distortion and likelihood of breaking down into an unsteady and non-oscillating signal.

Ultimately, it makes the most sense for the function generator being interpreted as two separate tools and circuitry for the Electronic Tool Bench. In this manner, both tools' problems could be tackled separately, avoiding the nuisance of fixing one bug causing another someplace else.

Sine Wave Builder

The GPIO pin chosen to generate the sine wave is GPIO 26 because it is the other available pin which uses PWM, so both square and sine waves are generated by GPIO 13 and 26 respectively. The sine wave builder has a collection of calibration tables for the frequencies 1kHz, 2.5kHz, 5kHz, 7.5kHz, and 10kHz are used, with each one containing the desired and measured amplitudes for 0.625V, 1.25V, 2.5V, 5V, 7.5V, and 10V. The frequency value is already generated by the PWM pin, so the software implementation solely focuses on returning the calibrated amplitude value.

It has been observed from the digital oscilloscope that without calibration, increased frequencies dampen the overall amplitude. This means that an amplitude set to 10V at 1kHz would be much larger than an amplitude set to 10V at 10kHz. The sine wave builder aims to recalibrate it by the preset frequency value.

The frequency is converted to the necessary sampling rate depending on its magnitude, as long as it is situated in the constraints of [1, 10] kHz. Sine waves with lower frequencies need higher number of samples within a period while those with higher frequencies need lower samples. The following conditions show a frequency, f, measured in kHz and its corresponding number of samples.

$$1 \leq f \leq 1.5 \rightarrow 48$$

$$1.5 \leq f \leq 3 \rightarrow 32$$

$$3 \leq f \leq 6 \rightarrow 16$$

$$6 \leq f \leq 10 \rightarrow 8$$

```
def _get_samples(self):
    f = self._frequency
    if f <= 1500:
        return 48
    if f <= 3000:
        return 32
    if f <= 6000:
        return 16
    return 8
```

Figure 44. The function that returns the number of samples depending on the frequency value inside the range [1, 10] kHz.

Converting Amplitude to Step

There are several constants that keep the amplitude and its converted step value under the constraints of the project requirements and the MCP4231 potentiometer. MAX_AMP, or the maximum allowable amplitude, is 10V, and AMP_STEP, the amplitude step increment, is 0.625V. The amplitude is snapped, or clamped within the acceptable range of [0, 10] V.

```
def _snap_amplitude(self, amp):
    snapped = round(float(amp) / AMP_STEP) * AMP_STEP
    return _clamp(round(snapped, 3), 0.0, MAX_AMP)
```

Figure 45. The function _snap_amplitude ensures that the amplitude is within the range of [0, 10] V and is ready for conversion to a 7-bit step value for the MCP4231.

To successfully convert the amplitude to step value at negligible frequency, the amplitude is calibrated in two stages: normalization and table correction. The normalization stage corrects the decaying trendline of frequency and amplitude to a steady amplitude. More specifically, the value determines how much correction the frequency value requires to handle a certain amplitude. The resulting correction value, c, is then multiplied with the target amplitude for correct output. Following repeated trials of graphing the frequency f and the normalized value c, the following equation has been determined for the task:

$$c = 0.4065 \cdot \left(\frac{f}{1000}\right)^{0.312}$$

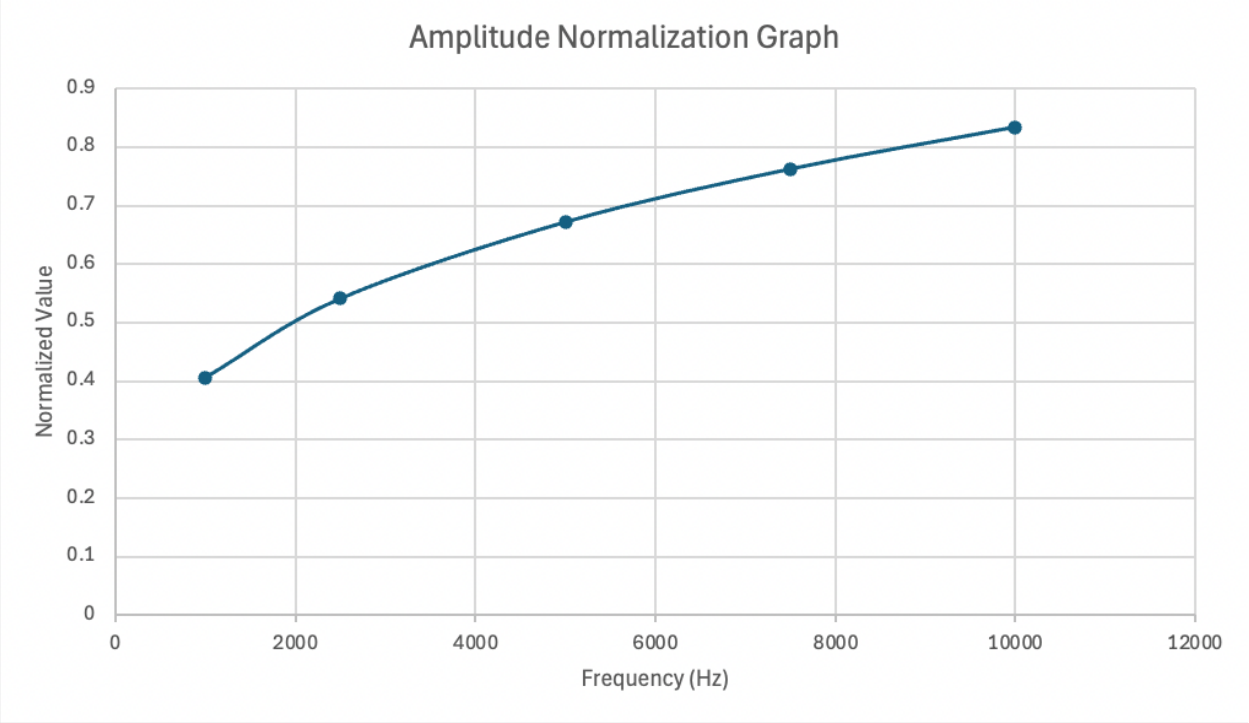


Figure 46. The graph shows the equation for the correction value c, or the relationship between frequency and the normalized value to multiply with the amplitude. Because amplitude values decay at a larger scale for higher frequencies, the normalized value must increase to recorrect it.

```
def _exp_correction(self):
    """
    Exponential frequency correction.
    Keeps the smooth frequency trend correction in place.
    """
    f = float(self._frequency)
    corr = 0.4065 * ((f / 1000.0) ** 0.312)
    return _clamp(corr, 0.30, 1.00)
```

Figure 47. The _exp_correction function returns the correction value using the equation represented in the above graph. This value stays within the range of [0.3, 1] in any case the frequency range could be expanded.

Because a selected frequency value is not guaranteed to be inside the calibration table, two of the closest frequency values are chosen. These are named *bracketing frequencies*. For example, if the desired frequency is 3kHz, then the closest known frequency values with known uncalibrated amplitudes are 2.5kHz and 5kHz.

The linear interpolation equation returns two predicted amplitude values from a graph of known amplitudes and their uncalibrated versions. These two values correspond to the left-hand and right-hand *bracketing frequencies*.

If the two frequencies are equal, meaning that they have the same predicted amplitudes, then this amplitude value is returned. Otherwise, these equations are done for the desired frequency value f:

$$\lambda = \frac{f - f_0}{f_1 - f_0}$$

$$y = c_0 + \lambda(c_1 - c_0)$$

f_0 and $f_1$are the left-hand and right-hand *bracketing frequencies* which correspond to the corrected amplitudes c_0 and c_1. λ is the fraction of the f within the range of f_0 and f_1. Using the same example, in the range of 2.5kHz and 5kHz, 3kHz, f, is 20% of the way there to reach 5kHz from 2.5kHz. This makes 20%, or $\frac{1}{5}$, the λ value. The y value is the final amplitude value within the known range of corrected amplitudes set by c_0 and c_1.

```
def _table_correction(self, amp):
    """Returns the calibrated amplitude value based on the
       set frequency using CAL_TABLES to do linear
       interpolation."""
    # Find the left-bound and right-bound freqs of the set freq
    f0, f1 = self._find_bracketing_freqs(self._frequency)

    def corr_at_freq(freq):
        table = CAL_TABLES[freq]
        xs = sorted(table.keys())
        ys = [table[x] for x in xs]
        return _interp(amp, xs, ys)

    """ Use amp to find adjusted amp values for left-bound and
        right-bound freqs. """
    c0 = corr_at_freq(f0)
    c1 = corr_at_freq(f1)

    if f0 == f1:
        return c0

    frac = (self._frequency - f0) / (f1 - f0)
    return c0 + frac * (c1 - c0)
```

Figure 48. The function _table_correction takes the argument of the desired amplitude and uses the set frequency to perform linear interpolation on its two known surrounding frequencies for their amplitude values. The calibrated amplitude at the set frequency is realized and allows the sine wave builder to run with the corrected frequency and amplitude.

The two predicted amplitude values of their *bracketing frequencies* are found using the linear interpolation technique that is more complicated than with the ohmmeter. This is because this is done to an object of calibration tables with keys and values. This mathematical procedure interprets a graph of known amplitudes on the x-axis and their uncalibrated versions on the y-axis.

Before the linear interpolation, however, a bounds check is done for x, the set amplitude, which must be contained inside [1, 10] kHz. It is then that this procedure is done inside of a loop when x is less than or equal to an amplitude value of the horizontal axis.

First, x_1 is assigned to a valid x-axis amplitude value and x_0 is assigned to the one immediately before it. The same is done for the y-axis amplitude values for assigning y_1 and y_0. If x_1 is equivalent to x_0, then y_0 is returned.

If this condition is not met, then the final procedure mirrors that which finds the final amplitude value within the known range of corrected amplitudes, except this is a more generalized version using the set amplitude x:

$$\lambda = \frac{x - x_0}{x_1 - x_0}$$

$$y = y_0 + \lambda(y_1 - y_0)$$

```
def _interp(x, xp, fp):
    if x <= xp[0]:
        return fp[0]
    if x >= xp[-1]:
        return fp[-1]

    for i in range(1, len(xp)):
        if x <= xp[i]:
            x0, x1 = xp[i - 1], xp[i]
            y0, y1 = fp[i - 1], fp[i]
            if x1 == x0:
                return y0
            frac = (x - x0) / (x1 - x0)
            return y0 + frac * (y1 - y0)

    return fp[-1]
```

Figure 49. The linear interpolation logic to be handled inside an object of calibration tables, CAL_TABLES. The variables xp corresponds to the horizontal axis of known measured amplitudes, and fp corresponds to the vertical axis of their real uncalibrated amplitudes.

Finally, with this corrected amplitude value, it is converted to a 7-bit step value and is returned. This completes the conversion process from amplitude to step.

```
def _amp_to_step(self, amp):
    amp = self._snap_amplitude(amp)

    exp_corr = self._exp_correction()
    table_corr = self._table_correction(amp)

    corrected_amp = table_corr * exp_corr
    corrected_amp = _clamp(corrected_amp, 0.0, MAX_AMP)

    if self._debug:
        print(
            f"[Cal] freq={self._frequency}Hz "
            f"target={amp:.3f}Vpp "
            f"exp={exp_corr:.3f} "
            f"table={table_corr:.3f} "
            f"final={corrected_amp:.3f}"
        )

    step = round((corrected_amp / MAX_AMP) * MAX_WIPER_STEP)
    return int(_clamp(step, 0, MAX_WIPER_STEP))
```

Figure 50. The complete conversion from amplitude to a 7-bit step for the sine wave builder is contained inside the _amp_to_step function. The corrected amplitude is retrieved from the calibration table of the desired frequency, and does linear interpolation to include cases where the frequency is not included within it. This amplitude is used to convert into a step and is clamped within the 7-bit range of [0 127].

Building the Sine Wave

To build a sine wave, the number of samples, or n, must be retrieved. This is determined by the magnitude of the frequency. A lookup table is then made within the range of $[0, n]$ with this expression to execute at each index i:

$$\sin(\frac{2\pi i}{n})$$

This expression achieves a complete sine wave within a period of $[0, 2\pi]$ with n number of even slices, or samples. Next, the period T, measured in microseconds, is calculated like so:

$$\mathrm{T} = 10^6 \frac{1}{f}$$

Both the lookup table and the period are required to convert a continuous sine wave into a series of discrete samples. Each sample inside the lookup table has a time value indicating the

The slot list is created to encompass the entire wave period to store the time duration for each sample. These are measured in microseconds, and they ideally last $\frac{\mathrm{T}}{\mathrm{n}}$ microseconds long. However, the slot list must contain microsecond intervals in integer form to send time durations to the PWM pin for holding a voltage at each sample.

To account for this error, a loop is done across the sample list to ensure all slots times add up to match the period. The program uses two variables to track the ideal elapsed time, and another to track the time to report to the PWM pin. These variables are called *acc* and *used* respectively. The time in each slot is applied rounding to the nearest integer of the subtraction *acc – used*. If the slot time is less than 1, then it is rounded to 1.

The resulting slot list has some sampling error e against a T period put in integer form, or $\mathrm{T_i}$. Along with the sum of the slot list, s_{sum}, e is calculated like so:

$$e = T_i - s_{sum}$$

The starting value of the sampling error is the time value of the final slot found at the end of the loop. Once more, the slot time is adjusted to 1 if the slot time falls below 1, since the PWM cannot interpret zero time.

```
def _build_wave(self):
    """ Outputs the sine wave from the PWM GPIO pin using samples
        in slot_list to switch the pin on/off based on duty cycle. """
    n = self._get_samples()
    lut = [math.sin(2 * math.pi * i / n) for i in range(n)]

    period_us = 1_000_000.0 / self._frequency

    slot_list = [] # contains the samples for storing time
    acc = 0.0 # the ideal time elapsed in a period
    used = 0 # the actual time used per sample

    # Fill the slot list with approx pieces of the ideal period
    for _ in range(n):
        acc += period_us / n
        slot = int(round(acc - used))
        if slot < 1:
            slot = 1
        slot_list.append(slot)
        used += slot

    # The resulting slot list has an error compared to acc
    target_total = int(round(period_us))
    error = target_total - sum(slot_list)
    slot_list[-1] += error

    # If needed, fix the final slot time from 0 to 1
    if slot_list[-1] < 1:
        slot_list[-1] = 1
```

Figure 51. The first stage of the _build_wave function starts with the slot list to store integer portions of the period T across all samples, determined earlier by the frequency value. The complete slot list has a sampling error at the last slot because the PWM pin cannot decipher decimal time values, so the period must be rounded to the nearest integer T_i, paving way for this error to occur.

With the slot list complete, the sine wave is then modified to be interpreted as a PWM signal. To turn a GPIO pin on and off, a bit mask can be used by shifting a bit value to the left by the ON/OFF status of the GPIO. Therefore, both the on_mask and off_mask have the same assignment using the PWM pin. This is used in example code in the *pigpio* library documentation. The pulses list holds pulse information throughout a period.

```
# Bit masks for turning the PWM GPIO on and off
on_mask = 1 << PWM_GPIO
off_mask = 1 << PWM_GPIO
# Stores the pulses in a list
pulses = []
```

Figure 52. The on and off masks use bitshift left operators on the current ON/OFF status of the PWM GPIO.

With the slot list complete, the sine wave is interpreted as a PWM signal. This is done with another, simultaneous loop across the lookup table and the slot list. In each iteration, a sample value inside the lookup table is converted into a duty cycle is found using this equation while clamped between 0 and 1:

$$0.5 + 0.5s$$

The variable s is the sample between -1 and 1. To better illustrate how sample signifies the duty cycle, the sample values of -1, 0, and 1 correspond to the duty cycles 0%, 50%, and 100% respectively.

Next, a slot value is split into a high pulse and a low pulse. The high pulse is found by multiplying the duty cycle by the slot value, and the low pulse is the difference of the slot value and the high pulse. A high pulse switches the PWM GPIO on, while a low pulse switches it off for the duration determined by the duty cycle. Each pulse is appended to the list of pulses while the program is running. Finally, the sine wave is generated from the Raspberry Pi 4 and can be smoothened using hardware circuitry.

```
# Find the duty cycle
for sample, slot_us in zip(lut, slot_list):
    # The duty cycle from a lut sample between 0 and 1
    duty = _clamp(0.5 + 0.5 * sample, 0.0, 1.0)

    # high and low pulses comprise the sample value
    high_us = int(round(duty * slot_us))
    low_us = slot_us - high_us

    # high_us > 0 turns on the GPIO, low_us > 0 turns off the GPIO
    if high_us > 0:
        pulses.append(pigpio.pulse(on_mask, 0, high_us))
    if low_us > 0:
        pulses.append(pigpio.pulse(0, off_mask, low_us))
```

Figure 53. The second stage of the _build_wave function iterates through both the lookup table and the slot list to find the duty cycle. This determines the durations to switch the PWM GPIO on and off within a sample value.

Sine Wave Frequency Measurement

In the codebase, the tool that measures the frequency from an external sine wave signal is treated as a separate module from the sine wave generator as a result of functional decomposition. This is the same approach used to divide up the function generator into square wave and sine wave generators. And, in the main menu of the user interface, the measurement tool is treated as detached from the sine wave generator, with both being separate options.

Since the measurement is not voltage, it does not use the SAR ADC, yet the GPIO pin used to seek the output of the comparator pin is GPIO 5. The frequency measurement tool is stored inside of a class for a clean import into the driver program. The method to update the frequency is repeated inside the driver instead of within the class because the frequency must update while its page remains open. One iteration of the loop updates the frequency value to be shown on the LCD. The measurement tool's purpose is to assign a newly found frequency value for its getter method to return while the page is shown.

The callback function seeks the rising edge from GPIO 5 out of the comparator because one cycle of the sine wave is equivalent to 2 ticks. This is reflected in the EDGE_DIVISOR constant

which is assigned to 1. This is used to directly calculate the frequency before reassignment. The variable dt is defined as the period between two of the comparator's most recent ticks, or high signals from the comparator. If this is below the value of MIN_DT_US, or 50µs, then it ignores these high values and skips frequency update.

The program uses a double-ended queue (Python deque) to calculate frequency. It has an explicit length of 32, referring to the number of periods the averaged frequency value is evaluated from. In each page cycle, the following queue procedures are done. First, the queue appends the dt value. Then, it finds the average frequency by calculating the mean across the dt values in the queue. Finally, this equation is done:

$$f = \frac{10^6}{avg_dt} \cdot \frac{1}{EDGE_DIVISOR}$$

In this setup equation, 10^6 and avg_dt are measured in microseconds, with 1 second equivalent to 10^6 microseconds. The EDGE_DIVISOR constant may divide the frequency depending on the number of cycles that are detected from the comparator and the size of the queue. Since the queue's length is suitable for one cycle of the sine signal, the EDGE_DIVISOR is redundant because of its value of 1.

```
class FrequencyMeter:
    """ Calculate the frequency of a sine wave on a button press."""
    def __init__(self, pi, gpio_pin=GPIO_PIN):
        self.pi        = pi
        self.gpio_pin  = gpio_pin
        self.last_tick = None
        self.frequency = 0.0
        # The buffer stores number of periods to average (update) frequency
        self._buf      = deque(maxlen=BUFFER_LEN)

        # 1 sine wave cycle = 2 ticks, so seek rising edge
        pi.set_mode(self.gpio_pin, pigpio.INPUT)
        self.cb = pi.callback(self.gpio_pin, pigpio.RISING_EDGE, self._cb)

    def _cb(self, gpio, level, tick):
        if self.last_tick is not None:
            dt = pigpio.tickDiff(self.last_tick, tick)
            # On button press, change avg_dt to update frequency
            if dt >= MIN_DT_US:
                self._buf.append(dt)
                avg_dt = sum(self._buf) / len(self._buf)
                self.frequency = (1_000_000.0 / avg_dt) / EDGE_DIVISOR
```

Figure 54. The class which stores the frequency measurement and update logic being executed inside the page loop stored in the driver program whenever this page is open.

Obtaining Voltages on Raspberry Pi

When the group was completing Deliverable 3 and testing the needed components, there was an error in obtaining the proper 3.3V to build a simple LED/Resistor circuit.

The error was in attempting to obtain the voltage using the GPIO extender on the breadboard from the Pi Hat Protector. Researching the proper ribbon cable direction and orientation, we first figured that our ribbon cable seemed to not match the proper color orientation, so that was why we were not obtaining 3.3V through the GPIO extender. However, after further group discussion, we realized that the GPIO extender may be unable to supply 3.3V, and we instead used the 3.3V port on the Pi Hat Protector to obtain the needed 3.3V for our LED and Rotary encoder.

To program the various values of resistance, the Python program opens the SPI handle, the number that represents the potentiometer, using a chosen SPI channel, bits per second, and flags settings. Then, the program attempts to start the cycle to approximate the resistance across Terminal A and the wiper by iterating through the necessary Ohm (Ω) values and the step values in [0, MAX_STEPS] in two modes. Each iteration has some delay measured in seconds.

PCB Design and Considerations

In this section, we discuss the technical considerations used to design our printable circuit board. To create a PCB that can perform well without causing significant problems for whatever system it is in, strategic grouping of components to reduce the physical length of traces is very important. When components that share a signal net are placed in proximity, the resulting copper traces are naturally shorter, which directly reduces the parasitic resistance of the path. Looking at

the formula of resistance $R = \rho \frac{L}{A}$, where L represents length, shortening the copper path is the most effective way to prevent unwanted voltage drops to ensure power reaches components. This is particularly critical in power delivery networks and high-current paths where even milliohms of resistance can lead to significant stability issues.

Furthermore, minimizing trace length through tight component grouping is the primary method for decreasing parasitic capacitance and inductance. In any circuit, long traces act as unintentional capacitors and inductors that can degrade signal integrity by causing rise-time delays or high-frequency ringing. By keeping the layout compact and reducing the overall loop area of the circuit, a designer also improves the electromagnetic compatibility of the device. Shorter traces are less likely to act as miniature antennas that emit or receive unwanted electromagnetic interference, which is vital for maintaining the performance of sensitive analog nodes or high-speed digital buses. This practice ensures that the final board operates as intended by keeping the electrical characteristics of the interconnects as close to the ideal circuit model as possible.

Throughout the project, our PCB design stayed relatively the same, only adding components to free space, rather than redesigning the board after every new component was added. This was mostly done to make our board as compact as possible even if it required traces to be longer. Only until all our designs were confirmed on our schematic, was when we decided to reduce the length of all of our traces and create a board that of course was larger than we anticipated but yielded less resistance through wiring, thus creating a board more fit with our design specifications.

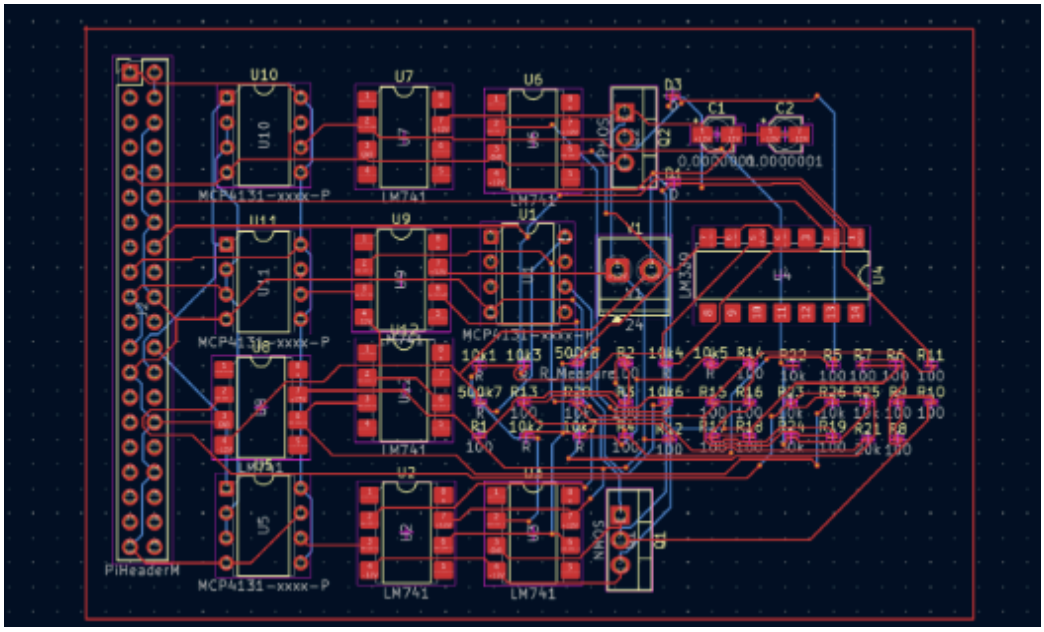


Figure 55. PCB layout designed for a small board size.

Figure 55 above describes our PCB design that was completely focused on board size. As the viewer might be able to see, to reduce size components that should group closely together are not, rather in some instances moving farther apart to reduce space. This created a board where most components are connected by long traces. In addition, this PCB design uses a large quantity of TL0 741 op-amps which was reduced in further iterations of PCB designs, namely as our designs changed and our designer used better footprints to represent components. The number of resistors was also a huge factor that was reduced as new designs did not utilize as many.

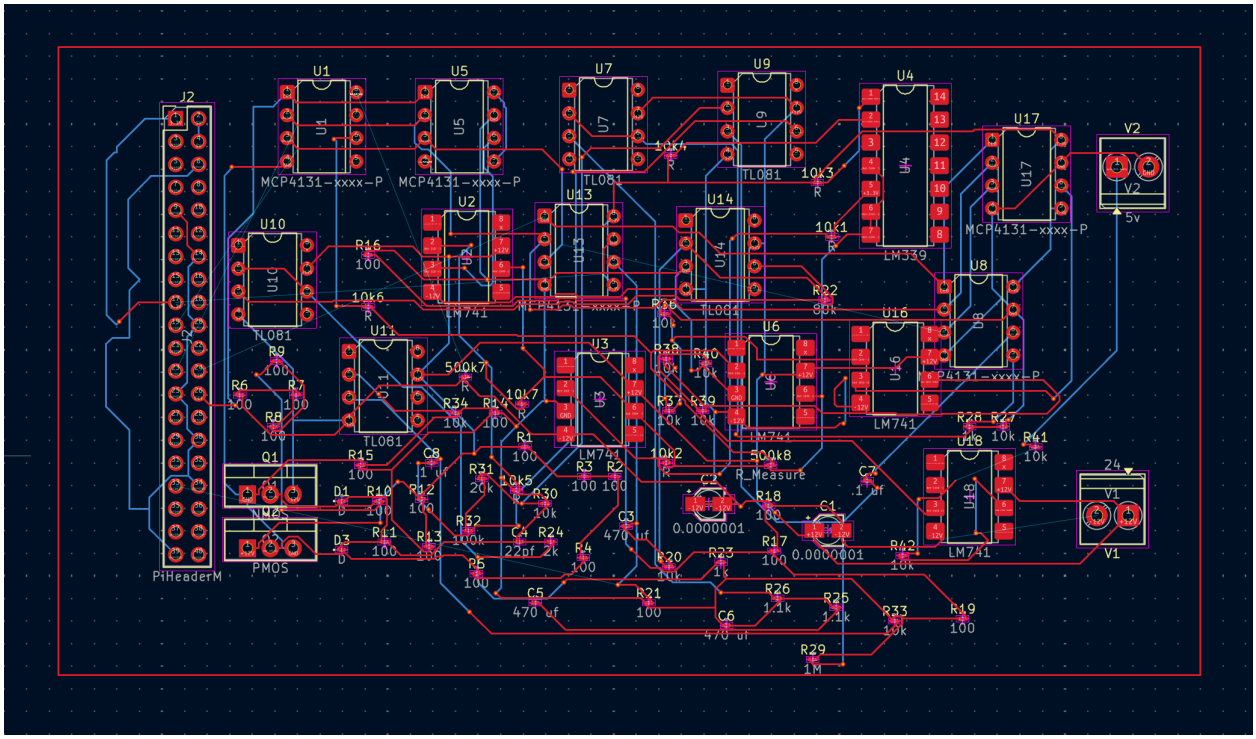


Figure 56. PCB design with main goal of reducing trace length.

In this iteration, the design philosophy shifts from absolute miniaturization to signal integrity through trace length minimization. Unlike previous layouts where components were scattered to fit available gaps, this design prioritizes functional grouping, ensuring that bypass capacitors, feedback resistors, and operational amplifiers are clustered as tightly as possible around their respective signal paths. By significantly shortening these connections, we effectively lower the parasitic resistance and inductance of the board, which is critical for maintaining the sharp transitions and high-frequency response required for the square wave output. This proximity ensures that the signal does not have to travel across the board, reducing its exposure to electromagnetic interference and preventing the traces from acting as unintended antennas.

The hardware footprint was further optimized by consolidating the op-amp stages; instead of multiple single-package ICs, we utilize quad-channel integrated circuits to naturally condense the layout and eliminate sprawling interconnects. This transition, coupled with a more disciplined approach to component selection, allowed for a significant reduction in the total number of resistors and a cleaner routing architecture. By utilizing more accurate footprints and shared

power distribution networks, the design achieves a lower parasitic capacitance without sacrificing performance. The final result is a robust, low-noise signal conditioning stage that proves a well-grouped, electrically efficient layout is far superior to a design that is merely compact.

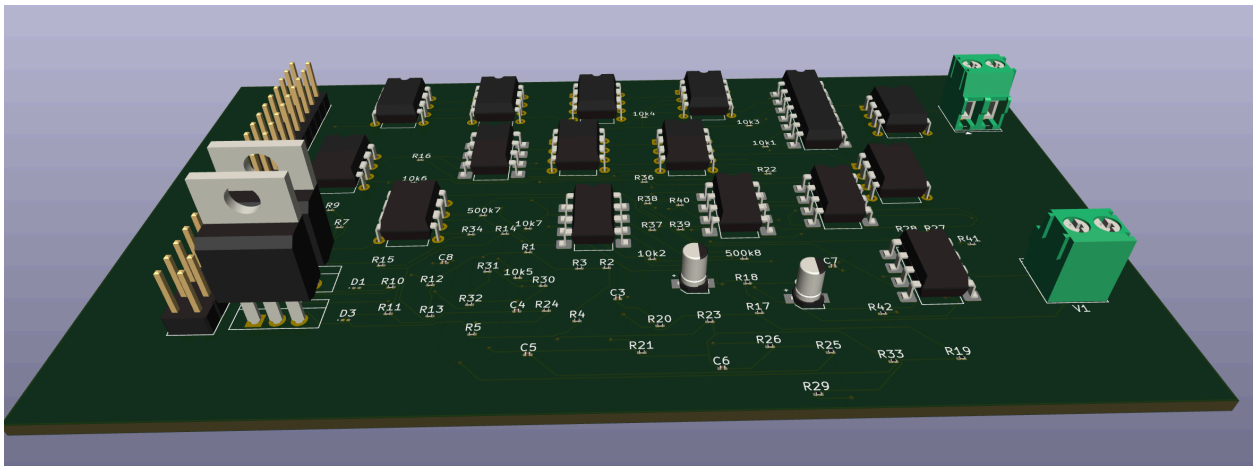


Figure 57. 3D View of the final PCB Design.

Ethics

Design Integrity and Original Work

Throughout the development of the Electronics Tool Bench, our team made a conscious effort to ensure that all design work was original and properly understood. While we referenced course materials, datasheets, and general engineering concepts, all circuit designs, system integration decisions, and implementation details were developed by our team. Each subsystem, including the power supply, measurement circuits, and signal generation components, was built and tested based on our own understanding rather than copied directly from external sources.

When external resources were used, such as datasheets for components or general reference designs, they were used strictly as learning tools to guide our understanding of how components operate. We ensured that any design choices made were adapted to meet the specific

requirements of our system. This approach allowed us to maintain academic integrity while also developing a deeper understanding of the engineering principles involved.

Additionally, collaboration within the team was handled in a way that ensured equal contribution and transparency. Each member contributed to different aspects of the design, including hardware construction, debugging, and documentation. By maintaining clear communication and shared responsibility, we ensured that all work represented a collective and honest effort.

Ethical Considerations for Real-World Application

If the Electronics Tool Bench were to be developed as a real product, several ethical considerations would need to be addressed. One of the most important factors would be user safety. Since the system involves multiple voltage levels, including ±12V rails, proper protection mechanisms would need to be implemented to prevent electrical hazards. This could include current limiting, proper insulation, secure connectors, and clear labeling to ensure that users are not exposed to unsafe conditions.

Another important consideration would be reliability and accuracy. As a measurement tool, users would rely on the voltmeter and ohmmeter readings to make decisions about their circuits. Inaccurate measurements could lead to incorrect conclusions or even damage to external equipment. Therefore, it would be ethically necessary to ensure that the system is properly calibrated, tested, and validated before being used by others. Providing clear documentation about the accuracy and limitations of the system would also be important so that users understand how to interpret the results.

Transparency and honesty in design would also be critical. If certain limitations exist, such as reduced resolution due to the use of a digital potentiometer, those limitations should be clearly communicated to the user. Misrepresenting the capabilities of the system would violate engineering ethical standards and could lead to misuse or unrealistic expectations.

Finally, considerations such as sustainability and long-term usability would play a role in a real product. This includes selecting components that are reliable and available, designing the system in a way that allows for maintenance or repair, and minimizing unnecessary waste. While these

factors were not the primary focus of the class project, they would become significantly more important in a real-world engineering context.

Overall, the design of the Electronics Tool Bench reflects an effort to follow ethical engineering practices by maintaining originality, ensuring honesty in design decisions, and recognizing the importance of safety and reliability in real-world applications.

Self-Critique

Overall, our team performed well in developing the Electronics Tool Bench and successfully met the primary project requirements. The final system operates as intended, with the major subsystems working together as a complete design. Throughout the project, we were able to work through challenges in both hardware and software, and we showed steady progress as the system came together. While the final result was successful, there are still areas where we could have improved in how we approached the project.

One of the strongest aspects of our team was how we handled the distribution of work. We made a consistent effort to keep things fair, and no single person was responsible for only one part of the project. Instead of dividing strictly into hardware or software roles, we all contributed across multiple areas, including circuit design, coding, testing, and documentation. This helped ensure that everyone understood the full system and allowed us to support each other when issues came up. It also made the overall workflow more balanced and prevented any one person from becoming overloaded.

Communication was generally effective, but there were times when it could have been more consistent. At certain points, progress was made individually without fully updating the rest of the group, which led to some confusion when integrating different parts of the system. More frequent updates and clearer communication about what each person was working on would have helped avoid this. Even so, when it came time to debug and finalize the system, the team worked well together and was able to resolve issues collaboratively.

On the hardware side, one of the biggest challenges was working with breadboards. While they made it easier to build and modify circuits, they also introduced problems such as loose

connections, noise, and difficulty tracing errors. Some of the issues we encountered were not due to the design itself, but rather the physical implementation. A more organized wiring approach or moving to a PCB earlier would have made the system more stable and easier to troubleshoot.

From a design perspective, some of our choices were effective but not optimal. The comparator-based successive approximation method used for measurement worked well enough to meet the requirements, but it limited the resolution and introduced some measurement uncertainty. A more advanced approach using dedicated hardware would improve accuracy. The waveform generation also functioned as intended, but the quality of the output could be improved with a more precise method of signal generation.

The software integration was successful, but it required a significant amount of debugging alongside the hardware. In some cases, it was difficult to determine whether issues were caused by the code or the circuit. A more structured approach to testing individual components before full integration would have made this process smoother. Despite this, the final system demonstrates a clear understanding of how software and hardware interact.

Overall, the team met expectations and produced a working system that reflects a strong understanding of the project goals. The way we shared responsibilities across all aspects of the project was a major strength, and it allowed us to stay engaged with the full design rather than working in isolated sections. With improvements in organization, communication, and implementation, the system could be developed into a more refined and reliable design.

Revision Plan

Based on the results of the Electronics Tool Bench, there are several areas where the design could be improved with additional time and refinement. While the system successfully meets the required functionality and demonstrates the integration of analog and digital subsystems, many of the design choices were made with simplicity and feasibility in mind rather than long-term optimization. A future iteration of this project would focus on improving measurement accuracy, system stability, physical implementation, and overall usability.

One of the most noticeable limitations of the current design is the accuracy of the measurement subsystems. Both the voltmeter and ohmmeter rely on a comparator-based successive approximation method using a digital potentiometer, which inherently limits resolution. Because the system operates in discrete steps, the measured values include a level of uncertainty that is directly tied to the step size of the digital potentiometer. If more time were available, this approach could be replaced with a dedicated analog-to-digital converter, which would provide significantly higher resolution and more precise measurements. In addition to this, better component selection, such as tighter tolerance resistors and more consistent reference elements, would help reduce error and improve repeatability across different measurements.

Another area that could be improved is the power supply and overall signal stability. Although the ±12V supply functions correctly and supports the analog subsystems, the breadboard-based implementation introduces noise and potential instability. In future iterations, additional filtering and more refined regulation could be implemented to improve the quality of the supply rails. This would directly benefit the performance of the measurement circuits and waveform generators, which are sensitive to noise. Improving grounding techniques and separating analog and digital return paths more carefully would also help reduce interference and produce cleaner signals throughout the system.

The physical construction of the system is another major area for improvement. The current design relies heavily on breadboards, which, while useful for prototyping, introduce issues such as loose connections, wiring complexity, and increased susceptibility to noise. Transitioning the design to a printed circuit board would significantly improve reliability, reduce wiring errors, and create a much more organized and compact system. A PCB would also allow for better layout control, which is especially important for maintaining signal integrity in mixed analog and digital systems like this one.

From a functionality standpoint, there are several ways the system could be expanded or enhanced. The sine wave generator, for example, could be improved by replacing the PWM-based approach with a more precise digital-to-analog conversion method, resulting in a smoother and more accurate waveform. Additional features such as current measurement or higher resolution signal generation could also be added to make the system more versatile. These

improvements would bring the Electronics Tool Bench closer to the capabilities of professional laboratory equipment.

If this project were to be developed into a real product rather than a class assignment, several additional considerations would need to be addressed. Safety would become a primary concern, requiring proper isolation, protection circuitry, and compliance with electrical standards. The system would also need to be housed in a durable enclosure with clearly labeled inputs and outputs, making it safe and intuitive for users. Thermal management, long-term reliability, and consistent manufacturing processes would also need to be considered. In its current form, the system demonstrates the core functionality, but it would require significant refinement to meet the expectations of a commercial product.

Overall, the Electronics Tool Bench successfully achieves its intended goals and demonstrates a strong understanding of both analog and digital design principles. However, with additional time and resources, the system could be improved in terms of accuracy, stability, and usability, ultimately resulting in a more robust and professional-level design.

Individual Reflections

George Braun

I feel proud of myself and my group for all that we have accomplished. I am extremely happy with the group that was created by the instructors. This course was time consuming and difficult at many points throughout the semester but being able to rely on the members of my team made it not only easier, but actually enjoyable especially on late nights in the circuits lab.

Looking back at the first few weeks of this semester, I laughed at how hard I thought it was to get just the raspberry pi setup and to get a digipot to work. Now having completed through checkpoint A-, I am much more confident in my abilities as an engineer to problem solve. I have learned a lot about combining hardware and software as well as how to ascertain where the problem lies between the two.

I definitely still have some improvements to be made regarding working within a team. There were instances of allowing others to lead or tackle issues, and I would not rise to the occasion as

much as I could have to assist. I need to speak up more on design decisions in future projects as well as being more proactive in assisting others if they need help. I did my part when asked or needed to the best of my ability, but did not go above and beyond when I should have for my team.

Overall, I am grateful to have experienced this class and persevered alongside my great team members. This design project was a mountain to climb at the beginning but now looking back I am confident I could tackle something like this myself if given enough time. I want to thank my group members for pushing us to achieve checkpoint A- and for all the hard work they put in to accomplish this goal.

Tillman Clark

As a group, I think that we really did a great job. I additionally am very thankful that I got such a good group of smart and like-minded people that I would be more than happy to call my friends. In my past 4 semesters here at the University of Georgia, though, I do not think that I have taken a more demanding and challenging course.

However, I do not think that I have ever taken a course that has really pushed me, along with my group, to try to complete a very difficult thing outside of class with little to no guidance. Some of the courses here are more in one ear and out the other, but I really think that this project has been very meaningful to us. This course overall helped me relearn how to do group work and stretch the boundaries of managing my time.

Something that was significantly hard for me during this semester was managing work, band gigs, going to the gym, and my personal social life. Because a lot of my days outside of our meeting times were taken by other activities, it really felt like a good chunk of my time was dedicated to meeting with my group. I am very thankful for the communication and clarity I had with my groupmates. If any of us couldn’t make it to a meeting or were occupied, we could very easily text the group chat and let them know our situation and maybe find something different to work on. One week, I had the flu, and I remember vividly taking an hour or two to reorganize our completely messy GitHub, since I couldn’t be there to test and work on hardware.

Something I wish I could've done better was be more present for my team. I feel that, even though I might've been at work one night, or having to play a band gig somewhere, it still makes me feel guilty that I missed even a small amount of testing or design changes, because of how much our whole group bonded and how much we all really want to be there for each other.

During the start of the semester, we all said that we were going to pass Checkpoint A-. As of last week, our group accomplished that, and through a lot of trial and error, and GitHub commits, we kept our promise. Go Group 5!

Kai DeVito

I am thankful to be valued as a dedicated member of my group. This has been the most difficult course I have ever taken in my entire life, not because of the topics, but because of the workload and hours that must be expended to achieve the best grades possible. I am relieved that I listened to my academic advisor to not add other major required courses at the same time as this class, because it really did consume the most time and effort out of any class I have taken.

Because the expectations of this class demand us to schedule work meetings outside of class times, I make sure that I communicate with everyone the times I am available to meet, or if I cannot due to personal obligations. However, when I am not visiting my family back home, I always meet and let the others know if I would be arriving late when plans change. I have learned to exercise my punctuality and manage my schedule with my other classes.

My perfectionism has been the biggest obstacle that I had to hurdle over, and I feel confident that I had with this class. There were countless times where I had to accept the quality of my work as is because I needed time to focus on other classes, and it has been hard from the start. Now, I am using my time more effectively by benchmarking my goals every day to get my work done.

To be honest, I could never ask for a better and more awesome group of students to work with than the ones also doing their self-reflections. I do not feel so bad for the other groups who are breaking up or going through a time of trauma because we all had an equal access to class resources, circuits equipment, and the technology to complete our project. Every group had the

same expectations, so it is a matter of knowing how the system works to be on top of deadlines and cooperate with the other group members.

I am glad that I suffered though this course together with the people who I see as more than members of Group 5. I have made four new friends this semester and all of us share their hardships and make each other laugh when times get tough. At the same time, all of us understand what to do to get each deliverable and other assignments completed on time, and we have applied time and time again the concepts we have been taught (or currently being taught from other classes). Passing Checkpoint A- would not be possible without our contributions as a team. In my opinion, I am experiencing the greatest ending to this course. I am looking forward to place that sticker at the back of my laptop!

Logan Griffy

Looking back on this project, I feel like I gained a much better understanding of how different parts of a system actually come together. At the start, it felt like we were just building separate circuits, but as we moved forward, it became clear that everything depended on everything else. The power supply affected the measurements, the measurements depended on clean signals, and the software had to work with whatever the hardware gave it. Seeing all of that connect was probably the most valuable part of the experience for me.

One thing I think I did well was staying involved across multiple parts of the project instead of focusing on just one area. I spent time working on hardware, helping debug circuits, and also contributing to the documentation. That helped me understand not just what we were building, but why it worked the way it did. There were definitely moments where things did not work the way we expected, especially when dealing with breadboards and wiring issues, but those situations forced me to slow down and really think through what was happening instead of just guessing.

At the same time, there are things I would improve if I were to do this again. I think I could have been more organized when it came to testing and documenting changes as we went. There were times when we would fix something, but not fully track what changed, which made it harder to

understand problems later. I also think I could have taken more initiative earlier on in breaking down larger problems into smaller steps instead of trying to tackle everything at once.

Overall, I feel like this project helped me move past just understanding concepts and actually apply them in a real system. It was not always smooth, but working through those challenges made the final result feel a lot more meaningful. I came away from this with a better understanding of both the technical side of the work and how to function as part of a team on a long-term project.

Andrew Primiano

Honestly, wrapping up this design methodology project feels like a massive weight has been lifted, but in the best way possible. I think every week our team put on average of 10 hours into it. Looking back at all the deliverables in the beginning, I am impressed at how far we have come. The technical side was a huge jump for me; I really dove headfirst into PCB design and KiCad, learning how to actually build custom circuit designs that met some pretty intense, specific qualifications. One of the biggest hurdles was definitely the software side.

I've always been more of a Java guy, so the learning curve for Python felt massive, and I really wish I'd had a longer runway to get comfortable with it before diving into the deep end. That said, finally getting the hang of the pigpio library and figuring out how to use a digipot to manipulate voltage felt like a major breakthrough for me. I'm so grateful for the team because we just clicked instantly, and being part of this group really forced me to level up as an intra-team communicator and stay way more organized than I usually am. Those late nights in the circuits lab were definitely a grind, but having a group I could actually rely on made the difficult parts actually enjoyable. If I had the chance to do it all over again, I'd definitely overhaul our breadboard from the start, using fewer wires and creating a much cleaner system for implementing the circuits would have saved us so much troubleshooting stress.

This project really challenged me to be a better intra-team communicator and stay organized, but it also showed me that being flexible and going above and beyond is the best way to be in a group. Overall, I'm just incredibly proud of what we accomplished together, and I'm walking

away feeling like I can actually bridge the gap between hardware and software to solve real-world problems on my own.

Appendix

Appendix A: Software

1 Terminal commands to download and install the latest version of the *pigpio* library are found in https://abyz.me.uk/rpi/pigpio/download.html

2 SPI protocol background information can be read in the article https://www.geeksforgeeks.org/electronics-engineering/what-is-serial-peripheral-interface-spi/

   a) The difference between SPI and I2C protocols are summarized in diagrams and tables inside the article which helped better understand the concepts without blindly implementing them https://www.totalphase.com/blog/2021/07/i2c-vs-spi-protocol-analyzers-differences-and-similarities/

   b) The Electronics Tool Bench software system uses the methods, including the byte addresses to send information stored in bits and bytes, of the example code from the 'I2C LCD Display' within the *pigpio* library website https://abyz.me.uk/rpi/pigpio/examples.html

3 This article on flash ADC topology helped better understand its advantages and disadvantages, which are found here https://www.analog.com/en/resources/technical-articles/understanding-flash-adcs.html

   a) Similarly, this article on integrating ADC contains essential information on the integrating ADC topology to decide on the most suitable topology https://www.monolithicpower.com/en/learning/mpscholar/analog-to-digital-converters/detailed-analysis-of-adc-architectures/integrating-adcs

4 Further information about this characteristic and the PWM approach to constructing the square wave is found in this article https://analogcircuitdesign.com/pulse-width-modulation-pwm/

Appendix B: Power Supply

1 Information on designing dual power supplies using a single DC source was referenced to better understand midpoint generation and rail splitting https://www.electronics-tutorials.ws/blog/dual-power-supply.html
2 Operational amplifier behavior and stability considerations were reviewed to properly implement the control stage https://www.electronics-tutorials.ws/opamp/opamp_1.html
   a) The use of feedback in op-amp circuits was essential for stabilizing the midpoint voltage and maintaining symmetric rails
   b) Improper feedback configuration can lead to instability, which was avoided through correct resistor selection and layout
3 MOSFET operation and characteristics were studied to understand their role in the output stage https://www.electronics-tutorials.ws/transistor/tran_7.html
   a) The push-pull configuration allowed the system to both source and sink current as required
4 Capacitor filtering and ripple reduction techniques were used to stabilize the output rails https://www.electronics-tutorials.ws/filter/filter_1.html

Appendix C: Ohmmeter

1 Voltage divider principles were used to convert resistance into a measurable voltage https://www.electronics-tutorials.ws/voltage/voltage-divider.html
2 Comparator operation was studied to understand how analog voltages are converted into digital signals https://www.electronics-tutorials.ws/opamp/op-amp-comparator.html
   a) The comparator output provides a HIGH or LOW signal used by the Raspberry Pi for decision making
3 Successive approximation methods were referenced to implement the SAR measurement approach https://www.analog.com/en/education/education-library/successive-approximation-adcs.html
   a) This method allowed resistance to be determined iteratively without requiring a dedicated ADC

Appendix D: Voltmeter

1 Input scaling techniques were used to map ±5V into a measurable range https://www.electronics-tutorials.ws/opamp/opamp_3.html
2 Signal buffering using op-amps ensured that the measured voltage was not affected by loading https://www.electronics-tutorials.ws/opamp/opamp_4.html
3 Comparator-based measurement methods were again used to determine voltage values
   a) The same SAR logic used in the ohmmeter was adapted for voltage measurement
   b) Scaling and inverse scaling were required to convert between measured and actual values

Appendix E: DC Voltage Reference

1 Non-inverting amplifier configurations were used to implement the gain stage https://www.electronics-tutorials.ws/opamp/opamp_3.html
2 Difference amplifier circuits were used for level shifting https://www.electronics-tutorials.ws/opamp/opamp_5.html
   a) The level shifting stage allowed conversion from a unipolar to bipolar output
3 The relationship between input and output voltage is defined by the equation
   a) $V_{out} = 3V_{in} - 5$
   b) This equation was used to verify system behavior and expected outputs

Appendix F: Square Wave Generator

1 Pulse-width modulation (PWM) was used to generate the square wave signal https://analogcircuitdesign.com/pulse-width-modulation-pwm/
2 GPIO control on the Raspberry Pi was used to adjust frequency and duty cycle
   a) A fixed 50% duty cycle ensured a symmetric waveform
3 Amplification stages were used to increase signal amplitude to the required range

Appendix G: Sine Wave Generator

1 Fourier series concepts were used to understand how a sine wave can be derived from a square wave https://www.electronics-tutorials.ws/accircuits/harmonics.html
2 Filtering techniques were applied to remove higher-order harmonics https://www.electronics-tutorials.ws/filter/filter_2.html
   a) Proper filter design was critical for achieving a smooth waveform
3 Amplification and buffering stages were used to maintain waveform integrity

Appendix H: Frequency Measurement

1 Frequency measurement techniques using edge detection were referenced https://www.electronics-tutorials.ws/waveforms/waveforms_4.html
2 The Raspberry Pi GPIO was used to detect signal transitions and calculate frequency
   a) Accurate measurement depended on clean signal edges
3 Timing and resolution limitations were considered when evaluating system performance